

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

Faculté Informatique et Communications

Section Data Science

Projet de master

DOM-Graph RAG Visuel

Tuilage guidé par la structure du document
pour le RAG visuel

Alexis Firome

Courriel : alexis.firome@epfl.ch

Adresse : 27 ter boulevard Diderot, 75012 Paris

Superviseur académique EPFL Pr. Amir Zamir

Laboratoire EPFL **[laboratoire EPFL de rattachement]**

Entreprise d'accueil Devoteam DRI

Encadrement en entreprise Dr. Valentin Noël

Date de soumission 09-09-2026

Déclaration

Je déclare que ce rapport présente le travail que j'ai personnellement réalisé dans le cadre de mon projet de master. Les contributions de tiers, les résultats issus de la littérature et les outils logiciels réutilisés sont explicitement identifiés et référencés. Le code source du système décrit dans ce rapport est disponible en accès ouvert à l'adresse **[URL du dépôt]**, à la révision **[identifiant de commit]** (annexe [D](#)).

Une version antérieure et condensée des travaux décrits a été soumise sous forme d'article court à un atelier associé à une conférence en traitement automatique des langues. Cette soumission est en cours d'évaluation et n'a fait l'objet d'aucune validation par les pairs à ce jour. Le présent rapport en constitue la version longue, complétée du contexte scientifique, du détail d'implémentation et de l'analyse critique des résultats.

Paris, le 09-09-2026

Alexis Firome

signature

Remerciements

Je tiens à remercier le Dr. Valentin Noël pour l'encadrement de ce projet. Le suivi hebdomadaire qu'il a assuré tout au long du semestre m'a permis d'avancer avec des retours réguliers et une écoute attentive, et ses conseils ont souvent débloqué les questions sur lesquelles je butais. Je lui suis également reconnaissant du soutien qu'il m'a apporté dans le choix du sujet, qu'il a encouragé dès le départ et laissé évoluer selon mes intérêts. La bonne humeur de nos échanges a rendu ces mois de travail particulièrement agréables.

Je remercie le Pr. Amir Zamir d'avoir accepté la supervision académique de ce projet de master, ainsi que Devoteam DRI pour l'accueil et pour l'accès aux ressources de calcul qui ont rendu possible la partie expérimentale de ce travail. Les erreurs et les choix discutables qui subsistent dans ce rapport sont les miens.

Table des matières

Introduction	3
Contexte	3
Le problème	4
Contribution personnelle	4
Objectifs et contributions	5
Résultats principaux	6
Organisation du rapport	6
1 État de l’art	7
1.1 Génération augmentée par récupération et découpage	7
1.1.1 Le biais de taille de vivier	8
1.2 Documents visuellement riches et prétraitement	8
1.2.1 Segmentation de pages web	9
1.3 Modèles vision-langage et RAG visuel	9
1.3.1 PixelRAG	9
1.4 RAG structuré par graphe	10
1.4.1 MAGE-RAG	10
1.5 Apprentissage contrastif et adaptation efficace	10
1.5.1 InfoNCE	10
1.5.2 Adaptation efficace d’un modèle génératif en encodeur	11
1.6 Synthèse et positionnement	11
2 Méthodologie	13
2.1 Préliminaires géométriques : le DOM et le moteur de rendu	13
2.1.1 Du balisage à la mise en page	14
2.1.2 Éléments flottants	14
2.1.3 Les deux API de géométrie et le repère absolu	15
2.1.4 Ordre du DOM et ordre visuel	16
2.2 Rendu de page et extraction géométrique	16
2.2.1 Rendu sans interface graphique	16
2.2.2 Retrait du bruit d’interface	17
2.2.3 Extraction des boîtes englobantes	17
2.2.4 Mesure serrée des balises de texte coulant	17
2.3 Tuilage adaptatif guidé par le DOM	18
2.3.1 Élagage des éléments imbriqués	18
2.3.2 Résolution des chevauchements partiels	18
2.3.3 Règles de tuilage	19
2.3.4 Modèle de coût : tokenisation visuelle et effet du tuilage	19
2.3.5 Découpage en sections	21
2.4 Construction du graphe multigranulaire	22
2.4.1 Ordre de lecture spatial	22
2.4.2 Voisinage de mise en page	23
2.4.3 Arbre des sections	23
2.4.4 Voisinage sémantique	24
2.4.5 Relations inter-documents	24

2.4.6	Graphe de corpus multi-pages	24
2.4.7	Rendu structuré pour le lecteur	25
2.5	Entraînement contrastif du lecteur	25
2.5.1	Génération synthétique de paires question-tuile	26
2.5.2	Minage de négatifs difficiles	26
2.5.3	Du modèle génératif à l'encodeur	27
2.5.4	Adaptation LoRA	27
2.5.5	Perte contrastive	28
2.5.6	Contraintes mémoire	29
2.5.7	Partitionnement pour l'évaluation	29
2.6	Contrôleur de preuves en ligne	29
2.6.1	Machine à états	30
2.6.2	Politique <i>best-first</i> budgétée	30
2.6.3	Expansion inter-documents	31
2.6.4	Score de pertinence et point d'extension	32
2.6.5	Lecture des tuiles retenues	32
2.7	Implémentation et ingénierie	32
2.7.1	Architecture modulaire	32
2.7.2	Reproductibilité des appels au modèle	33
2.7.3	Sauvegarde par unité de travail	33
2.7.4	Concurrence	33
2.7.5	Tests	33
2.8	Protocole expérimental	34
2.8.1	Construction du jeu de données synthétique	34
2.8.2	Reproduction de la référence en grille	34
2.8.3	Critère d'attachement de la réponse	35
2.8.4	Ablation de résolution	35
2.8.5	Questions multi-sauts	36
3	Résultats et limites	37
3.1	Jeu de données	37
3.2	Effet du tuilage guidé par le DOM	37
3.2.1	Protocole	37
3.2.2	Taux d'attachement	38
3.2.3	Budget de pixels	39
3.2.4	Qualité de classement	40
3.2.5	Pourquoi le texte plat reste compétitif	41
3.3	Effet de l'entraînement contrastif du lecteur	41
3.3.1	Protocole	41
3.3.2	Résultat et analyse	41
3.4	Évaluation C, contrôleur budgété contre sélection top- <i>k</i> statique	42
3.4.1	Protocole	42
3.4.2	Diagnostic, correctif et résultat	43
3.4.3	Analyse	44
3.5	Évaluation D, le système assemblé	44
3.5.1	Protocole et résultat	44
3.5.2	Taxonomie des erreurs, sous-rappel de preuves plutôt que fabulation	45
3.6	Synthèse, coûts et limites	46

Conclusion	49
Rappel	49
Travail établi	49
Verrou identifié	50
Enseignements méthodologiques	51
Perspectives	52
Conclusion	52
A Résultats complémentaires	53
A.1 Contraintes de protocole de l'évaluation A	53
A.2 Coûts de calcul	53
A.3 Limites	54
A.4 Directions de travail	56
B Hyperparamètres	57
C Invites utilisées	61
C.1 Génération de paires question-réponse	61
C.2 Instruction d'encodage PromptEOL	61
C.3 Juge de faux négatifs	61
C.4 Lecture d'une tuile ouverte	62
C.5 Synthèse de la réponse finale	62
D Organisation du dépôt	63
D.1 Modules du pipeline	63
D.2 Scripts d'expérimentation	64
D.3 Tests	64
E Protocole de reproduction	65
E.1 Installation	65
E.2 Vérification de l'installation	65
E.3 Reproduction des évaluations	65
E.4 Application de démonstration	65
E.5 Notes de reproductibilité	66
F Considérations éthiques	67
F.1 Accès aux données	67
F.2 Licences	67
F.3 Statut des données synthétiques	67
F.4 Aptitude au déploiement	67
Bibliographie	69

Introduction

Contexte

Les grands modèles de langue ont une mémoire figée à la date de leur entraînement et une propension bien documentée à produire des affirmations plausibles mais fausses lorsqu'on les interroge sur des faits qu'ils n'ont pas mémorisés. La génération augmentée par récupération [26] répond à ces deux limites par la même mécanique : avant de répondre, le système va chercher dans une base documentaire les passages pertinents pour la question, puis les fournit au modèle génératif comme contexte. Le modèle n'a plus à savoir, il a à lire. Cette architecture est devenue le mode de déploiement dominant des modèles de langue en environnement professionnel, et une littérature abondante en a exploré les variantes [16].

Le maillon faible de cette chaîne est rarement le générateur. C'est le prétraitement documentaire. Un pipeline RAG classique commence par convertir chaque document en texte brut : analyse syntaxique, extraction du flux textuel, reconnaissance optique de caractères sur un document numérisé. Cette conversion est une projection destructive. Un tableau devient une suite de mots dont on ne sait plus quelle cellule appartenait à quelle ligne. Une légende se retrouve orpheline de sa figure. Un encadré latéral s'intercale au milieu d'un paragraphe qu'il ne concernait pas. Cette perte d'information n'est pas marginale sur des documents que l'on qualifie de visuellement riches comme des pages web, des rapports financiers, des notices techniques ou des articles encyclopédiques. La perte d'information porte précisément sur ce que la mise en page encodait.

Deux réponses distinctes coexistent dans la littérature récente, et ce travail ci se situe exactement à leur intersection. La première rend l'extraction inutile : puisque les modèles vision-langage savent lire une image de document, autant leur donner l'image, une page rendue, découpée en imagerie indexées par un encodeur multimodal, la récupération s'opérant directement dans l'espace visuel des pixels [13, 28]. PixelRAG [40] porte cette idée à l'échelle du web et rapporte qu'elle égale ou dépasse la récupération textuelle sur les documents longs et fortement mis en page ; mais elle découpe la page rendue sur une grille de hauteur fixe, indifférente à ce qu'elle traverse. La seconde structure les preuves plutôt que de les traiter comme un sac plat d'unités indépendantes : MAGE-RAG [48] organise un document déjà segmenté en un graphe multigranulaire qu'un contrôleur budgété parcourt à la requête, plutôt que de présenter au modèle lecteur la totalité du document ou une sélection figée.

Ces deux travaux sont complémentaires et laissent la même pièce sur la table. PixelRAG établit que des tuiles de pixels lues par un modèle vision-langage constituent une unité de récupération viable, mais les découpe à l'aveugle. MAGE-RAG établit qu'un contrôleur de graphe budgété surpasse une sélection top- k statique, mais suppose les éléments du document déjà constitués, en amont de son propre pipeline. Ce travail assemble les deux par l'artefact que PixelRAG jette et que MAGE-RAG n'a jamais eu à construire : le DOM de la page rendue.

Segmenter une page web depuis son rendu n'a en soi rien de nouveau, et il faut le dire d'emblée. VIPS [3] le fait depuis vingt ans, et Cai et al. [4] ont montré qu'indexer ces blocs plutôt que la page entière améliore la recherche web. Ce qui est neuf ici est plus étroit : ces frontières servent à décider où découper des pixels destinés à un encodeur visuel, et non à alimenter un index textuel. La distinction avec Pix2Struct [25] précise la même idée sous un autre angle : Pix2Struct apprend la structure du DOM comme cible de supervision, quand ce travail la lit à l'inférence, sans apprentissage.

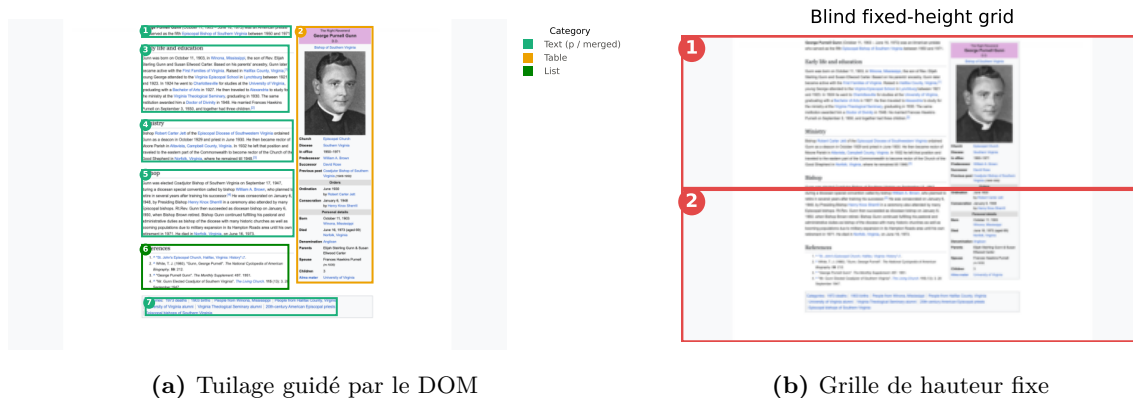


Figure 1 – La même page Wikipédia rendue (en.wikipedia.org/wiki/George_P._Gunn), découpée de deux manières. **(a)** Tuilage guidé par le DOM (section 2.3), chaque tuile suivant la frontière d’un élément. **(b)** Grille aveugle de hauteur fixe, réimplémentée d’après PixelRAG (section 2.8.2).

Le problème

Ce changement de représentation laisse intacte une décision de conception sur la manière dont on doit découper la page. Un modèle vision-langage ne peut pas ingérer une capture d’écran pleine page d’un article encyclopédique de 46 000 pixels de hauteur, et de toute façon une unité de récupération doit être suffisamment fine pour qu’un score de pertinence signifie quelque chose. Il faut donc découper la page en plusieurs captures. PixelRAG découpe sur une grille de hauteur fixe, sans aucune notion de ce que la découpe traverse. Chaque bande a la même hauteur, qu’elle contienne un paragraphe entier, la moitié d’un tableau, ou le bas d’une image et le haut de la section suivante.

La conséquence est mécanique. Une réponse courte noyée dans une bande de 1 024 pixels qui agrège trois sections hétérogènes devient très difficile à retrouver. De ce fait, le signal utile représente quelques mots sur plusieurs centaines. Symétriquement, un tableau coupé au milieu produit deux unités dont aucune ne porte l’information complète. Le système de récupération se voit alors confier une tâche supplémentaire qui n’était pas la sienne : reconstruire, à partir des pixels, les frontières sémantiques du document.

L’observation qui fonde ce travail est que cette reconstruction est en partie inutile pour une page web. Le moteur de mise en page du navigateur a déjà calculé la boîte englobante exacte de chaque élément avant qu’un seul pixel ne soit peint : les frontières géométriques sont donc exactes et gratuites, sans modèle à entraîner ni heuristique de segmentation à calibrer. Ce que le DOM ne donne pas, en revanche, est l’ordre de lecture, la granularité utile d’une unité et le tri entre contenu et bruit d’interface. Ces trois points sont à construire, et c’est ce à quoi la moitié du chapitre 2 est consacrée. La figure 1 montre ce que la seule information gratuite change déjà sur une page réelle.

Contribution personnelle

L’intégralité du système décrit dans ce rapport, avec la conception du tuilage, la construction du graphe, l’entraînement contrastif du lecteur, le contrôleur budgété, ainsi que le code, les jeux de données synthétiques et les évaluations qui les accompagnent ont été conçues et implémentées par l’auteur dans le cadre de ce projet de master. Les deux seuls emprunts à des travaux tiers, explicitement identifiés comme tels tout au long du rapport, sont l’architecture de graphe et de contrôleur de MAGE-RAG [48], dont ce travail généralise le contrôleur à

des frontières inter-documents qu’il ne traite pas, et le protocole de grille de PixelRAG [40], réimplémenté dans un module indépendant, à une autre largeur de rendu, pour servir de condition de référence à la section 3.2. Le code produit est publié en accès ouvert (annexe D).

L’originalité revendiquée porte sur l’assemblage et sur son évaluation, non sur l’idée de lire le DOM d’une page rendue, qui appartient à la littérature de segmentation web depuis VIPS [3] et Cai et al. [4]; la section 1.2.1 situe ce travail par rapport à elle.

Une version antérieure et condensée des chapitres 2 et 3 a été soumise en double aveugle, sous le titre « *DOM-Graph RAG : DOM-Guided Adaptive Tiling and Budgeted Online Evidence Control for Multimodal Web-Page RAG* » [14], à l’atelier *DocInsights 2026*, associé à EMNLP 2026. Le présent rapport en constitue la version longue : il ajoute le contexte scientifique complet (chapitre 1), le détail d’implémentation et d’ingénierie nécessaire à la reproduction, et l’analyse critique des quatre évaluations du chapitre 3, dont la version courte ne pouvait rapporter que les résultats les plus saillants.

Objectifs et contributions

L’objectif de ce projet de master est de transformer l’observation ci-dessus en un système complet et de mesurer ce qu’elle apporte réellement. Cela suppose de construire quatre composants et de les évaluer séparément puis ensemble. Les contributions de ce travail sont les suivantes.

1. **Un tuilage adaptatif guidé par le DOM** (section 2.3) qui remplace la grille de pixels aveugle de PixelRAG : chaque élément est recadré sur sa propre boîte englobante, les éléments trop petits sont fusionnés avec leurs voisins, ceux trop grands sont subdivisés, et les titres sont systématiquement appariés avec leur contenu. Le point technique le plus délicat est la mesure de la boîte englobante des balises de texte coulant, dont la boîte CSS s’étend sur toute la largeur du conteneur même lorsque le texte s’enroule autour d’un élément flottant. C’est cette étape, formalisée en détail au chapitre 2, qui porte la recherche centrale de ce travail : aligner la grille de *patches* que consomme le modèle vision-langage sur les frontières sémantiques du DOM plutôt que sur une géométrie arbitraire, de sorte que chaque unité visuelle soumise à l’encodeur porte un contenu homogène et complet.
2. **Un graphe de preuves multigranulaire** (section 2.4) organisant les tuiles en deux niveaux (page et élément) et instanciant les cinq relations du graphe de MAGE-RAG [48], augmentées de deux relations propres à notre contexte : `links_to`, qui suit les liens hypertextes pour relier plusieurs pages, et `continues`, qui chaîne les sections d’une page trop longue pour être traitée d’un bloc.
3. **Un lecteur vision-langage entraîné par contraste** (section 2.5) sur des paires (question, tuile) générées synthétiquement avec négatifs difficiles minés, adapté par LoRA sur le décodeur et la tour de vision, via une tête d’encodage de type PromptEOL [20].
4. **Un contrôleur de preuves en ligne budgété** (section 2.6) qui fait évoluer l’état de chaque nœud du graphe selon une politique best-first sous budget, généralisant l’algorithme de contrôle de MAGE-RAG à des frontières qui franchissent les limites de document.

À ces contributions logicielles s’ajoute une contribution expérimentale : **quatre évaluations** (chapitre 3) qui isolent chaque composant puis mesurent le système assemblé, avec une attention particulière portée à la comparabilité des métriques. Nous montrons notamment que le Recall@ k brut n’est pas comparable entre conditions de tuilage, puisqu’un vivier de

candidats plus petit le gonfle mécaniquement, et nous utilisons à la place un rang percentile invariant à la taille du vivier. Cette correction vaut en espérance sous classement aléatoire et ne dit rien sous budget absolu ; or le contrôleur ouvre un nombre fixe de nœuds, et en rang absolu la grille devance le tuilage DOM (tableau 3.2). Les deux lectures sont rapportées.

Résultats principaux

Deux résultats sont nets. Le tuilage guidé par le DOM double presque la proportion de questions dont la réponse survit dans une unité récupérable (51,8 % contre 26,4 % pour une grille aveugle pleine largeur), tout en consommant 2,7 fois moins de pixels par page ; ces deux effets vont dans le même sens, ce qui est contre-intuitif puisque notre tuilage produit cinq fois plus de tuiles. Le second de ces chiffres est mesuré à une largeur de rendu de 2048 px et sous un plafond de tokens relevé douze fois au-dessus du défaut de la bibliothèque, deux constantes dont il dépend (section 3.2.3). L'entraînement contrastif du lecteur fait presque doubler le Recall@1 sur un partitionnement par article strictement disjoint (0,433 → 0,785), sur des questions synthétiques issues de la même génération et sur une partie de validation qui ne compte qu'une poignée d'articles.

Les composants en aval donnent des résultats plus nuancés, que nous analysons en détail plutôt que de les minimiser. L'avantage de classement du tuilage DOM sur la grille est réel mais modeste, et il s'inverse face à un découpage textuel plat sur le sous-ensemble strictement apparié : nous montrons que ce résultat est cohérent avec le scoreur lexical employé, qui n'exploite aucune information spatiale, et non avec un défaut du tuilage lui-même. Le contrôleur budgété n'établit pas encore de supériorité sur une sélection top- k statique ; nous en diagnostiquons une cause architecturale identifiable, la corrigeons, mesurons le gain, et documentons ce qui reste à faire. Enfin, l'évaluation bout-en-bout montre que le système assemblé échoue par abstention plutôt que par fabulation, ce qui désigne le sous-rappel de preuves comme le verrou restant et pointe vers un correctif précis et déjà instrumenté dans le code.

Organisation du rapport

Le rapport suit une progression en trois chapitres.

Le **chapitre 1** situe brièvement DOM-Graph RAG dans son contexte scientifique avec la formulation du problème de récupération et de ses métriques, le prétraitement des documents visuellement riches, les modèles vision-langage et la famille du RAG visuel, le RAG structuré par graphe, et les techniques d'apprentissage contrastif mobilisées ensuite. Il reste volontairement concis et se clôt sur l'identification précise du verrou et le positionnement de ce travail face à PixelRAG et MAGE-RAG.

Le **chapitre 2** constitue le cœur de ce rapport. Il ouvre sur ce que le navigateur expose réellement de la géométrie d'une page rendue puis suit l'ordre du pipeline : rendu et extraction géométrique, tuilage adaptatif formalisé en détail, construction du graphe, entraînement contrastif du lecteur et contrôleur en ligne. Chaque étape est explicitée par ses choix de conception, sa formalisation mathématique, les cas limites rencontrés sur des pages réelles, et les décisions d'ingénierie qui rendent le pipeline exécutable sur du matériel modeste.

Le **chapitre 3** présente les quatre évaluations et leur analyse critique, y compris les deux dont le résultat ne va pas dans le sens attendu.

Un chapitre de conclusion synthétise l'apport et les enseignements du projet, et les annexes rassemblent les contraintes de protocole, les coûts de calcul observés, les limites du travail, les hyperparamètres, l'organisation du dépôt, les invites utilisées et les protocoles de reproduction.

Chapitre 1

État de l'art

Ce chapitre situe DOM-Graph RAG dans son contexte scientifique et se limite à ce qui est strictement nécessaire pour situer les choix de conception : le problème du découpage documentaire et ses métriques (section 1.1), le prétraitement des documents visuellement riches (section 1.2), la famille du RAG visuel et son représentant le plus proche, PixelRAG (section 1.3), le RAG structuré par graphe et son ancêtre direct, MAGE-RAG (section 1.4), et l'apprentissage contrastif mobilisé par le lecteur (section 1.5). Il se clôt sur le positionnement précis de ce travail (section 1.6).

1.1 Génération augmentée par récupération et découpage

La figure 1.1 donne les trois opérateurs qu'enchaîne un système RAG. Le découpeur $C : \mathcal{D} \mapsto \mathcal{U}$ transforme une collection de documents en unités récupérables. Le récupérateur $R : q \mapsto \mathcal{U}_k \subseteq \mathcal{U}$ sélectionne les k unités les plus pertinentes pour une question q selon un score $s(q, u)$. Le générateur $G : (q, \mathcal{U}_k) \mapsto a$ répond à partir de la question et de ces unités.

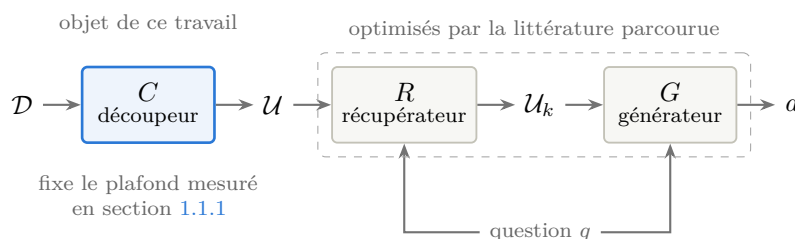


Figure 1.1 – Les trois opérateurs d'un système RAG. La composition impose un ordre de dépendance, car R ne peut sélectionner que des unités produites par C , et G ne peut répondre qu'à partir de ce que R transmet.

Lewis et al. [26] ont formalisé cette architecture et montré qu'affiner conjointement l'encodeur de requête et le générateur améliore les tâches à forte intensité de connaissances ; Izacard and Grave [19] ont montré qu'encoder chaque passage séparément avant de les fusionner dans le décodeur permet d'en traiter un grand nombre sans coût quadratique. Gao et al. [16] recensent ces variantes sans qu'aucune ne s'impose.

Le score $s(q, u)$ oppose deux familles. Les approches **lexicales** (TF-IDF, BM25 [34]) ne demandent aucun entraînement et restent compétitives : le *benchmark* BEIR [35] établit qu'en transfert zéro-coup BM25 n'est pas systématiquement dépassé par les récupérateurs denses, et la section 3.3 retrouve ce constat. Leur limite est le décalage de vocabulaire. Les approches **denses** projettent question et unité dans un espace vectoriel commun appris par contraste [15, 33] ; Karpukhin et al. [22] en tirent la recette de référence pour la récupération de passages, et ColBERT [23] en conserve un vecteur par token plutôt qu'un vecteur unique, une idée qui réapparaît dans le RAG visuel (section 1.3).

Cette littérature suppose C donné, alors qu'il fixe un plafond que ni R ni G ne peuvent franchir. Si la réponse à q est répartie sur deux unités distinctes, ou noyée dans une unité majoritairement hors sujet, aucun score ne peut compenser. Les stratégies de découpage textuel, fenêtres de taille fixe, découpage guidé par la structure du document ou découpage

sémantique aux points de rupture de similarité entre plongements [30], sont recensées par Gao et al. [16] sans que leur effet sur ce plafond soit mesuré. Toutes opèrent sur du texte déjà extrait, où les frontières de mise en page ont été détruites, alors qu’un moteur de rendu les calcule sans coût additionnel sur le web (section 2.1). Toutes mesurent aussi la qualité du classement en supposant l’attachement de la réponse acquis, ce que ne fait ni la section 1.1.1 ni la section 3.2.2.

1.1.1 Le biais de taille de vivier

Comparer deux stratégies de découpage produisant des viviers de candidats de tailles différentes pose un piège méthodologique peu discuté : sur une même page en 5 bandes contre 25 tuiles, un classement aléatoire place la bonne unité au premier rang avec une probabilité de 20 % dans le premier cas et de 4 % dans le second, de sorte que comparer des Recall@1 bruts revient en partie à comparer des tailles de vivier. Le rang percentile

$$\text{pct}(u^+) = 1 - \frac{\text{rang}(u^+) - 1}{n_{\text{items}} - 1}, \quad (1.1)$$

vaut 0,5 en espérance sous classement aléatoire quelle que soit la taille du vivier, et constitue la métrique de comparaison entre conditions de tuilage employée en section 3.2, aux côtés du taux d’attachement qui représente la proportion de questions dont la réponse survit au découpage, absente des protocoles usuels alors qu’elle détermine le plafond que ni R ni G ne peuvent franchir. Ce taux dépend entièrement du critère qui décide qu’une réponse a survécu ; ce critère est explicité et discuté en section 2.8.3, car il n’est pas neutre entre les conditions comparées.

1.2 Documents visuellement riches et prétraitement

Un document est dit visuellement riche lorsque sa mise en page porte de l’information sémantique, comme un tableau, une facture ou une page à encadrés. Pour un tel document, la chaîne de prétraitement classique enchaîne trois étapes à perte, détaillées par la figure 1.2. Le résultat cumulé est qu’une question du type « quelle est la valeur de la colonne revenus pour la ligne 2024 » devient difficile à satisfaire, non parce que le récupérateur est mauvais, mais parce que la correspondance ligne-colonne a été détruite en amont.

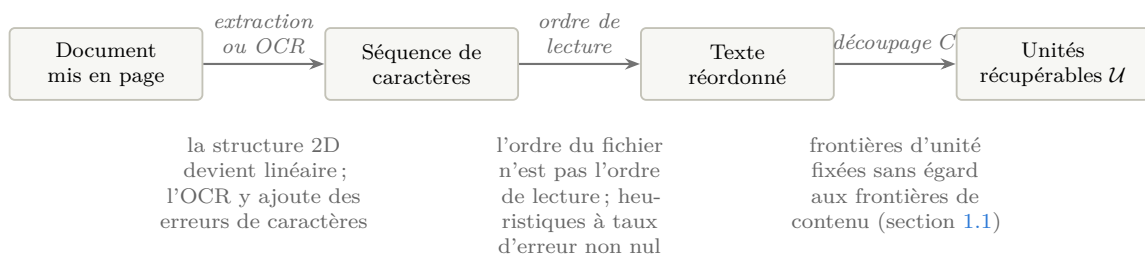


Figure 1.2 – Les trois étapes à perte de la chaîne de prétraitement classique et l’information détruite à chacune. Les modèles sensibles à la mise en page réinjectent la géométrie en aval de la première étape ; les modèles sans OCR suppriment les deux premières.

Une première réponse injecte la géométrie dans la représentation. LayoutLM [43] ajoute aux plongements de tokens des positions 2D dérivées de la boîte englobante de chaque mot ; LayoutLMv2 [42] et LayoutLMv3 [18] l’intègrent au pré-entraînement (92,1 de F_1 sur FUNSD, 83,4 d’ANLS sur DocVQA) ; MarkupLM [27] transpose l’idée au web en encodant le chemin XPath de chaque token. Ces modèles reçoivent tous des mots et boîtes déjà produits

par un OCR en amont, et la structure y est *apprise*, donc coûteuse à ré-entraîner sur un type de document non couvert ; surtout, elle sert la représentation d’une unité déjà découpée, non la décision de découpage : MarkupLM sait que deux tokens partagent un ancêtre DOM, il ne décide pas où couper la page.

Une seconde réponse supprime l’OCR : Donut [24] produit du texte structuré directement depuis l’image d’un document, et Pix2Struct [25] porte l’idée au web en pré-entraînant sur 80 millions de captures d’écran, avec pour cible un HTML simplifié dérivé du DOM rendu. Cela permet ainsi d’établir que le DOM porte la structure dont un modèle visuel a besoin, mais il l’exploite comme cible de supervision apprise, quand ce travail l’emploie comme décision de découpage à l’inférence, sans apprentissage.

1.2.1 Segmentation de pages web

Découper une page web à partir de son rendu n’est pas une idée neuve. VIPS [3] parcourt le DOM rendu et combine balises, séparateurs visuels et attributs de style pour produire une hiérarchie de blocs, et Cai et al. [4] établissent qu’indexer ces blocs plutôt que la page entière améliore la recherche web. WebSRC [6] prolonge cette ligne avec un jeu de données de compréhension structurelle dont les réponses sont ancrées dans des nœuds du DOM. Que le DOM rendu porte des frontières exploitables est donc acquis depuis vingt ans, et ce n’est pas ce point que ce travail revendique.

Trois différences délimitent ce qu’il ajoute. Les blocs de VIPS sont produits par une heuristique paramétrée par un degré de cohérence à calibrer par type de site, alors que le tuilage du chapitre 2 lit les boîtes que le moteur de rendu a déjà calculées et n’introduit aucun paramètre de segmentation appris. L’unité produite n’est pas non plus de même nature, car ces travaux alimentent un index textuel quand celui-ci alimente un encodeur visuel, ce qui fait de la surface en pixels une ressource à budgéter (section 2.3.4). La question évaluée diffère enfin, puisque la recherche par blocs mesure un gain de classement là où le chapitre 3 mesure d’abord si la réponse survit au découpage. La contribution défendue ici est donc étroite, à savoir employer ces frontières pour décider où découper des pixels destinés à un encodeur visuel.

1.3 Modèles vision-langage et RAG visuel

Les modèles vision-langage contemporains combinent une tour de vision de type *Vision Transformer* [11], qui découpe l’image en *patches* traités par auto-attention [38], et un modèle de langue décodeur consommant une séquence mixte de tokens visuels et textuels, sur la base des représentations image-texte alignées par contraste [31]. Qwen2-VL [39], employé au chapitre 2, apporte le support de la résolution dynamique : le nombre de tokens visuels y est proportionnel à l’aire réelle de l’image, une propriété dont la section 2.3.4 tire les conséquences pour ce travail.

Si un tel modèle produit un plongement d’image aligné sur le langage, on peut indexer des images de pages entières et récupérer dessus : *Document Screenshot Embedding* [28] le fait sans extraction de contenu préalable, et ColPali [13] porte le mécanisme d’interaction tardive de ColBERT aux *patches* visuels.

1.3.1 PixelRAG

Le travail le plus directement relié est PixelRAG [40], qui applique cette approche à l’échelle du web : les pages sont rendues, découpées en bandes et indexées avec un encodeur vision-langage, et les auteurs rapportent égalier ou dépasser la récupération textuelle sur plusieurs bancs d’essai, avec une réduction du coût en tokens allant jusqu’à un facteur trois

par sous-échantillonnage de la résolution. C’est son découpage qui nous intéresse : une grille de hauteur fixe, la page rendue tranchée en bandes de $875 \times 1\,024$ pixels indépendamment de leur contenu. La décision est simple et sans dépendance au type de document, mais son défaut est celui de la fenêtre de taille fixe qui crée une bande coupant là où elle tombe, au milieu d’un tableau comme d’un paragraphe. La condition mesurée au chapitre 3 n’est pas PixelRAG lui-même mais une grille aveugle réimplémentée d’après sa description (section 2.8.2), à une largeur de rendu différente dont la section 2.3.4 donne l’effet.

1.4 RAG structuré par graphe

Traiter $\mathcal{U}$ comme un sac plat empêche de récupérer une unité peu pertinente lexicalement mais indispensable contextuellement, et interdit le raisonnement multi-sauts. Deux issues coexistent : itérer la récupération ou structurer les preuves en amont. La première voie est celle de ReAct [45], IRCOT [36], FLARE [21] et Self-RAG [1], qui entrelacent raisonnement et appels de récupération, au prix d’appels au modèle en série. Dans la seconde, Edge et al. [12] construisent un graphe de connaissances sur des entités extraites par un modèle de langue, ce qui répond à des questions globales sur un corpus qu’aucune récupération locale ne satisferait ; le graphe porte sur des entités, non sur la structure documentaire.

1.4.1 MAGE-RAG

Zuo et al. [48] construisent au contraire un graphe de la *structure* du document, et c’est l’ancêtre direct de l’architecture proposée ici : des nœuds de niveau page et de niveau élément, reliés par cinq relations (`contains`, `reading_order`, `layout_adjacency`, `section_hierarchy`, `semantic_neighbor`), que parcourt à la requête un contrôleur budgété décidant quels nœuds le modèle lecteur ouvre réellement. La section 2.6 en généralise l’algorithme. La différence tient à la provenance du graphe : MAGE-RAG le construit hors ligne, à partir d’un document *déjà* segmenté et d’un ordre de lecture déjà déterminé, quand celui construit ici l’est depuis le DOM d’une page rendue, qui fournit sans coût additionnel frontières exactes et types d’éléments (section 2.1). Deux relations que le contexte web rend nécessaires s’y ajoutent, `links_to` et `continues` (tableau 2.2). MAGE-RAG est évalué sur LongDocURL et MMLongBenchDoc, deux jeux de données constitués de PDF déjà analysés ; ces chiffres sont rapportés au chapitre 3 comme référence de l’architecture, non comme comparaison numérique, faute d’un DOM à exploiter sur un PDF.

1.5 Apprentissage contrastif et adaptation efficace

1.5.1 InfoNCE

Entraîner un récupérateur revient à apprendre une similarité $s(q, u)$ qui place la bonne unité plus près de la question que les mauvaises. L’objectif standard, InfoNCE [37], largement repris en vision [5], s’écrit pour une question de plongement $\mathbf{z}_q$, une unité positive u^+ et des négatifs $\{u_1^-, \dots, u_K^-\}$:

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_q, \mathbf{z}_{u^+})/\tau)}{\exp(\text{sim}(\mathbf{z}_q, \mathbf{z}_{u^+})/\tau) + \sum_{i=1}^K \exp(\text{sim}(\mathbf{z}_q, \mathbf{z}_{u_i^-})/\tau)}, \quad (1.2)$$

où τ est une température qui concentre le gradient sur les négatifs proches du positif à mesure qu’elle diminue. Tout dépend des négatifs : les *in-batch negatives* [22] réutilisent gratuitement les positifs des autres exemples du lot, tandis que les négatifs difficiles sont minés délibérément. La section 2.5 reprend cette équation, l’étend au lot entier, et l’analyse

plus en détail. La génération synthétique de paires question-passage, établie par InPars [2] et Promptagator [7], est transposée ici à des tuiles d'image.

1.5.2 Adaptation efficace d'un modèle génératif en encodeur

Affiner intégralement un modèle de plusieurs milliards de paramètres est hors de portée d'une infrastructure académique modeste. LoRA [17] gèle les poids pré-entraînés et n'apprend, pour chaque matrice ciblée, qu'une mise à jour de rang faible ; QLoRA [10] y ajoute la quantification du modèle gelé. Dans le cas multimodal, LoRA peut cibler le décodeur seul ou aussi la tour de vision, puisque la tâche traitée à la section 2.5.4 exige les deux. Un second obstacle est que ces modèles sont génératifs et n'exposent aucun vecteur de représentation ; PromptEOL [20] fait suivre l'entrée d'une amorce à compléter et prend l'état caché du dernier token comme plongement, un procédé appliqué à l'identique à une question textuelle et à une image de tuile à la section 2.5.3.

1.6 Synthèse et positionnement

Quatre constats délimitent une lacune précise, que le tableau 1.1 résume en classant les travaux les plus proches selon l'origine des frontières de leurs unités récupérables.

Tableau 1.1 – Origine des frontières de l'unité récupérable chez les travaux les plus proches, et nature de l'unité produite. Les frontières sont soit arbitraires, fenêtre ou grille, soit héritées d'un découpage effectué ailleurs, soit lues dans le DOM rendu.

Travail	Origine des frontières	Unité
<i>Récupération textuelle</i>		
DPR [22]	fenêtre fixe	texte
GraphRAG [12]	fenêtre fixe	texte
<i>Segmentation de pages web</i>		
VIPS [3]	DOM rendu, heuristique calibrée	texte
Cai et al. [4]	DOM rendu, heuristique calibrée	texte
<i>Modèles sensibles à la mise en page</i>		
LayoutLM [43]	corpus segmenté en amont	texte
MarkupLM [27]	corpus segmenté en amont	texte
<i>RAG visuel, lecture des pixels</i>		
DSE [28]	format du fichier source, la page	pixels
ColPali [13]	format du fichier source, la page	pixels
PixelRAG [40]	grille de hauteur fixe	pixels
<i>RAG par graphe de structure</i>		
MAGE-RAG [48]	corpus segmenté en amont	texte
Ce travail, pixels et graphe	DOM rendu, sans calibrage	pixels

Premièrement, le RAG visuel démontre que lire les pixels préserve une information que l'extraction textuelle détruit, mais ne traite pas le découpage, puisqu'il l'évite en travaillant à l'échelle de la page ou le tranche à l'aveugle, comme PixelRAG. Deuxièmement, la segmentation de pages web lit bien le DOM rendu, mais pour alimenter un index textuel et au moyen d'une heuristique à calibrer, non pour décider où recadrer des pixels. Troisièmement, les modèles sensibles à la mise en page exploitent la structure, mais en l'apprenant à grands frais et pour représenter des unités déjà découpées. Enfin, MAGE-RAG démontre qu'un contrôleur de graphe budgété surpasse une sélection top- k statique, mais suppose les éléments

du document déjà constitués en amont de son propre pipeline, c'est-à-dire l'artefact que PixelRAG jette au rendu et que ce travail restitue.

La thèse défendue est que, pour une page web sémantiquement balisée, la segmentation dont le RAG visuel a besoin est déjà disponible et exacte dans le DOM de la page rendue, et qu'il est donc inutile de la reconstruire depuis les pixels ou de la contourner par une grille. Cette thèse n'est éprouvée ici que sur des articles Wikipédia, le cas le plus favorable qui soit, avec un balisage régulier, des infoboîtes standardisées et une colonne unique ; ce que sa portée laisse ouvert est repris en annexe [A.3](#). Le chapitre [2](#) formalise et met à l'épreuve cette thèse, en commençant précisément là où PixelRAG s'arrête : par ce qu'un navigateur expose de la géométrie d'une page rendue.

Chapitre 2

Méthodologie

Ce chapitre décrit le système construit pour mettre à l'épreuve la thèse formulée au chapitre précédent, et en détaille la formalisation mathématique. Il commence par établir ce qu'un navigateur expose réellement de la géométrie d'une page rendue (section 2.1) représentant le préalable technique dont dépendent tous les choix qui suivent, puis suit l'ordre de la chaîne de traitement : une page est rendue une fois par un navigateur sans interface graphique et sa géométrie est mesurée (section 2.2) ; cette géométrie pilote un tuilage adaptatif, dont la section 2.3 formalise les règles et le modèle de coût (section 2.3) ; les tuiles sont organisées en un graphe à deux niveaux, éventuellement étendu à plusieurs pages liées (section 2.4) ; un lecteur vision-langage est entraîné par contraste, hors ligne, à évaluer la pertinence d'une paire (question, tuile) (section 2.5) ; enfin, à la requête, un contrôleur en ligne parcourt le graphe sous budget pour décider quels nœuds le lecteur ouvre réellement (section 2.6). La section 2.7 détaille les choix d'ingénierie qui rendent l'ensemble exécutable, et la section 2.8 la construction des jeux de données évalués au chapitre 3.

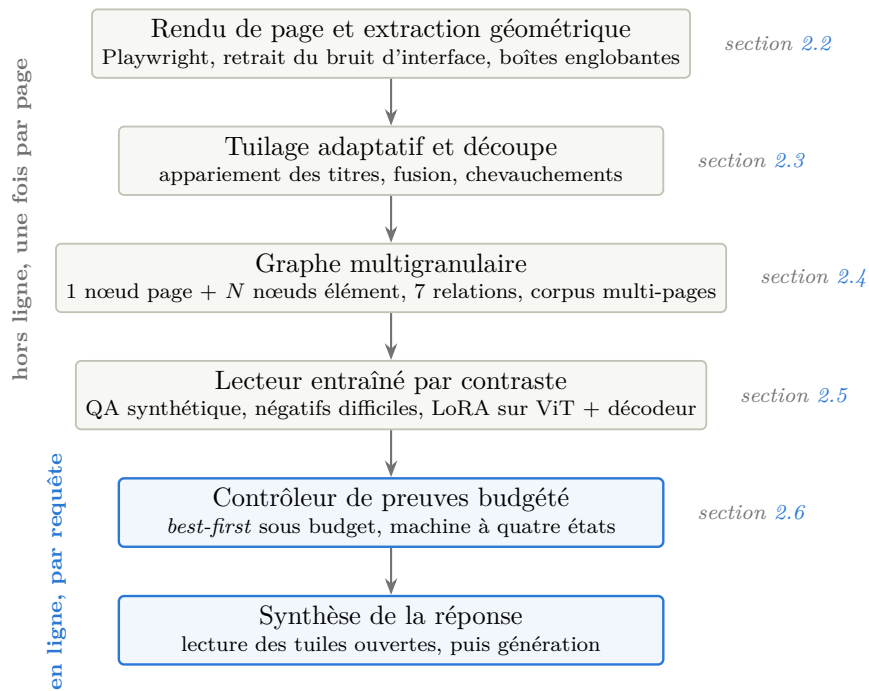


Figure 2.1 – Vue d'ensemble de la chaîne DOM-Graph RAG

2.1 Préliminaires géométriques : le DOM et le moteur de rendu

L'hypothèse centrale de ce travail est que la segmentation nécessaire au RAG visuel est déjà présente dans le DOM d'une page rendue. Cette section établit quelle information le navigateur expose, où se situent les pièges, et pose la notation reprise par le reste du chapitre.

2.1.1 Du balisage à la mise en page

Le traitement d'une page par un navigateur enchaîne quatre étapes. L'analyse du HTML produit le DOM, un arbre $T = (V, E)$ dont chaque nœud $v \in V$ correspond à un élément ou à un nœud texte. La cascade attribue ensuite à chaque élément ses valeurs de propriétés calculées. La mise en forme (*reflow*) détermine la position et les dimensions exactes de chaque boîte dans un repère absolu. La peinture produit enfin les pixels.

La mise en forme se termine intégralement avant que le premier pixel ne soit peint, si bien que la géométrie de tout élément est déjà connue au moment de la peinture. Un système qui recadre des régions dans l'image peinte travaille donc en aval d'une source de vérité géométrique qu'il n'a jamais consultée.

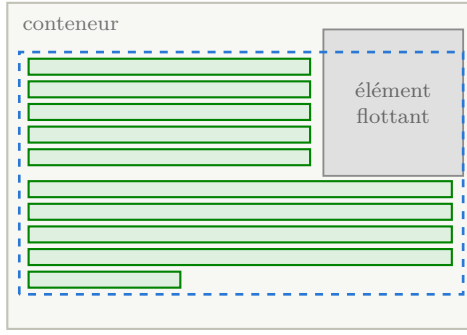
La fonction $\mathcal{B} : V \rightarrow \mathbb{R}_{\geq 0}^4$ associe à un élément rendu sa boîte $\mathcal{B}(v) = (x_v, y_v, w_v, h_v)$ dans le repère absolu du document, où (x_v, y_v) est le coin supérieur gauche et (w_v, h_v) la largeur et la hauteur. C'est elle qu'exploite le tuilage de la section 2.3, aucune partie de la chaîne ne redérivant une géométrie que le navigateur a déjà calculée.

Une boîte est faite de quatre régions concentriques, contenu, *padding*, bordure et marge. Les API de géométrie rapportent par défaut celle qui correspond visuellement à l'élément, et $\mathcal{B}$ désigne implicitement cette région.

Le type d'affichage détermine le comportement de $\mathcal{B}$. Un élément de type **bloc** (`<p>`, `<div>`, `<h1>`–`<h6>`, `<ul>`) a par défaut `width: auto`, donc $w_v = w_{\text{parent}(v)}$ quel que soit son contenu. Un titre de trois mots dans une colonne de 1 200 px a ainsi une boîte de 1 200 px de large. Un élément de type **en ligne** (`<span>`, `<a>`, `<em>`) ne forme pas une boîte unique mais autant de rectangles que de fragments de ligne. Un `<table>`, enfin, s'ajuste à son contenu au lieu de remplir son conteneur. La difficulté décrite ci-dessous ne frappe donc que les blocs de texte.

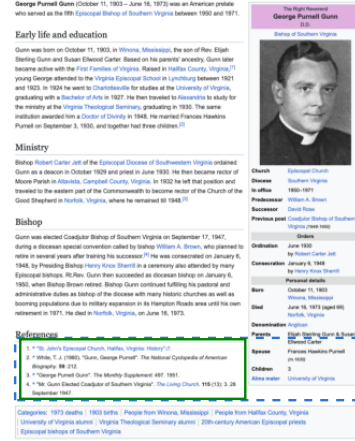
2.1.2 Éléments flottants

Le cas déterminant pour la mesure des tuiles est celui des éléments flottants. Un élément déclaré `float: right` est retiré du flux normal et poussé sur le bord droit de son conteneur, et le contenu en ligne des blocs qui le suivent s'enroule autour de lui. L'effet est asymétrique. Pour un bloc v qui suit un flottant, $\mathcal{B}(v)$ n'est pas réduite et continue de s'étendre sur $w_{\text{parent}(v)}$ en passant sous le flottant, alors que seules les lignes de texte à l'intérieur de v sont raccourcies. La figure 2.2 met les deux géométries en regard.



--- getBoundingClientRect
 ■■■ getClientRects, nœuds texte

(a) Les deux géométries d'un même bloc



(b) La liste de références d'une page réelle

Figure 2.2 – (a) Un élément flottant à droite raccourcit les lignes des blocs qui le suivent sans réduire leur boîte de bloc, laquelle continue de passer sous lui. (b) La boîte de bloc de la liste de références, en tirets, s'étend sous l'infoboîte, alors que l'union de ses rectangles de lignes s'arrête à son bord

La correction employée en section 2.2.4 se formalise ainsi. Soit v un élément de texte coulant et $N_1, \dots, N_k$ ses nœuds texte descendants. `getClientRects`, appliqué à un objet `Range` construit sur chaque N_i , retourne les rectangles de ligne effectivement rendus $\{r_{i,1}, \dots, r_{i,m_i}\}$. Chaque r est un quadruplet $(x^{(0)}, y^{(0)}, x^{(1)}, y^{(1)})$ de coordonnées de coins. La boîte serrée est l'enveloppe rectangulaire de leur union.

$$\mathcal{B}_{\text{serré}}(v) = \left[\min_{i,j} x_{i,j}^{(0)}, \min_{i,j} y_{i,j}^{(0)}, \max_{i,j} x_{i,j}^{(1)} - \min_{i,j} x_{i,j}^{(0)}, \max_{i,j} y_{i,j}^{(1)} - \min_{i,j} y_{i,j}^{(0)} \right]. \quad (2.1)$$

Le `Range` doit être construit sur les nœuds texte et non sur l'élément lui-même. Un `Range` englobant tout v retournerait en plus le rectangle de sa boîte pleine largeur, ce qui ramènerait l'union à $\mathcal{B}(v)$. Un nœud texte n'a par construction aucune boîte propre, et ses rectangles ne peuvent donc être que des lignes réellement rendues.

En l'absence de flottant, $\mathcal{B}_{\text{serré}}(v) = \mathcal{B}(v)$, les lignes remplissant la largeur du bloc. En présence d'un flottant qui intersecte les rangées de v , la boîte serrée est strictement plus étroite, et l'écart d'aire

$$\Delta A(v) = \text{aire}(\mathcal{B}(v)) - \text{aire}(\mathcal{B}_{\text{serré}}(v)) \geq 0 \quad (2.2)$$

mesure le fond blanc qu'un recadrage sur $\mathcal{B}(v)$ soumettrait en pure perte à l'encodeur visuel. C'est ce terme, sommé sur toutes les balises de texte coulant d'une page, que la mesure serrée de la section 2.2.4 élimine, et dont la section 3.2.3 rapporte l'effet agrégé sur le pixels par page.

Pour un `<table>`, un `<figure>` ou un `<img>`, en revanche, bordure, fond et marge intérieure font partie du contenu visuel. La boîte brute $\mathcal{B}(v)$ y reste la mesure correcte, et $\Delta A(v) = 0$ par convention.

2.1.3 Les deux API de géométrie et le repère absolu

Le navigateur expose deux méthodes de mesure. `getBoundingClientRect` retourne $\mathcal{B}(v)$, et `getClientRects` la liste des rectangles effectivement occupés, base de $\mathcal{B}_{\text{serré}}$ ci-dessus. Toutes deux sont relatives au *viewport* et non au document, alors que le recadrage d'une capture

pleine page exige le repère absolu. Le décalage de défilement courant (s_x, s_y) leur est donc ajouté.

$$\mathcal{B}_{\text{abs}}(v) = \mathcal{B}_{\text{viewport}}(v) + (s_x, s_y, 0, 0). \quad (2.3)$$

Cette conversion est appliquée systématiquement à l'extraction (section 2.2). La profondeur d'un nœud dans T fournit par ailleurs un ordre de traitement commode pour résoudre l'imbrication (section 2.3.1), un parent étant toujours à une profondeur strictement inférieure à celle de ses descendants.

2.1.4 Ordre du DOM et ordre visuel

L'ordre des nœuds dans T est celui dans lequel l'auteur a écrit le balisage, et non l'ordre de lecture visuel, le positionnement CSS pouvant faire diverger les deux arbitrairement. Le déduire du parcours de T produirait, sur une mise en page non triviale, une séquence qui ne correspond pas à ce qu'un lecteur humain suit. La relation d'ordre de lecture (section 2.4.1) est donc calculée par une heuristique spatiale sur les coordonnées $\mathcal{B}_{\text{abs}}$ plutôt que par un parcours de T . Le DOM fournit $\mathcal{B}$ et les types d'éléments, pas un ordre de lecture total.

2.2 Rendu de page et extraction géométrique

2.2.1 Rendu sans interface graphique

La page est chargée sans interface graphique, via Playwright, avec une largeur de fenêtre d'affichage de 2048 px et un facteur d'échelle de périphérique de 1. La hauteur de la fenêtre, fixée à 1000 px, est sans effet, la capture pleine page produisant une image de la hauteur totale du document rendu, soit plusieurs dizaines de milliers de pixels sur les articles les plus longs (section 2.3.5).

La largeur de 2048 px égale la hauteur maximale de tuile (section 2.3.3), de sorte que la plus grande unité produite est un carré de 2048 pixels de côté et qu'un seul paramètre borne l'aire maximale soumise à l'encodeur. Elle n'améliore en revanche pas la résolution du texte, la taille en pixels d'un caractère ne dépendant pas de la largeur de fenêtre à facteur d'échelle constant. Élargir le rendu ajoute de la marge, non de la définition.

Cette marge n'est pas neutre pour le chapitre 3, et le choix de 2048 px décide en partie du résultat qui en est tiré. La colonne de contenu occupe environ 40 % de la largeur de rendu (figure 2.3), si bien qu'une bande de grille est majoritairement du fond blanc là où une tuile mesurée serrée (section 2.2.4) ne l'est pas. Le rapport de budget de pixels de la section 3.2.3 est mesuré à cette largeur et décroît avec elle : il vaut 2,7 ici et tomberait à environ 1,16 à la largeur de 875 px employée par PixelRAG, la grille étant la seule des deux conditions à payer la marge. Ce rapport de budget est en réalité l'inverse du taux de couverture des tuiles, et non une propriété du tuilage seul. La courbe du gain en fonction de la largeur de rendu n'a pas été mesurée, et c'est le travail restant le moins coûteux qui consoliderait ce chiffre (annexe A.4). Le taux d'attachement, lui, est insensible à la largeur, une bande plus étroite portant le même texte.

Ordre des opérations. Le chargement attend la condition `networkidle`, et l'extraction géométrique est évaluée après la capture pleine page, la condition d'attente étant réévaluée entre les deux. La capture fait en effet défiler le document et déclenche le chargement différé des images, qui modifie la hauteur des blocs qui les contiennent. Une mesure prise avant ce défilement décrirait une mise en page antérieure au chargement, et les recadrages seraient décalés d'autant sans qu'aucune métrique ne le signale.

2.2.2 Retrait du bruit d'interface

Avant la capture, un ensemble de sélecteurs CSS retire du document le bruit d'interface, c'est-à-dire les barres de navigation, pieds de page, panneaux latéraux du logiciel de wiki, bandeaux d'avertissement, liens d'édition de section et boîtes de navigation. Ces éléments ne portent aucune information sur le sujet de la page et produiraient des tuiles coûteuses en pixels et systématiquement non pertinentes.

La liste combine des sélecteurs génériques (`nav`, `footer`, `header`, `script`, `style`) et des sélecteurs propres à MediaWiki (`#mw-navigation`, `.vector-header`, `.navbox`, `.mw-editsection`, `.noprint`). Cette seconde catégorie est une dépendance explicite au type de site et constitue la principale limite de portée des résultats quantitatifs (annexe A.3). Elle porte sur le nettoyage et non sur la logique de tuilage, laquelle ne connaît aucun sélecteur propre à MediaWiki. Cette séparation reste toutefois une propriété du code et non un résultat, faute d'avoir été éprouvée sur une page hors Wikipédia.

2.2.3 Extraction des boîtes englobantes

L'extraction est un unique appel de fonction JavaScript évalué dans le contexte de la page. Pour un ensemble configurable de balises (`h1` à `h4`, `p`, `table`, `figure`, `img`, `ul`, `ol`, `blockquote`, `pre`), le script relève, pour chaque élément correspondant, sa position absolue au sens de l'équation (2.3), ses dimensions, un aperçu de son texte, sa profondeur dans l'arbre DOM et, le cas échéant, les liens hypertextes qu'il contient.

Extraction sélective des liens. Les liens ne sont relevés que pour les éléments `p`. Une page encyclopédique porte quelques dizaines de liens dans son texte et plusieurs centaines dans ses tableaux, boîtes de navigation et listes de références, dont le suivi ferait croître le coût du parcours de corpus (section 2.4.6) sans apporter de contenu pertinent. Les liens conservés sont résolus en URL absolue via `document.baseURI`, les ancres internes et les schémas non HTTP étant écartés.

2.2.4 Mesure serrée des balises de texte coulant

Recadrer sur $\mathcal{B}$ plutôt que sur $\mathcal{B}_{\text{serré}}$ produit deux défauts mesurables, l'un et l'autre conséquences de l'asymétrie formalisée en section 2.1.2.

Le premier est un double comptage. Recadrer sur $\mathcal{B}(v)$ capture les pixels de l'élément flottant voisin en plus du contenu visé, et ces pixels apparaissent alors dans deux unités de récupération, celle du bloc et celle de l'infoboîte. Le budget est payé deux fois, et les scores de pertinence de deux tuiles sont alimentés par le même contenu.

Le second est un coût. Une tuile de texte recadrée sur $\mathcal{B}(v)$ consomme la même surface qu'une bande de grille quel que soit son contenu réel, soit 2 058 000 pixels en moyenne contre 157 170 pour une tuile recadrée sur $\mathcal{B}_{\text{serré}}(v)$ (section 3.2.3). Le cas dominant est la liste de références en fin d'article, rendue en `<ol>`, dont la boîte traverse toute la largeur du conteneur alors que ses lignes occupent une colonne étroite (figure 2.2b).

Le tuilage recadre donc sur $\mathcal{B}_{\text{serré}}(v)$ pour les balises dont la boîte de bloc est structurellement plus large que leur contenu (`h1` à `h4`, `p`, `blockquote`, `ul`, `ol`), désignées balises de texte coulant et configurables. Un `TreeWalker` parcourt les nœuds texte non vides de v , un `Range` est construit sur le contenu de chacun, et l'équation (2.1) est évaluée sur l'union des rectangles retournés par `getClientRects`. La granularité du `Range` est imposée par la contrainte établie en section 2.1.2. Les autres balises conservent $\mathcal{B}(v)$, avec $\Delta A(v) = 0$.

2.3 Tuilage adaptatif guidé par le DOM

Le tuilage instancie pour le cas web le découpeur $C : \mathcal{D} \mapsto \mathcal{U}$ de la section 1.1. La fonction C_{DOM} définie ci-dessous, construite à partir de $\mathcal{B}$ et $\mathcal{B}_{\text{serré}}$ (section 2.1), remplace la grille de hauteur fixe de PixelRAG. Elle transforme une liste d'éléments géolocalisés E et une capture pleine page I en une liste de tuiles $T = C_{\text{DOM}}(E, I)$, chacune formée d'un recadrage d'image accompagné de ses coordonnées, de la balise DOM d'origine, d'un aperçu textuel, des identifiants des éléments sources et des liens agrégés.

Le nombre de tuiles retenues par unité de page est borné à 25. Cette borne est héritée du coût de génération des paires question-réponse (section 2.8) et restreint la portée de l'évaluation A (annexe A.3), car elle est atteinte sur dix-sept des dix-huit pages évaluées, de sorte que la comparaison porte sur le haut de ces pages plutôt que sur les pages entières. Cette restriction n'est pas neutre, car le haut d'un article encyclopédique est précisément où se trouvent les infoboîtes flottantes, c'est-à-dire le cas pour lequel la mesure serrée de la section 2.2.4 est conçue. Trois traitements préparatoires précèdent le tuilage proprement dit.

2.3.1 Élagage des éléments imbriqués

Les balises retenues s'imbriquent, un `<img>` dans un `<figure>`, un `<p>` dans une cellule de `<table>`, une liste `<ul>` dans un `<blockquote>`. Sans traitement, chaque niveau produirait sa propre tuile et le contenu de l'enfant apparaîtrait deux fois.

L'élagage traite les éléments par profondeur DOM croissante et ne compare chaque candidat qu'aux éléments déjà retenus. Le candidat est écarté si sa boîte est entièrement contenue dans celle d'un élément retenu, à une tolérance d'un pixel près qui absorbe les arrondis de rendu. L'ordre par profondeur croissante garantit qu'un parent est examiné avant ses descendants, donc qu'un enfant n'est jamais comparé à un autre enfant lui-même susceptible d'être élagué.

2.3.2 Résolution des chevauchements partiels

L'élagage ne traite que les inclusions complètes. Deux éléments indépendants peuvent se chevaucher sans que l'un contienne l'autre, même après la mesure serrée de la section 2.2.4. Une infoboîte flottante et le bloc qui partage sa rangée s'intersectent ainsi sur quelques lignes, là où le texte reprend la pleine largeur sous l'élément flottant. Les deux sont conservés, mais les tuiler séparément dupliquerait la région commune. Ils sont donc réunis en un seul recadrage couvrant leur union. Le critère de regroupement est le rapport de l'aire d'intersection à l'aire du plus petit des deux éléments,

$$\rho(a, b) = \frac{\text{aire}(B_a \cap B_b)}{\min(\text{aire}(B_a), \text{aire}(B_b))}, \quad (2.4)$$

deux éléments étant regroupés dès que $\rho(a, b) \geq 0,15$. Le seuil strictement positif absorbe les recouvrements de quelques pixels dus aux marges et aux bordures.

Le regroupement s'effectue par composantes connexes sur cette relation, au moyen d'une structure *union-find*. La transitivité est nécessaire. Si a chevauche b et b chevauche c sans que a et c ne se chevauchent, les trois doivent finir dans le même groupe, faute de quoi la tuile de $\{a, b\}$ et celle de $\{b, c\}$ partageraient les pixels de b .

Les titres sont exclus du regroupement, la règle d'appariement de la section 2.3.3 exigeant qu'ils restent des éléments isolés en attente.

2.3.3 Règles de tuilage

Les groupes sont triés de haut en bas puis de gauche à droite et traités séquentiellement selon trois règles.

Appariement des titres. Un titre n'est pas une tuile autonome. Celle d'un `<h2>` seul ne contiendrait que quelques mots sur un fond blanc, coûteuse en unité de récupération et sémantiquement vide, puisqu'un titre sans son contenu ne répond à aucune question. Un titre rencontré est donc mis en attente, et le groupe suivant est fusionné avec lui dans une tuile unique dont la balise composée (`h2+p`) enregistre la fusion, réutilisée en section 2.4.3. Deux cas font exception et émettent un titre seul, deux titres consécutifs et un titre en fin d'unité de page.

Fusion des éléments courts. Un élément non titre dont la hauteur est inférieure à 80 px est mis en tampon. Le tampon s'accumule jusqu'à ce que l'étendue verticale de son union franchisse ce minimum, puis est vidé en une tuile unique de balise `merged`. Sans cette règle, une succession de courtes lignes de catégories en bas de page produirait une avalanche de micro-tuiles quasi vides, consommant chacune une entrée d'index et un appel de génération. La tuile 7 de la figure 2.3 en donne le résultat. Un groupe issu d'un chevauchement (section 2.3.2) échappe à cette règle et n'est jamais mis en tampon, la non-duplication de pixels primant sur l'heuristique de fusion.

Découpe des éléments démesurés. Un élément dont la hauteur dépasse 2048 px, un très long tableau ou une liste de références étendue, est tranché en $\lceil h/h_{\max} \rceil$ sous-tuiles de hauteur égale. C'est le seul endroit où le tuilage recourt à une découpe indifférente au contenu, à l'intérieur d'un unique élément DOM devenu trop grand pour l'encodeur. Cette découpe est une version strictement plus étroite du mode de défaillance de la grille aveugle, signalée comme telle en annexe A.3.

L'algorithme 1 donne la procédure complète.

`RECADRERDÉCOUPER( $S$ )` recadre I sur l'union des boîtes de S , puis lui applique la découpe ci-dessus si la hauteur obtenue dépasse $h_{\max}$. `VIDERTAMPON` et `VIDERTITRE` émettent respectivement une tuile fusionnée pour le tampon accumulé et, dans les deux cas d'exception ci-dessus, une tuile pour un titre resté sans contenu suivant.

Complexité et comptages. L'élitage compare chaque candidat aux éléments déjà retenus et le regroupement examine toutes les paires, ce qui place les deux étapes en $O(n^2)$ dans le pire cas, où n est le nombre d'éléments extraits. L'unité la plus dense du corpus, une section de l'article consacré aux Beatles, en compte 161 avant élitage. La chaîne des comptages médians par unité de page s'établit à 9 éléments extraits, 7 après élitage, 7 groupes, pour 6 tuiles. Ces deux étapes restent négligeables devant le rendu de page et, à plus forte raison, devant les appels au modèle vision-langage (annexe A.2). Une structure d'indexation spatiale serait l'optimisation naturelle sur des documents d'un ordre de grandeur plus denses.

2.3.4 Modèle de coût : tokenisation visuelle et effet du tuilage

Le tuilage guidé par le DOM produit davantage d'unités que la grille aveugle (section 3.2.3), et rien ne garantit d'avance que cela coûte moins cher à l'encodeur qui les consomme. Le modèle de coût qui suit tranche la question.

Un modèle vision-langage à tour de vision de type *Vision Transformer* [11] découpe une image en *patches* carrés de côté fixe p et les traite comme une séquence de tokens par

Algorithme 1 — Tuilage adaptatif guidé par le DOM

Entrée: éléments E avec boîtes englobantes, capture pleine page I , hauteurs $h_{\max}$ et $h_{\min}$, seuil $\rho_{\min}$

Sortie: liste de tuiles T

```
1  $E \leftarrow \text{ÉLAGUERIMBRIQUÉS}(E)$  // profondeur DOM croissante
2  $G \leftarrow \text{GROUPERCHEVAUUREMENTS}(E, \rho_{\min})$  // union-find
3 trier  $G$  par  $(y, x)$  du premier membre
4  $T \leftarrow \emptyset$ ;  $\text{tampon} \leftarrow \emptyset$ ;  $\text{titre} \leftarrow \perp$ 
5 pour chaque  $g \in G$  faire
6   si  $|g| = 1$  et  $g_1$  est un titre alors
7      $\text{VIDERTAMPON}()$ ;  $\text{VIDERTITRE}()$ 
8      $\text{titre} \leftarrow g_1$  continuer
9   si  $\text{titre} \neq \perp$  alors
10     $T \leftarrow T \cup \text{RECADRERDÉCOUPER}(\{\text{titre}\} \cup g)$ 
11     $\text{titre} \leftarrow \perp$ ; continuer
12   si  $|g| > 1$  alors
13     $\text{VIDERTAMPON}()$ 
14     $T \leftarrow T \cup \text{RECADRERDÉCOUPER}(g)$  continuer
15   si  $\text{hauteur}(g_1) < h_{\min}$  alors
16     $\text{tampon} \leftarrow \text{tampon} \cup \{g_1\}$ 
17    si  $\text{étendue}(\text{tampon}) \geq h_{\min}$  alors
18       $\text{VIDERTAMPON}()$ 
19   sinon
20      $\text{VIDERTAMPON}()$ 
21      $T \leftarrow T \cup \text{RECADRERDÉCOUPER}(\{g_1\})$ 
22  $\text{VIDERTAMPON}()$ ;  $\text{VIDERTITRE}()$ 
23 return  $T$ 
```

auto-attention [38]. Pour une image de dimensions $w \times h$, le nombre de tokens visuels produits est, à un facteur de regroupement près,

$$n_{\text{tokens}}(w, h) \approx \frac{w \cdot h}{p^2} = \frac{\text{aire}(\mathcal{B})}{p^2}, \quad (2.5)$$

soit une quantité proportionnelle à l'aire de la boîte recadrée et indifférente à sa forme. Un recadrage deux fois plus étroit à hauteur égale coûte donc deux fois moins de tokens. Chaque pixel de $\Delta A(v)$ épargné par la mesure serrée est ainsi un token visuel que l'encodeur n'a jamais à calculer, sans perte d'information puisque ce pixel était du fond blanc.

Cette proportionnalité n'est pas illimitée, le processeur d'image bornant l'aire encodée entre un plancher $A_{\min}$ et un plafond $A_{\max}$. Le coût réel suit donc une fonction d'écrtage.

$$n_{\text{tokens}}^{\text{écrté}}(\mathcal{B}) = \frac{\text{clamp}(\text{aire}(\mathcal{B}); A_{\min}, A_{\max})}{p^2}, \quad (2.6)$$
$$\text{clamp}(a; a_{\min}, a_{\max}) = \min(\max(a, a_{\min}), a_{\max})$$

Comparer deux stratégies de découpage sur leur aire totale de pixels ne revient à les comparer sur leur coût en tokens qu'à la condition que leurs unités respectent $A_{\min} \leq \text{aire}(\mathcal{B}) \leq A_{\max}$. C'est une condition à vérifier plutôt qu'une propriété acquise.

Pour une page découpée en tuiles $T = \{t_1, \dots, t_M\}$, le coût total d’encodage vaut $\mathcal{N}(T) = \sum_{i=1}^M n_{\text{tokens}}^{\text{écrêté}}(t_i)$. Deux propriétés de cette somme rendent un découpage plus fin favorable sur cette famille de modèles.

Tant qu’aucune unité n’atteint $A_{\max}$, $\mathcal{N}$ ne dépend que de l’aire totale occupée, et non du nombre d’unités qui la composent. Produire davantage d’unités plus petites ne coûte alors pas plus cher tant que l’aire totale utile ne croît pas.

Tant qu’aucune unité ne descend sous $A_{\min}$, aucune ne paie de coût plancher. Cette seconde propriété tient à la résolution dynamique de Qwen2-VL [39], employé ici. Sous la résolution canonique fixe des générations antérieures, chaque unité payait le même coût quelle que soit sa taille réelle, si bien que le mécanisme exploité ici y serait inopérant.

Pour Qwen2-VL, le côté effectif de *patch* après regroupement vaut 28 px, et le plafond a été porté à $A_{\max} = 12\,845\,056 \text{ px}^2$, douze fois le défaut de la bibliothèque (annexe B). La plus grande unité que la chaîne puisse produire est une tuile carrée de 2048 pixels de côté, soit $4\,194\,304 \text{ px}^2$, et la bande de grille en fait $2\,097\,152 \text{ px}^2$. Aucune n’atteint le plafond, donc aucune n’est écrêtée et la condition ci-dessus est vérifiée. Ce relèvement du plafond n’est pas un détail de configuration, car c’est lui qui épargne l’écrtage à la bande de grille et fait donc passer le gain en tokens de 1,3 à 2,7. Les deux chiffres tiennent au même couple de constantes, la largeur de rendu et le plafond du processeur, et ce couple est rappelé partout où le 2,7 est annoncé. Le tableau 2.1 applique les équations (2.5) et (2.6) aux unités des deux conditions.

Tableau 2.1 – Aire, en pixels carrés, et nombre de tokens visuels des unités produites par les deux conditions de l’évaluation A. Les deux lignes par page donnent des totaux mesurés et non le produit des deux moyennes, lesquelles sont corrélées d’une page à l’autre ; pour la grille, la dernière bande d’une page est en outre plus courte qu’une bande pleine, si bien que l’aire moyenne d’une bande vaut 2 075 510 et non 2 097 152.

Unité	Aire (px)	Tokens visuels	
		sans écrêtage	sous 1,00 Mpx
Tuile DOM, moyenne	157 170	200	200
Bande de grille, 2048×1024	2 097 152	2 675	1 280
Tuile DOM maximale, 2048^2	4 194 304	5 350	1 280
Par page, tuilage DOM (24,0 unités)	3,79 M	4 810	4 810
Par page, grille (4,9 unités)	10,17 M	13 100	6 270
Rapport grille / DOM	2,7	2,7	1,3

Le facteur 2,7 de la section 3.2.3 est donc un rapport de tokens autant que de pixels, mais seulement sous le plafond relevé. La dernière colonne montre ce qu’il devient au plafond par défaut, où la bande de grille est écrêtée quand la tuile DOM ne l’est pas, ce qui ramène le gain à 1,3. Deux constantes de configuration décident ainsi de l’amplitude annoncée, la largeur de rendu pour les pixels (section 2.2) et le plafond du processeur pour les tokens, et toute reproduction qui les changerait obtiendrait un autre chiffre.

2.3.5 Découpage en sections

Une capture pleine page peut atteindre plusieurs dizaines de milliers de pixels de hauteur, 46 000 pour l’article consacré aux Beatles, dont la discographie et la liste de récompenses dominent la page. À cette échelle, la génération séquentielle de questions dépasse plusieurs heures par page, sans point de reprise intermédiaire.

Exclure ces pages serait contraire à l’objectif du projet, qui vise les documents longs. Elles sont donc découpées en amont du tuilage. Une page dont la hauteur dépasse dix fois la

hauteur maximale de tuile, soit 20 480 px, est scindée aux frontières de titres h1 et h2. Le contenu précédant le premier titre forme une section « préambule » sans titre. Une section restant trop haute à elle seule, un h2 suivi d'un tableau de discographie géant, est retranchée par un budget de hauteur fixe de 20 480 px. C'est le compromis de la section 2.3.3 appliqué un cran au-dessus, entre éléments plutôt qu'à l'intérieur d'un seul.

Chaque section est ensuite traitée par le reste de la chaîne comme une page à part entière, avec son propre nœud page et sa propre unité de travail sauvegardée indépendamment. Le seuil est franchi par 12 des 19 articles du corpus, qui produisent 185 unités de page (tableau 3.1). Le découpage est donc le régime courant sur ce corpus, non un filet de sécurité. L'évaluation A le désactive et rend chaque article comme une unité unique, faute de quoi les conditions comparées ne porteraient pas sur la même étendue de document (section 2.8.2).

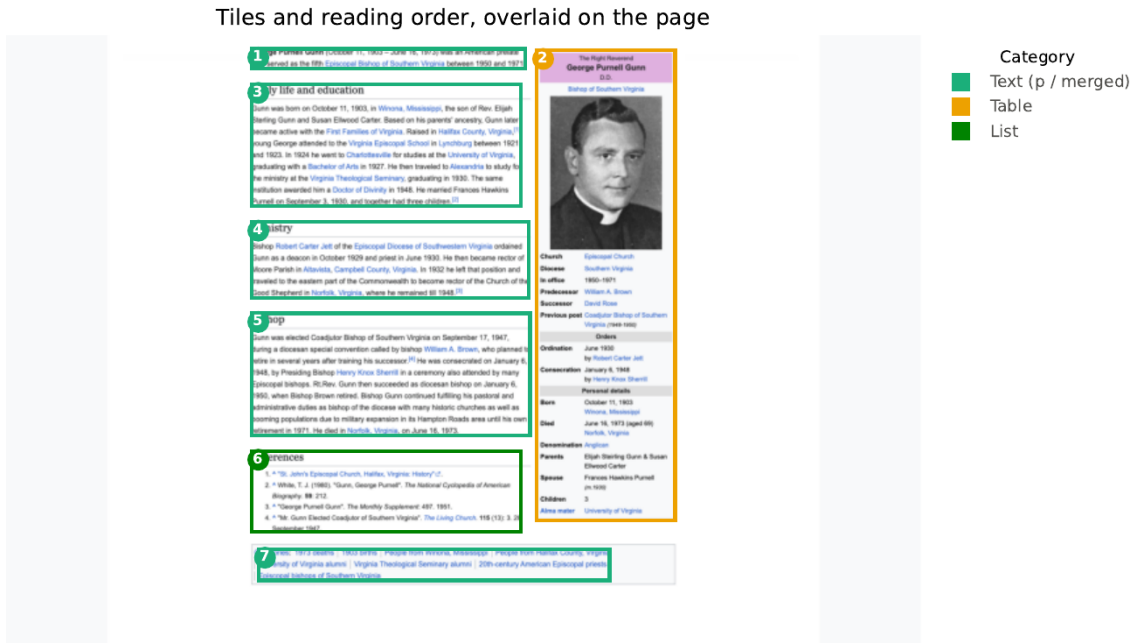


Figure 2.3 – Tuiles produites sur en.wikipedia.org/wiki/George_P._Gunn, colorées par catégorie de contenu et numérotées dans l'ordre de lecture calculé

2.4 Construction du graphe multigranulaire

Les tuiles sont organisées en un graphe orienté multi-arêtes $G = (V, E)$, avec $V = V_{\text{page}} \cup V_{\text{élé}} un nœud **page** et un nœud **élément** par tuile, et $E = \bigcup_{r \in \mathcal{R}} E_r$ l'union de sept relations typées, $E_r \subseteq V \times V$. Cinq d'entre elles, $\mathcal{R}_{\text{MAGE}} \subset \mathcal{R}$, reprennent le graphe de MAGE-RAG [48]. Les deux autres, $\{\text{links_to}, \text{continues}\} = \mathcal{R} \setminus \mathcal{R}_{\text{MAGE}}$, répondent à un besoin propre au contexte web (section 2.4.5). Le tableau 2.2 les récapitule. Le contrôleur ne propage que le long de trois d'entre elles, $\mathcal{R}_{\text{parcourue}} = \{\text{reading_order}, \text{links_to}, \text{continues}\}$, ce que discutent la section 3.6 et l'annexe A.3.$

2.4.1 Ordre de lecture spatial

L'ordre de lecture n'est pas l'ordre du DOM, pour la raison établie en section 2.1.4. Il est calculé par une heuristique spatiale sur les coordonnées mesurées, dans la lignée des travaux d'analyse de mise en page qui traitent l'ordre de lecture comme une prédiction plutôt que comme une lecture de l'arbre [41]. Les tuiles sont d'abord regroupées en rangées. Partant de

Tableau 2.2 – Les sept relations du graphe multigranulaire et leur densité mesurée (*la dernière colonne donne la médiane des arêtes par unité de page*)

Relation	Domaine	Définition	Arêtes
<code>contains</code>	page $\rightarrow$ élém.	rattachement plat de chaque tuile à sa page	6
<code>reading_order</code>	élém. $\rightarrow$ élém.	successeur dans l’ordre de lecture spatial	5
<code>layout_adjacency</code>	élém. $\leftrightarrow$ élém.	voisinage 2D (gauche, droite, dessus, dessous)	8
<code>section_hierarchy</code>	titre $\rightarrow$ contenu	arbre des sections respectant l’imbrication h1–h4	4
<code>semantic_neighbor</code>	élém. $\leftrightarrow$ élém.	proximité lexicale TF-IDF au-delà d’un seuil (<i>désactivée par défaut</i>)	0
<code>links_to</code>	élém. $\rightarrow$ page	lien hypertexte vers un autre document	8
<code>continues</code>	page $\rightarrow$ page	section suivante d’une page découpée	1

la tuile la plus haute non encore assignée, toutes les tuiles dont l’ordonnée est à moins de 20 px de la sienne forment une rangée. Chaque rangée est ensuite triée de gauche à droite, et les rangées sont concaténées de haut en bas.

La tolérance de 20 px est le seul paramètre libre de l’heuristique. Elle absorbe les décalages de ligne de base entre un titre et un élément flottant voisin sans fusionner deux rangées distinctes.

Cet ordre n’est vérifié par aucune mesure. Il n’existe ni vérité terrain annotée, ni taux d’erreur, ni comparaison à Wang et al. [41], alors que `reading_order` est la seule relation que le contrôleur parcourt à l’intérieur d’une page (section 2.6.3) et que tout ce que le graphe apporte au niveau intra-page en dépend. Annoter à la main l’ordre de lecture de quelques pages donnerait ce chiffre à coût faible, et cette vérification manque au travail rapporté ici (annexe A.4).

2.4.2 Voisinage de mise en page

L’ordre de lecture aplatit la structure 2D en une chaîne, ce qui rend deux tuiles côte à côte indiscernables de deux tuiles empilées. La relation `layout_adjacency` restitue ce voisinage en réutilisant le même regroupement en rangées. Elle produit deux types d’arêtes.

- Les voisins gauche et droite, entre tuiles consécutives d’une même rangée.
- Les voisins dessus et dessous, entre une tuile et la tuile de la rangée suivante qui la recouvre horizontalement. En cas de plusieurs candidates, celle dont le centre horizontal est le plus proche est retenue.

Chaque arête est insérée dans les deux sens, car la règle du centre le plus proche appliquée en sens inverse ne désignerait pas nécessairement la même voisine. La symétrie est une décision de construction et pas une propriété de la règle.

Le second type capture les voisinages que l’ordre linéaire fait diverger. Le bas d’une colonne de gauche et le haut de la colonne de droite sont consécutifs dans l’ordre de lecture mais visuellement éloignés, et inversement.

2.4.3 Arbre des sections

La relation `section_hierarchy` construit un arbre de sections, distinct du rattachement plat de `contains`. Un parcours de l’ordre de lecture maintient une pile de titres ouverts, chacun associé à son niveau. À la rencontre d’un titre de niveau ℓ , la pile est dépilée jusqu’à ce que son sommet ait un niveau strictement inférieur à ℓ ; une arête est créée depuis ce sommet vers le nouveau titre, qui est empilé. Une tuile de contenu est rattachée au sommet courant. Un **h3** est ainsi rattaché au dernier **h2** ouvert, non au dernier **h1**, et le contenu précédant le premier titre ne produit aucune arête.

L'appariement de la section 2.3.3 interagit avec ce parcours. Une tuile de balise composée `h2+p` est traitée comme un titre de niveau 2 et comme le contenu de cette même section, qui n'engendre donc pas d'arête vers elle-même. En pratique l'arbre est moins dense qu'avec des titres isolés : la relation n'apparaît qu'entre titres, et vers les tuiles de contenu qui suivent une section déjà ouverte sans porter elles-mêmes de titre. La densité mesurée figure au tableau 2.2 ; l'annexe A.3 discute ce que le contrôleur en tire.

2.4.4 Voisinage sémantique

La cinquième relation de MAGE-RAG relie des éléments proches sémantiquement, indépendamment de leur position. Elle est implémentée par similarité cosinus sur vecteurs TF-IDF entre les aperçus textuels des tuiles, avec un seuil de 0,2.

Deux garde-fous l'encadrent. Une sélection gloutonne borne le degré afin que les paires au-delà du seuil soient triées par similarité décroissante, puis retenues tant qu'aucune des deux tuiles n'a atteint trois voisins sémantiques. Cela évite un graphe excessivement dense sur une page au contenu répétitif comme une discographie ou un tableau de résultats sportifs. Cette relation est désactivée par défaut. Elle devait servir de condition d'ablation, mais cette comparaison n'a pas été conduite (annexe A.3), si bien qu'il s'agit en l'état d'une fonctionnalité écrite et jamais évaluée plutôt que d'une ablation. Toutes les mesures du chapitre 3 portent donc sur des graphes à six relations sur sept, dont le contrôleur n'en parcourt que trois (section 2.6.3) : sept relations décrites, six instanciées, trois parcourues, aucune évaluée séparément. La relation inactive est celle que ce travail hérite de MAGE-RAG. Les deux relations qui lui sont propres sont actives, et ce sont elles qui produisent l'effet mesuré en section 3.4.

2.4.5 Relations inter-documents

Les deux relations ajoutées répondent à un besoin que MAGE-RAG n'a pas, puisqu'il opère sur un document unique déjà délimité.

links_to. Cette relation relie un nœud élément à une page externe, à partir des liens extraits en section 2.2. Un second budget s'y applique, puisque seules les $k_{\text{liens}} = 3$ premières tuiles porteuses de liens, dans l'ordre de lecture de l'unité de page, voient leurs liens conservés. Sur une page non découpée, l'ordre de lecture commence par l'introduction de l'article, et ce budget retient donc les liens qui définissent le sujet. Sur une section issue du découpage de la section 2.3.5, il retient les premiers liens de cette section, sans garantie de saillance comparable. Au-delà, les liens sont ignorés pour borner le facteur de branchement du parcours de corpus.

continues. Cette relation chaîne les sections d'une page découpée dans leur ordre d'origine. Elle est sémantiquement distincte de `links_to`, un lien menant à un autre document quand une continuation mène à la suite du même. Le contrôleur les traite néanmoins symétriquement, puisque dans les deux cas franchir l'arête signifie sortir du périmètre de la page courante.

2.4.6 Graphe de corpus multi-pages

À partir d'une page d'amorce, le système parcourt jusqu'à $m = 3$ pages atteignables par ses arêtes `links_to`. Le parcours est d'un seul saut et non récursif, car suivre les liens de proche en proche ferait croître le nombre de pages exponentiellement, pour un bénéfice incertain sur des questions qui portent rarement à plus d'un saut du sujet (annexe A.3).


```

1 <page url="en.wikipedia.org/wiki/George_P._Gunn" title="George P. Gunn">
2   <element id="tile_0000" tag="merged" state="inactive">...</element>
3   <element id="tile_0003" tag="p" state="opened">
4     Gunn was born on October 11, 1903, in Winona, Mississippi, ...
5   </element>
6   <element id="tile_0004" tag="h2+p" state="pruned">...</element>
7   ...
8 </page>

```

Extrait 2.1 – Rendu XML d’un graphe de page après passage du contrôleur pour la requête « When and where was George Purnell Gunn born? »

Chaque page est traitée indépendamment par la même chaîne, puis les graphes sont fusionnés. Deux mécanismes rendent la fusion possible.

Le premier est le préfixage par espace de noms. Chaque page utilise indépendamment les identifiants `page`, `tile_0000` et suivants. Tous les nœuds d’un sous-graphe sont donc préfixés avant fusion par un identifiant court dérivé de son URL, ce qui garantit l’unicité sans coordination.

Le second est le point d’entrée. Le graphe d’une page expose un nœud `page` canonique comme cible d’un lien entrant, le nœud unique si la page n’a pas été découpée et la première section sinon. Lors de la fusion, chaque nœud `external_page` tenant lieu de substitut est remplacé par le point d’entrée réel. Ce recâblage transforme une collection de graphes de pages en un graphe de corpus navigable, et rend possible l’expansion inter-documents du contrôleur (section 2.6.3).

2.4.7 Rendu structuré pour le lecteur

Un sous-graphe en portée pour une requête peut être sérialisé en XML, dans l’esprit de l’étape de rendu structuré multimodal de MAGE-RAG. Un élément `<page>` porte l’URL et le titre, et contient un élément `<element>` par tuile directement rattachée, avec son identifiant, sa balise DOM, son état courant et son texte. Ce format donne au modèle lecteur une vue structurée de ce qui a été retenu et de ce qui a été écarté, plutôt qu’une concaténation de fragments sans provenance. Il est employé par l’application de démonstration (section 2.7), tandis que les quatre évaluations du chapitre 3 transmettent les preuves sous la forme concaténée de la section 2.6.5.

Les identifiants `tile_MNNN` suivent l’ordre de production du tuilage, qui trie les groupes par position. Il coïncide avec l’ordre de lecture de la section 2.4.1 sauf lorsque deux tuiles d’une même rangée visuelle ont des ordonnées distinctes.

2.5 Entraînement contrastif du lecteur

Un modèle vision-langage générique sait décrire une tuile, mais rien ne le prépare à juger si elle permet de répondre à une question donnée. Il y est entraîné par contraste, en quatre temps. Les données sont d’abord construites, par génération synthétique de paires question-tuile puis minage de négatifs difficiles. Le modèle génératif est ensuite converti en encodeur, puis adapté par LoRA. Une perte InfoNCE est enfin optimisée sur les plongements obtenus.

2.5.1 Génération synthétique de paires question-tuile

Pour chaque tuile retenue, l'image seule est présentée au modèle vision-langage d'inférence (qwen3-v1, annexe B), avec la consigne de formuler une question factuelle et autonome dont la réponse est visible dans cette image et uniquement dans cette image. L'invite exige une question spécifique, portant sur un nombre, un nom, une date, une définition ou une légende, et une réponse impossible à deviner sans avoir vu la tuile. Si la tuile ne contient aucune information exploitable, le modèle répond exactement SKIP. La réponse attendue est un objet JSON strict, ce qui rend l'analyse syntaxique déterministe. L'invite complète figure en annexe C.

C'est la transposition à des tuiles d'image de la recette de génération synthétique de requêtes établie par InPars [2] et Promptagator [7].

Ce que ce protocole met en circulation. Chaque question est écrite à partir d'une tuile DOM, une par tuile, par un modèle de la même famille que celui qui est ensuite affiné. Le jeu d'évaluation est donc produit par la condition même que les évaluations A et B cherchent à valider, et la tuile positive d'une question est exactement l'image dont cette question a été tirée. Deux conséquences en découlent, qui bornent ce que ces deux évaluations établissent. Un découpage différent aurait engendré d'autres questions, si bien que la comparaison de l'évaluation A ne se fait pas à jeu de questions constant. Et ce que l'entraînement contrastif apprend pourrait être de retrouver l'image dont un texte a été tiré plutôt que de récupérer une preuve. Le contrôle qui trancherait est de rejouer ces mesures sur des questions écrites par des humains et indépendantes du découpage, ce que permettent les questions HotpotQA de la section 2.8.5 ; le script existe et n'a pas été exécuté (annexe A.4).

Toutes les tuiles retenues sont sauvegardées sur disque, y compris celles ayant reçu SKIP, car une telle tuile reste un candidat valide comme négatif difficile pour une question portant sur une autre tuile de la même page. Sur le corpus, 67 tuiles sur 1 784 reçoivent SKIP (tableau 3.1).

2.5.2 Minage de négatifs difficiles

Pour chaque question, toutes les autres tuiles de la même page sont classées par similarité cosinus TF-IDF avec la question, et les $K = 2$ meilleures qui franchissent un filtre anti-faux-négatif sont retenues. Le scoreur lexical est choisi pour son coût, puisqu'il ne demande ni GPU ni modèle à charger alors que le minage s'exécute sur des milliers de tuiles. Sa configuration est reportée en annexe B.

Le filtre anti-faux-négatif opère en deux temps, de coût très différent.

Le premier est lexical et gratuit. Tout candidat dont l'aperçu textuel contient déjà la réponse verbatim est rejeté, ce qui attrape les faux négatifs littéraux, par exemple deux blocs mentionnant la même date.

Le second, optionnel, est un juge vision-langage. Le filtre lexical ne détecte que les recouvrements textuels et laisse passer un candidat exprimant la même information par un synonyme, ou visuellement, comme une couleur dans un graphique, un logo ou une valeur lue sur un axe. L'image candidate est alors soumise au modèle avec la question et la réponse attendue. Ce filtre coûte un appel par candidat non écarté en amont. Il est désactivable, et les jeux de données du chapitre 3 sont construits avec le filtre lexical seul.

Le minage conserve une question même lorsque moins de K candidats franchissent le filtre, et l'exigence de K négatifs n'est appliquée qu'au chargement du jeu de données pour l'entraînement. Sur les 1 339 exemples construits, 1 282 portent les deux négatifs attendus, 27 n'en portent qu'un et 30 aucun (tableau 3.1). Ces 57 derniers sont écartés au chargement, après le partitionnement de la section 2.5.7.

Le minage opère au sein d’une même page, les identifiants de tuiles n’étant uniques qu’à cette échelle. L’étendre au corpus exigerait de reprendre en amont le préfixage par espace de noms de la section 2.4.6. Cette extension est mise en œuvre pour le chemin HotpotQA (section 2.8.5), où les positifs proviennent de deux pages et où elle est obligatoire.

2.5.3 Du modèle génératif à l’encodeur

Le modèle affiné, `Qwen2-VL-7B-Instruct` [39], est génératif. Il transforme du texte et des images en texte et n’expose aucun vecteur de représentation, alors qu’une perte contrastive en exige un.

La technique PromptEOL [20] lève cet obstacle. L’entrée est suivie d’une courte instruction fixe, reproduite en annexe C, et l’état caché de la dernière couche à la position du dernier token est pris comme plongement. L’instruction contraint le modèle à comprimer le sens de l’entrée en une position unique.

Le point qui rend la technique adaptée ici est qu’elle est indifférente à la modalité. Le plongement d’une question textuelle et celui d’une image de tuile sont calculés de la même manière, avec la même instruction, seule l’entrée qui la précède changeant. Question et tuile vivent donc par construction dans le même espace, sans tête de projection additionnelle à apprendre. En notant f_θ cette fonction d’encodage, $\mathbf{z}_q = f_\theta(q)$ pour une question et $\mathbf{z}_u = f_\theta(u)$ pour une tuile u , la perte contrastive de la section 2.5.5 porte exclusivement sur θ , à travers les LoRA appliqués aux deux tours (section 2.5.4).

Où se joue la compréhension visuelle. La tour de vision de f_θ reçoit une séquence de *patches* (équation (2.5)). Pour une tuile issue de la grille aveugle, qui chevauche potentiellement plusieurs sections hétérogènes (section 3.2.2), cette séquence mélange les *patches* de plusieurs entités sans qu’aucune limite ne les sépare, et le gradient de l’équation (1.2) doit pousser f_θ à extraire un signal discriminant d’une représentation dont une partie, potentiellement majoritaire, est hors sujet. Pour une tuile issue du tuilage guidé, chaque *patch* appartient par construction à l’unique élément sur lequel porte la question, et le signal à amplifier occupe toute la séquence. Ce paragraphe est une hypothèse : la section 3.3 mesure l’effet de l’entraînement contrastif sur des tuiles alignées, mais aucun entraînement identique n’a été mené sur des tuiles de grille, de sorte que ce que l’alignement DOM apporte à l’entraînement lui-même n’est pas mesuré. C’est l’expérience manquante la moins coûteuse du projet (annexe A.4), et rien de ce qui suit n’en dépend.

Chemin d’exécution et modèle affiné. L’affinage n’emprunte pas le chemin utilisé pour la génération de questions et la lecture des tuiles (sections 2.5.1 et 2.6.5), qui s’adresse à un serveur Ollama n’exposant qu’une API de génération, sans accès aux poids ni aux gradients. Il charge le modèle via `transformers` et `peft`, ce qui exige un accès complet aux poids et un GPU loué. Le modèle affiné n’est donc pas celui de l’inférence, et `Qwen2-VL-7B-Instruct` a été retenu pour la stabilité de sa classe `peft` au moment de ce travail, quand `Qwen2.5-VL` et `Qwen3-VL` l’étaient moins pour l’affinage par LoRA. C’est un choix d’outillage, non une revendication d’équivalence entre les deux modèles. Cette proximité de famille sans identité est reprise en annexe A.3 comme risque de biais à contrôler.

2.5.4 Adaptation LoRA

L’adaptation utilise LoRA [17] avec un rang $r = 16$, un facteur d’échelle $\alpha = 32$ et un *dropout* de 0,05, soit 44 302 336 paramètres entraînaibles sur les sept milliards du modèle de base.

Cibler les deux tours. LoRA est appliqué aux projections d’attention du décodeur (`q_proj`, `k_proj`, `v_proj`, `o_proj`) et à celles de la tour de vision (`qkv`, `proj`). Juger si une tuile répond à une question exige d’affiner simultanément ce qui est vu dans l’image et ce qui est compris de la question. Geler la tour de vision reviendrait à demander au décodeur de compenser seul une représentation visuelle inadaptée à la tâche de récupération. La section 1.5.2 situe ce choix dans les variantes multimodales de LoRA.

Découvrir les modules cibles par introspection. Les noms de modules internes de Qwen2-VL varient d’une version de `transformers` à l’autre. Seuls des suffixes sont donc spécifiés, et le code parcourt le modèle chargé pour découvrir les modules linéaires correspondants. LoRA n’est appliqué qu’aux modules effectivement trouvés, ce qui rend l’adaptation robuste aux changements d’architecture interne mais impose de journaliser la liste obtenue à chaque exécution.

Ce mécanisme a une conséquence qui n’était pas voulue. Le suffixe générique `proj`, pensé pour la tour de vision, correspond par construction à tout nom de module s’y terminant, si bien qu’il capture aussi les projections du réseau à propagation avant du décodeur (`gate_proj`, `up_proj`, `down_proj`) en plus des projections d’attention visées. La portée effective de l’adaptation est donc plus large que celle décrite au paragraphe précédent, et les 44 302 336 paramètres entraînaibles annoncés reflètent cette portée élargie. Les mesures de la section 3.3 portent sur cette configuration, non sur l’attention seule ; corriger le suffixe imposerait de réentraîner et de rejouer l’évaluation B.

2.5.5 Perte contrastive

Soit un lot de $B = 4$ questions, chacune accompagnée d’une tuile positive et de $K = 2$ négatifs difficiles minés. Les plongements des B questions et des $B(1 + K)$ tuiles sont calculés, et la perte minimisée est l’InfoNCE de l’équation (1.2) étendue au lot entier,

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\exp(\text{sim}(\mathbf{z}_{q_i}, \mathbf{z}_{u_i^+})/\tau)}{\sum_{j=1}^B \left[\exp(\text{sim}(\mathbf{z}_{q_i}, \mathbf{z}_{u_j^+})/\tau) + \sum_{l=1}^K \exp(\text{sim}(\mathbf{z}_{q_i}, \mathbf{z}_{u_{j,l}^-})/\tau) \right]}, \quad (2.7)$$

avec une température $\tau = 0,05$. Le dénominateur porte sur les $B(1 + K)$ tuiles du lot et non sur les $1 + K$ tuiles associées à la question courante, si bien que chaque positif entre en compétition à la fois avec ses propres négatifs minés et avec les tuiles de toutes les autres questions du lot, les *in-batch negatives* de la section 1.5.1. À $B = 4$ et $K = 2$, le nombre de négatifs effectifs par question,

$$N_{\text{eff}} = B(1 + K) - 1 = 11, \quad (2.8)$$

passé de 2, les seuls négatifs minés si l’on entraînait avec $B = 1$, à 11, sans plongement supplémentaire à calculer. Le coût du lot croît linéairement en B , le nombre de comparaisons quadratiquement.

Lecture de l’équation (2.7) comme classification. Pour une question q_i fixée, notons $\mathbf{s}_i \in \mathbb{R}^{B(1+K)}$ le vecteur des similarités $\text{sim}(\mathbf{z}_{q_i}, \cdot)/\tau$ contre les $B(1 + K)$ tuiles du lot, et y_i l’indice de la tuile positive u_i^+ dans ce vecteur. Le terme sommé de l’équation (2.7) est alors exactement l’entropie croisée catégorielle $-\log \text{softmax}(\mathbf{s}_i)_{y_i}$. Apprendre par contraste revient donc à résoudre, pour chaque question du lot, un problème de classification à $B(1 + K)$ classes dont une seule est correcte. Le gradient par rapport à $\mathbf{z}_{q_i}$ se décompose en une force qui rapproche $\mathbf{z}_{q_i}$ de $\mathbf{z}_{u_i^+}$ et des forces qui l’éloignent de chaque négatif u , chacune pondérée

par la probabilité $\text{softmax}(\mathbf{s}_i)_u$ que le modèle attribue déjà à ce négatif. Un négatif que le modèle confond encore avec le positif reçoit ainsi un gradient plus grand qu’un négatif déjà écarté avec confiance.

La température τ contrôle l’acuité de cette pondération. En la faisant tendre vers 0, le softmax se concentre sur l’argument maximal et le gradient se resserre presque entièrement sur le négatif le plus proche du positif, ce qui accélère l’apprentissage sur les cas difficiles mais amplifie d’autant l’effet d’un faux négatif introduit par le minage (section 2.5.2). À $\tau = 0,05$, ce compromis penche délibérément du côté de la discrimination fine, cohérent avec l’objectif de la section 2.5.1 de produire des questions dont la réponse n’est visible que dans une tuile précise.

2.5.6 Contraintes mémoire

Sur une carte de 80 GB, le chemin de calcul par lots des plongements a d’abord échoué sur un dépassement de mémoire GPU, avec 79,16 GB occupés sur les 79,25 GB disponibles. L’usage est réel et non fragmenté, puisqu’un lot pousse $B(1 + K) = 12$ tuiles à travers le modèle simultanément. Aucune de ces tuiles n’étant écrêtée (section 2.3.4), la longueur de séquence croît proportionnellement à l’aire cumulée du lot et varie donc d’un lot à l’autre. À la tuile moyenne de 200 tokens (tableau 2.1), un lot ne pèse que 2 400 tokens visuels ; c’est le lot de pire cas, formé de douze tuiles de taille maximale, qui en atteint environ 64 000 et provoque le dépassement. Le dimensionnement mémoire est donc gouverné par ce pire cas et non par la moyenne.

L’activation du *gradient checkpointing*, qui recalcule les activations lors de la passe arrière plutôt que de les conserver, résout le problème au prix de 20 % à 30 % de temps de calcul supplémentaire. C’est le réglage par défaut retenu à cette résolution de tuile.

2.5.7 Partitionnement pour l’évaluation

Un partitionnement au niveau des exemples serait trompeur, car deux tuiles voisines d’une même page produisent des questions fortement corrélées, et un modèle ayant vu l’une en entraînement bénéficierait indûment sur l’autre en validation.

Le partitionnement est donc au niveau de l’**article**, en proportion 80/20 visée puis arrondie à l’article entier, avec des articles strictement disjoints entre les deux parties. Aucune tuile et aucune question quasi dupliquée ne peut traverser la frontière. L’adaptateur évalué en section 3.3 est entraîné sur la seule partie d’entraînement.

Ce choix a un coût qu’il faut énoncer avec lui. Le corpus ne compte que dix-neuf articles (tableau 3.1), si bien que la partie de validation n’en compte qu’une poignée et que les questions qu’elle porte ne sont pas des observations indépendantes. Un article atypique déplace donc l’ensemble de la mesure, et le chiffre rapporté en section 3.3 est un point sans dispersion. Répéter l’entraînement sur plusieurs partitionnements donnerait cette dispersion ; cela n’a pas été fait, faute de budget de calcul (annexe A.4).

2.6 Contrôleur de preuves en ligne

À la requête, le contrôleur décide quels nœuds élément le lecteur ouvre réellement, plutôt que de lui présenter la page entière ou une sélection top- k figée. Il généralise l’algorithme de contrôle de MAGE-RAG.

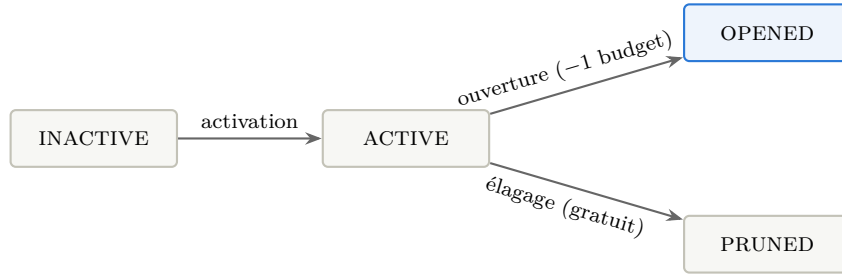


Figure 2.4 – Machine à états du contrôleur. L’ouverture d’un nœud consomme une unité de budget et peut activer ses voisins encore inactifs, le long de `reading_order` sur la même page ou en franchissant une frontière de document via `links_to` et `continues`. L’élagage est gratuit et s’applique à un nœud actif dont le score est sous le seuil d’ouverture, puis à tout nœud resté actif lorsque le budget est épuisé. Un nœud jamais activé reste INACTIVE.

2.6.1 Machine à états

Chaque nœud élément porte un attribut d’état, initialisé à INACTIVE lors de la construction du graphe, que le contrôleur fait évoluer à travers la machine à quatre états de la figure 2.4. Un nœud INACTIVE n’appartient pas à la frontière courante. Un nœud ACTIVE est candidat à l’ouverture. Un nœud OPENED a été lu et son contenu fait partie des preuves. Un nœud PRUNED a été jugé non pertinent, ou laissé de côté faute de budget.

2.6.2 Politique *best-first* budgétée

Le contrôleur cherche un sous-ensemble $O \subseteq V_{\text{élem}}$ de nœuds à ouvrir sous la contrainte de budget $|O| \leq B$, où B désigne ici le nombre d’ouvertures autorisées et non la taille de lot de la section 2.5.5. Il maximise informellement la pertinence cumulée $\sum_{n \in O} \text{score}(n)$ tout en respectant une contrainte d’accessibilité, un nœud ne pouvant entrer dans O que s’il a d’abord été rendu ACTIVE par l’ouverture d’un voisin déjà dans O , ou par l’amorçage initial.

C’est un problème de sélection sous contrainte de cardinalité et de connexité, pour lequel l’algorithme 2 implémente une heuristique *best-first* gloutonne plutôt qu’une optimisation exacte. Rien ne garantit que O maximise $\sum \text{score}(n)$ sur les sous-graphes connexes de cardinalité B atteignables depuis l’amorce, seulement que chaque décision locale est rationnelle au moment où elle est prise. Ce choix reprend celui de MAGE-RAG et privilégie le calcul en une passe.

La politique comporte trois phases. L’amorçage active les k_{seed} nœuds inactifs les mieux notés pour la requête, seule phase où la sélection est globale. La boucle principale prend répétitivement le nœud actif le mieux noté, l’élague sans consommer de budget si son score est sous le seuil d’ouverture, et sinon l’ouvre, ce qui consomme une unité de budget et rend actifs ses voisins encore inactifs qui franchissent le seuil d’activation. La finalisation élague tout nœud resté actif une fois le budget épuisé, de sorte qu’aucun nœud ne subsiste dans un état transitoire.

Ce qui distingue cette politique d’une sélection top- k n’est ni le score ni le budget, identiques, mais le fait que les voisins ne sont considérés qu’après l’ouverture de leur nœud. La frontière est conditionnée par ce qui s’est déjà révélé pertinent plutôt que par un classement global figé.

Effet de l’égalité des deux seuils. Les deux seuils valent 0,05 (annexe B), et leur égalité restreint l’élagage gratuit. Un nœud activé par un voisin a franchi θ_{act} , donc son score atteint aussi θ_{ouv} et il ne peut pas être élagué gratuitement. Seules les amorces, activées sans test de seuil, le peuvent. Le contrôleur n’ouvre donc moins de B nœuds que pour deux

Algorithme 2 — Contrôle de preuves *best-first* sous budget

Entrée: graphe G , requête q , budget B , amorce k_{seed} , seuils θ_{ouv} et θ_{act}

Sortie: liste des nœuds OPENED, en ordre de lecture

```
1 pour chaque nœud élément  $n$  de  $G$  faire
2    $\text{score}(n) \leftarrow \text{sim}(q, \text{texte}(n))$  // TF-IDF
3 activer les  $k_{\text{seed}}$  nœuds inactifs de plus haut score
4  $b \leftarrow 0$ 
5 tant que  $b < B$  et il existe un nœud ACTIVE faire
6    $n \leftarrow \arg \max \text{score}$  parmi les nœuds ACTIVE
7   si  $\text{score}(n) < \theta_{\text{ouv}}$  alors
8      $n \leftarrow \text{PRUNED}$  // gratuit
9   continuer
10   $n \leftarrow \text{OPENED}$ ;  $b \leftarrow b + 1$ 
11   $N \leftarrow \text{VOISINSORDREDELECTURE}(n) \cup \text{VOISINSINTERDOCUMENTS}(n)$ 
12  pour chaque  $n' \in N$  tel que  $n'$  est INACTIVE faire
13    si  $\text{score}(n') \geq \theta_{\text{act}}$  alors
14       $n' \leftarrow \text{ACTIVE}$ 
15 élaguer tout nœud encore ACTIVE
16 return les nœuds OPENED, triés par ordre de lecture
```

raisons, l'élagage d'une amorce ou l'épuisement de la frontière, et la section 3.4.3 mesure 4,86 ouvertures en moyenne pour un budget de 5.

Cette égalité n'est pas un détail de réglage. Elle vide de sa substance le mécanisme qui distingue la politique d'une sélection top- k , car un nœud une fois activé sera ouvert quoi qu'il arrive et le budget est presque toujours consommé intégralement. Elle explique la parité observée en section 3.4 plus directement que le diagnostic de frontière qui y est proposé. Balayer θ_{ouv} et θ_{act} indépendamment ne demande aucun appel au modèle, le scoreur étant lexical, et c'est la vérification manquante la moins coûteuse de ce rapport (annexe A.4). Elle n'a pas été conduite, aucun hyperparamètre n'ayant fait l'objet d'un balayage (annexe A.3).

2.6.3 Expansion inter-documents

Le contrôleur de MAGE-RAG, et la première implémentation à sa suite, ne propagent l'activation que le long de `reading_order`, relation qui ne franchit jamais une frontière de page. Un nœud ouvert près de la fin d'une page ne peut donc activer aucun nœud d'une page liée ni de la section suivante, alors même que les deux arêtes existent. Sur une question multi-sauts dont les preuves sont réparties sur deux articles, le second n'est atteignable que si l'amorce globale y a déjà placé un nœud, c'est-à-dire exactement ce que fait une sélection top- k statique, et moins bien dès que l'amorce en manque un.

Ce point est la cause diagnostiquée de l'incapacité du contrôleur à dépasser la référence statique sur les questions inter-pages (section 3.4.2). À l'ouverture d'un nœud, en plus de ses voisins d'ordre de lecture, sont désormais rendus candidats

- tous les éléments de toute page atteignable depuis ce nœud par une arête `links_to`, la page cible étant, grâce au recâblage de la section 2.4.6, une page réellement construite et non un substitut,
- et tous les éléments de toute page atteignable depuis sa page conteneuse par une arête `continues`.

Dans les deux cas, franchir la porte n'entraîne aucune activation automatique, chaque

élément candidat devant franchir le même seuil θ_{act} qu'un voisin d'ordre de lecture. Le mécanisme est strictement symétrique à celui qui existait déjà, seul le voisinage considéré s'élargit, et il reste limité à trois des sept relations du graphe (section 2.4), `reading_order`, `links_to` et `continues`, les quatre autres restant inatteignables quel que soit le scoreur employé. La section 3.4 mesure l'effet de ce correctif.

2.6.4 Score de pertinence et point d'extension

Par défaut, la pertinence est mesurée par similarité cosinus TF-IDF entre la requête et le texte de chaque nœud, avec le même scoreur et la même configuration que le minage de négatifs (section 2.5.2). C'est un substitut destiné à être remplacé.

Le texte lu par ce scoreur n'a pas la même longueur selon la condition. Pour une tuile ou une bande de grille, c'est l'aperçu textuel de 200 caractères relevé à l'extraction (section 2.2). Pour une fenêtre de la condition textuelle plate, c'est la fenêtre entière de 400 caractères. Employer le même scoreur dans les trois conditions isole donc l'effet du découpage à la troncature près, laquelle n'est pas la même, et cette réserve accompagne la comparaison de classement de la section 3.2.4.

Le contrôleur expose à cet effet un paramètre de scores précalculés. Si un dictionnaire de scores est fourni, il remplace intégralement le scoreur lexical, un nœud absent recevant un score nul. Ce point d'extension est conçu pour y brancher la similarité du lecteur affiné de la section 2.5, sans toucher à la politique. Le calcul de ces scores, un passage avant du modèle par tuile candidate, reste hors de ce module. L'annexe A.3 explique pourquoi ce branchement n'a pas été effectué et pourquoi il constitue la modification la plus prometteuse restant à faire.

2.6.5 Lecture des tuiles retenues

L'algorithme 2 retourne une liste de nœuds, non des preuves. Les tuiles ouvertes sont ensuite relues par le modèle vision-langage, qui en extrait les informations utiles à la requête. Les preuves finales sont la concaténation, en ordre de lecture, de ces extractions, et remplacent donc l'aperçu textuel qui a servi au scorage.

La politique a une propriété structurelle que la section 2.7.4 exploite. Quel nœud est ouvert ne dépend jamais du contenu lu par le modèle, mais uniquement des scores figés en amont. La simulation complète de la machine à états peut donc être déroulée sans aucun appel au modèle, les lectures n'étant effectuées qu'ensuite, sur une liste déjà connue.

2.7 Implémentation et ingénierie

2.7.1 Architecture modulaire

Le système est écrit en Python, avec un module par étage. Le rendu et l'extraction reposent sur Playwright, le tuilage sur des recadrages Pillow, le graphe sur `networkx`. Le client vision-langage s'adresse à un serveur Ollama, ce qui permet de démarrer avec un seul téléchargement de modèle et sans dépendance à `torch`. Le module d'affinage, seul à dépendre de `torch`, `transformers` et `peft`, n'est pas importé par le reste du système et ne charge ces bibliothèques qu'à l'exécution. La cartographie des modules figure en annexe D.

Une application web expose la chaîne complète derrière un formulaire « URL plus question ». La requête déclenche l'exécution en direct, puis affiche la réponse synthétisée, le graphe interactif (catégorie de contenu par couleur, état du contrôleur par bordure, arêtes filtrables par relation) et les pages liées, avec un marqueur sur celles portées par une tuile retenue comme preuve. C'est le seul consommateur du rendu XML de la section 2.4.7. Chaque

requête recalcule tout, sans corpus préconstruit, et le temps de réponse va de quelques dizaines de secondes à plusieurs minutes selon la page et le budget.

2.7.2 Reproductibilité des appels au modèle

Décodage glouton. Aucune température n'étant fixée, le serveur échantillonnait avec ses réglages par défaut, donc à température strictement positive. Deux appels sur la même image et la même invite pouvaient produire des réponses différentes, y compris une réponse correcte et une abstention sans aucune différence dans les tuiles ouvertes. La température est fixée à zéro, et à entrée fixée la sortie est déterministe.

Fenêtre de contexte. Le modèle d'inférence raisonne avant de répondre, sur 1 500 à 2 500 tokens pour une extraction factuelle sur une tuile. Ce raisonnement s'ajoute à l'invite et aux tokens visuels de la tuile, soit 200 en moyenne et jusqu'à 5 350 pour une tuile de taille maximale (tableau 2.1). Avec la fenêtre de contexte par défaut du serveur, fixée à 4 096 tokens, la somme épuisait la fenêtre avant l'émission de la réponse finale, et le résultat était un contenu vide, silencieusement interprété comme SKIP par l'analyseur de réponses. La fenêtre est portée à 8 192 tokens, ce qui laisse peu de marge dans le cas le plus défavorable. Cette classe d'erreurs, une défaillance d'infrastructure qui se déguise en résultat de modèle légitime, est celle contre laquelle la journalisation des longueurs de séquence protège le mieux.

Cache sur disque. Chaque appel au modèle est mis en cache sur disque, indexé par un condensat du triplet (image, invite, configuration de décodage). Le décodage étant glouton, un appel déjà effectué se rejoue sans coût. Ce cache rend praticable l'itération sur le code d'évaluation, puisque relancer un script après correction d'un défaut ne repaie pas les heures d'appels déjà effectuées.

2.7.3 Sauvegarde par unité de travail

L'état est sauvegardé par unité de page, une unité étant une page ou une section d'une page découpée. Ce grain répond directement au cas de la section 2.3.5, un échec en cours de traitement d'une page démesurée ne coûtant plus que la section en vol.

2.7.4 Concurrency

Deux étapes exploitent la concurrence, avec au plus 4 appels au modèle simultanés, et dans les deux cas la légitimité du parallélisme découle d'une propriété du système.

La génération de questions émet ses appels en parallèle au sein d'une page. Ces appels sont indépendants, chacun ne recevant qu'une image et une invite fixe.

La lecture des tuiles ouvertes fait de même, et c'est ici que la propriété de la section 2.6.5 devient opérationnelle. Puisque la trajectoire d'états ne dépend jamais du contenu lu, la liste des tuiles à lire est entièrement connue avant la première lecture, et ces lectures sont donc mutuellement indépendantes. Un unique client est réutilisé entre fils d'exécution, chaque appel étant une requête HTTP indépendante sans état mutable partagé à protéger.

2.7.5 Tests

La stratégie de test suit une séparation par dépendances, détaillée en annexe D. Les tests sans dépendance lourde couvrent la logique géométrique, la construction du graphe, le minage de négatifs, l'assemblage du jeu de données, le cache et la concurrence du client, et

la compression d’images. Ils s’exécutent sur des cas synthétiques, sans réseau, sans modèle et sans GPU.

Les tests adossés à Playwright vérifient l’extraction DOM de bout en bout contre une page HTML locale fournie avec le dépôt. Deux cas y sont verrouillés. Un élément flottant partageant sa rangée avec du texte fixe le comportement de la mesure serrée de la section 2.2.4. Deux éléments en chevauchement partiel, tous deux conservés, détectent une régression de la section 2.3.1 vers un élagage trop agressif.

2.8 Protocole expérimental

2.8.1 Construction du jeu de données synthétique

Un script pilote la chaîne sur une liste de pages, avec reprise sur échec du téléchargement. Son unité de travail est l’unité de page, et chaque unité est sauvegardée indépendamment dans un fichier au format JSON lignes. Trois fichiers en résultent, un manifeste associant chaque identifiant de tuile à son image, les paires question-réponse générées, et les exemples contrastifs assemblés avec leurs négatifs. Cette reprise ne couvre pas un échec survenant après la génération des questions, dont l’annexe A.3 rapporte l’effet.

Le plafond de 25 tuiles par unité de page (section 2.3) borne le coût de génération. Il borne aussi la portée de l’évaluation A, ce qu’analyse l’annexe A.3.

2.8.2 Reproduction de la référence en grille

Pour l’ablation du tuilage (section 3.2), un module réimplémente une grille aveugle indépendamment du module de tuilage. La condition mesurée n’est donc pas PixelRAG, mais une grille écrite pour ce travail d’après la description qu’en donne PixelRAG ; tout le rapport la nomme grille aveugle pour cette raison. La page rendue est tranchée en bandes de 1024px de haut à la largeur de rendu, ce qui reproduit la hauteur de tuile documentée par PixelRAG mais non sa largeur (section 1.3.1).

L’écart de largeur et son effet sur le budget de pixels. Les bandes de PixelRAG mesurent 875×1024 pixels, celles mesurées ici 2048×1024 , soit 2,34 fois plus de pixels offerts à la condition adverse. Comme élargir le rendu n’ajoute que de la marge blanche et non de la définition (section 2.2), cette marge est facturée entièrement à la grille et pas au tuilage, qui recadre chaque unité sur la largeur réellement occupée. Le rapport de budget de pixels rapporté en section 3.2.3 est donc mesuré à cette largeur et n’est pas transposable telle quelle à la largeur de PixelRAG : à largeur égale, il tomberait de 2,7 à environ 1,16. La mesure à 875px n’a pas été rejouée et figure parmi les travaux restants (annexe A.4). Le résultat de trouvabilité, lui, ne dépend pas de la largeur, puisqu’une bande plus étroite contient le même texte.

Troisième condition. Un découpage du texte extrait de la même région de page en fenêtres plates de 400 caractères, sans aucune notion de frontière d’élément, sert de référence textuelle standard que ni PixelRAG ni MAGE-RAG ne mesurent directement. Le scoreur est le même dans les trois conditions, mais son entrée ne l’est pas, car il lit un aperçu de 200 caractères pour une tuile ou une bande et la fenêtre entière de 400 caractères pour la condition textuelle (section 2.6.4) ; la troncature n’est donc pas neutre entre conditions et s’ajoute aux raisons pour lesquelles cette comparaison est interprétée en section 3.2.4 plutôt que prise pour un verdict.

Trois décisions de protocole propres à cette évaluation. Elles se cumulent et sont énoncées ensemble ici plutôt que dispersées. Premièrement, le plafond de 25 tuiles par unité de page est atteint sur dix-sept des dix-huit pages, si bien que la comparaison porte sur le haut de page et non sur la page entière (annexe A.3). Deuxièmement, les trois conditions sont construites avec un seuil de découpage en sections relâché pour cette seule évaluation, une hauteur de page maximale de 25 fois la hauteur maximale de tuile plutôt que 10, soit 51 200 px contre 20 480 px pour le reste du corpus, de sorte que chaque article soit traité comme une unité de page unique et que les trois conditions portent sur la même étendue de document. Troisièmement, l'article le plus long du corpus est exclu, faute de tenir en mémoire rendu comme page unique, ce qui retire du corpus d'évaluation le document qui ressemble le plus à la cible annoncée par ce travail. Le mécanisme de découpage en sections de la section 2.3.5 le traiterait, au prix d'abandonner la condition d'étendue identique. L'évaluation A porte donc sur dix-huit articles d'un seul site, tronqués en haut de page, sous un seuil qui n'est celui d'aucune autre mesure du rapport.

L'indépendance de ce module vis-à-vis du module de tuilage est un choix méthodologique, puisqu'elle garantit qu'une modification du tuilage guidé ne peut pas altérer silencieusement la référence à laquelle on le compare.

2.8.3 Critère d'attachement de la réponse

Le taux d'attachement (section 1.1.1) compte les questions dont la réponse survit au découpage. Ce que « survivre » veut dire doit être posé, car le critère n'est pas neutre entre les trois conditions comparées et détermine le chiffre le plus mis en avant de ce rapport.

Une réponse est déclarée attachée à une unité lorsque sa chaîne normalisée apparaît dans le texte associé à cette unité. Ce texte n'a pas la même provenance selon la condition. Pour une tuile DOM, c'est le texte des éléments du DOM que la tuile recadre. Pour une bande de grille, c'est le texte des éléments dont la boîte englobante intersecte la bande, aucun OCR n'étant appliqué à l'image. Pour une fenêtre plate, c'est la fenêtre elle-même. Le critère est donc lexical et géométrique, jamais visuel.

Deux biais en découlent, et ils vont en sens opposés. Le critère favorise la condition textuelle, qui manipule directement la chaîne comparée, ce que la section 3.2.2 rapporte. Il défavorise la grille pour une raison sans rapport avec la perte d'information, puisqu'une bande large intersecte des éléments de colonnes latérales dont le texte n'est pas celui qu'un lecteur y trouverait, et qu'un élément à cheval sur deux bandes voit son texte compté entièrement dans chacune. Ce que la mesure établit est donc la récupérabilité lexicale d'une unité plutôt que la survie de la réponse au sens strict. Un contrôle manuel de la correspondance entre le critère et le jugement humain, sur un échantillon de questions, reste à faire (annexe A.4).

Pour les questions HotpotQA (section 2.8.5), l'attribution d'une phrase de support à une tuile est plus lâche, car elle procède par recouvrement de tokens filtrés des mots vides au-delà d'un seuil de **[seuil de recouvrement de tokens]**, et non par correspondance de sous-chaîne. Ce second critère définit les tuiles positives des évaluations C et D.

2.8.4 Ablation de résolution

PixelRAG documente la résolution d'image comme levier d'efficacité, sans préciser ni le filtre de ré-échantillonnage ni les résolutions testées. Un couple pleine résolution et résolution réduite est donc construit sur les tuiles de ce travail, par sous-échantillonnage Lanczos à un facteur d'échelle linéaire de 0,5, soit quatre fois moins de pixels. Les images compressées sont écrites sous les mêmes noms dans un dossier miroir, de sorte que le manifeste et les exemples contrastifs restent réutilisables tels quels, seul le dossier racine des images changeant entre les deux expériences.

2.8.5 Questions multi-sauts

La génération synthétique de la section 2.5.1 a un biais structurel, puisqu'elle demande une question dont la réponse est visible dans une seule tuile. Un contrôleur omniscient pourrait donc répondre à n'importe laquelle de ces questions depuis un unique nœud, ce qui ne sollicite jamais le graphe.

Cet écart est comblé avec HotpotQA [44], dont chaque question nomme, via ses phrases de support, deux articles Wikipédia et les phrases exactes nécessaires dans chacun. Ce signal annoté par des humains est réutilisé plutôt que régénéré, selon la procédure suivante.

1. Les titres d'articles cités sont convertis en URL Wikipédia et rendus par la chaîne existante, inchangée.
2. Pour chaque article, les tuiles dont le texte recouvre une phrase de support sont retenues comme positives. Une même question peut donc avoir des tuiles positives sur deux pages distinctes.
3. Les négatifs difficiles sont minés sur le vivier réunissant les tuiles des deux articles, l'extension inter-pages que la section 2.5.2 laissait de côté étant ici obligatoire.

Une question est abandonnée dans deux cas, plutôt que de produire un exemple incomplet. Le premier est un article qui ne se rend plus, renommé ou supprimé depuis le *dump* Wikipédia de 2017 sur lequel HotpotQA est construit. Le second est une phrase de support introuvable dans le rendu actuel, l'article ayant été réécrit. Le script journalise la cause de chaque abandon, mais les deux comptes n'ont pas été relevés, si bien que la part respective des deux causes n'est pas connue.

L'attribution d'une phrase de support à une tuile se fait par recouvrement de tokens plutôt que par correspondance de sous-chaîne exacte, celle-ci survivant rarement à une décennie de modifications éditoriales. Le critère exact et son seuil sont donnés en section 2.8.3.

Chapitre 3

Résultats et limites

Ce chapitre rapporte les quatre évaluations du système décrit au chapitre 2, y compris celles dont le résultat ne va pas dans le sens attendu. Deux d’entre elles étayent solidement un gain, le tuilage guidé par le DOM (section 3.2) et l’entraînement contrastif du lecteur (section 3.3). Les deux autres portent sur les composants en aval, le contrôleur budgété (section 3.4) et le système assemblé bout-en-bout (section 3.5), et n’établissent pas de supériorité. Les détails de protocole, les coûts de calcul observés, les limites et les directions de travail sont reportés dans les [Résultats complémentaires](#) (annexe A).

MAGE-RAG est évalué sur LongDocURL [8] et MMLongBench-Doc [29], deux jeux de données constitués de PDF déjà analysés. Ils ne peuvent solliciter un tuilage guidé par le DOM, puisqu’un PDF n’expose ni boîtes englobantes, ni sélecteurs CSS, ni frontières de titres HTML, et ne sont donc pas repris ici comme comparaison numérique. Toutes nos évaluations portent sur des pages web réelles.

Une réserve vaut pour l’ensemble du chapitre. Le corpus compte dix-neuf articles d’un seul site, et les quatre évaluations sont des vues différentes de ces mêmes pages. C’est cette taille de corpus, plus encore que le nombre de questions, qui borne ce que ce travail démontre.

3.1 Jeu de données

Tableau 3.1 récapitule les volumes construits par le protocole de la section 2.8.

Le taux de SKIP vaut 3,8 %. Il n’a pas été mesuré pour la grille aveugle ni pour le texte plat, faute de quoi il ne dit rien de comparatif et n’est rapporté ici que comme volume. Les 378 questions qui ne produisent pas d’exemple contrastif proviennent de seize unités de page absentes du jeu construit, et non d’un filtrage par question (annexe A.3). Le taux de conservation des questions HotpotQA vaut 61 % (91/150). Deux causes d’abandon le produisent, un article qui ne se rend plus et une phrase de support introuvable dans le rendu actuel ; leurs poids respectifs n’ont pas été relevés (section 2.8.5), de sorte que l’attribution de ce taux à la dérive éditoriale de Wikipédia depuis le *dump* de 2017 reste une hypothèse plausible et non une mesure.

3.2 Effet du tuilage guidé par le DOM

3.2.1 Protocole

Cette évaluation compare trois représentations du même contenu de page, nos **tuiles DOM** (section 2.3), une **grille aveugle** de bandes de 1 024 px écrite pour ce travail d’après la description de PixelRAG [40] mais à une autre largeur (section 2.8.2), et un **texte plat** découpé en fenêtres de 400 caractères sans notion de frontière d’élément. La condition adverse n’est donc pas PixelRAG, et tout ce qui suit se lit comme une comparaison à une grille aveugle pleine largeur. Dix-huit des dix-neuf articles du corpus sont rendus comme unité de page unique ; les trois représentations sont construites sur cette unité, et 409 questions, celles dont la page n’a pas été découpée en sections, sont classées contre chacune par similarité cosinus TF-IDF, le même scoreur dans les trois conditions, pour isoler l’effet du découpage. Une question n’est évaluée dans une condition que si sa réponse apparaît dans l’une de ses

Tableau 3.1 – Volumes du jeu de données utilisé par les évaluations. Les tuiles sans question exploitable sont celles pour lesquelles le modèle a répondu **SKIP** ; elles restent disponibles comme négatifs difficiles (section 2.5.1).

Grandeur	Valeur
Articles Wikipédia distincts	19
dont découpés en sections (section 2.3.5)	12
Unités de page (articles + sections)	185
Tuiles extraites	1 784
Paires question-réponse générées	1 717
tuiles sans question exploitable (SKIP)	67 (3,8 %)
Exemples contrastifs (question + négatifs difficiles)	1 339
questions perdues entre génération et minage	378 (16 unités)
exemples portant les $K = 2$ négatifs attendus	1 282
Questions inter-pages construites depuis HotpotQA	91 / 150 échantillonnées

unités au sens du critère formalisé en section 2.8.3, ce qui produit le taux d’attachement ci-dessous.

Trois décisions de protocole restreignent ensemble la portée de cette évaluation, et leur effet se cumule. Le plafond de 25 tuiles par page est atteint sur dix-sept des dix-huit pages, si bien que la comparaison porte sur le haut de page. Le seuil de découpage en sections est relâché pour cette seule évaluation. Et le dix-neuvième article (Beatles, le plus long du corpus) est exclu, faute de tenir en mémoire rendu comme page unique, ce qui retire du corpus le document le plus proche de la cible annoncée par ce travail. Le détail de ces trois décisions figure en section 2.8.2, et la conséquence sur ce qui est mesuré en annexe A.1. La valeur de 24,0 tuiles par page qui sert de dénominateur au calcul de pixels est donc un plafond atteint et non une densité mesurée, et le rapport d’environ cinq entre tuiles DOM et bandes de grille est fixé par ce plafond au moins autant que par le tuilage.

Les trois viviers de candidats diffèrent fortement en taille (24,0 tuiles DOM par page en moyenne, contre 4,9 bandes de grille et 12,3 fenêtres de texte), de sorte que le Recall@ k brut y est gonflé mécaniquement par un vivier plus petit (section 1.1.1), et la comparaison de classement de la section 3.2.4 utilise le rang percentile invariant à la taille du vivier plutôt que le Recall brut.

3.2.2 Taux d’attachement

Le tuilage guidé par le DOM porte la réponse dans une unité récupérable pour 51,8 % des questions (212 sur 409), contre 26,4 % pour la grille aveugle (108 sur 409), soit presque le double.

L’explication est mécanique. Une bande de 1 024 px découpée sans égard au contenu agrège plusieurs sections hétérogènes, si bien qu’une réponse courte, une date, un nombre ou un nom propre, s’y trouve noyée dans un volume de texte sans rapport, au point que le critère d’attribution ne la retrouve plus. Une tuile alignée sur une frontière d’élément contient au contraire une unité de sens complète et délimitée. Ce résultat détermine un plafond, car une question dont la réponse ne survit dans aucune unité est perdue quelle que soit la qualité du récupérateur en aval, et doubler le taux d’attachement double donc le nombre de questions auxquelles le système a la possibilité de répondre.

Le texte plat obtient un taux encore légèrement supérieur, 56,7 %. Le critère d’attachement n’est pas neutre entre les trois conditions, et c’est l’explication la plus simple de cet ordre. Il favorise la condition textuelle, qui manipule directement la chaîne comparée, et il défavorise la

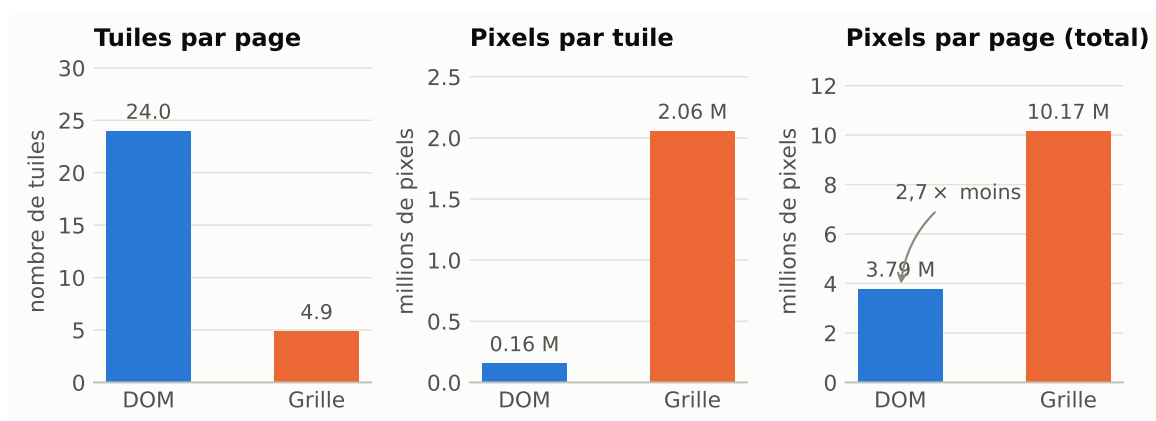


Figure 3.1 – Coût de représentation par page, moyenné sur les 18 pages de l’évaluation, pour la même région de page dans les deux conditions. Le tuilage DOM produit près de cinq fois plus de tuiles, mais chacune est treize fois plus petite, de sorte que le total par page est 2,7 fois inférieur.

grille pour une raison sans rapport avec la perte d’information, une bande large intersectant des éléments dont le texte n’est pas celui qu’un lecteur y trouverait (section 2.8.3). Ce qui est mesuré ici est donc la récupérabilité lexicale d’une unité plus que la survie de la réponse au sens strict, et le contrôle manuel qui rapprocherait les deux reste à faire (annexe A.4).

3.2.3 Budget de pixels

Notre tuilage produit environ cinq fois plus d’unités que la grille (24,0 contre 4,9 par page). On attendrait un coût de représentation supérieur ; c’est l’inverse qui se produit.

*Le total des pixels par page est de 3,79 millions pour le tuilage DOM contre 10,17 millions pour la grille, soit **2,7 fois moins**, à la largeur de rendu de 2048 px et sous le plafond de tokens relevé de la section 2.3.4.*

L’explication est la mesure serrée des balises de texte coulant (section 2.2.4, équation (2.2)). Une bande de grille occupe par construction toute la largeur de rendu quel que soit son contenu, tandis qu’une tuile de texte DOM est recadrée sur la largeur que ses lignes occupent réellement, soit 157 170 pixels carrés en moyenne contre 2 075 510 pour la grille, un rapport de treize qui compense largement le rapport de cinq sur le nombre d’unités.

Ce rapport dépend de deux constantes de configuration, et l’annoncer sans elles serait trompeur. La première est la largeur de rendu, car la marge blanche qu’une bande pleine largeur emporte est facturée à la grille seule, si bien qu’à la largeur de 875 px employée par PixelRAG le rapport tomberait à environ 1,16 (section 2.2). Le rapport mesuré ici est en réalité l’inverse du taux de couverture des tuiles à cette largeur, et non une propriété du tuilage seul. La seconde est le plafond du processeur d’image, car le nombre de tokens visuels croît avec l’aire sous réserve de ce plafond (section 2.3.4). Relevé douze fois au-dessus du défaut, ce plafond épargne l’écrtage à la bande de grille et le rapport de 2,7 se transpose alors à l’identique au coût en tokens, quand il tomberait à 1,3 au plafond par défaut (tableau 2.1).

Le gain de trouvabilité est le plus robuste des deux. L’écart de vingt-cinq points de taux d’attachement écrase largement la variation de quatre points observée entre deux exécutions du même protocole (tableau E.1), et il ne dépend d’aucune constante de rendu, une bande plus étroite portant le même texte. Le gain de budget de pixels, lui, n’est établi qu’au couple de constantes ci-dessus. Aucun test apparié n’a été conduit sur les 409 questions, et les deux chiffres sont des mesures ponctuelles sur dix-huit pages d’un seul site.

Tableau 3.2 – Qualité de classement des trois conditions, même scoreur TF-IDF, 18 pages et 409 questions. %rang désigne le rang percentile moyen, qui vaut 0,5 sous classement aléatoire et reste invariant à la taille du vivier. Les Recall bruts ne sont pas comparables entre conditions (section 1.1.1) et sont donnés à titre de diagnostic.

Condition	<i>n</i>	unités/page	R@1	R@5	%rang
<i>Toutes les paires attribuables</i>					
Tuiles DOM	212	24,0	0,547	0,774	0,868
Grille aveugle	108	4,9	0,722	0,991	0,839
Texte plat	232	12,3	0,612	0,875	0,867
<i>Paires appariées (107 attribuables dans les trois conditions)</i>					
Tuiles DOM	107	24,0	0,514	0,841	0,901
Grille aveugle	107	4,9	0,729	0,991	0,845
Texte plat	107	12,3	0,710	0,944	0,928

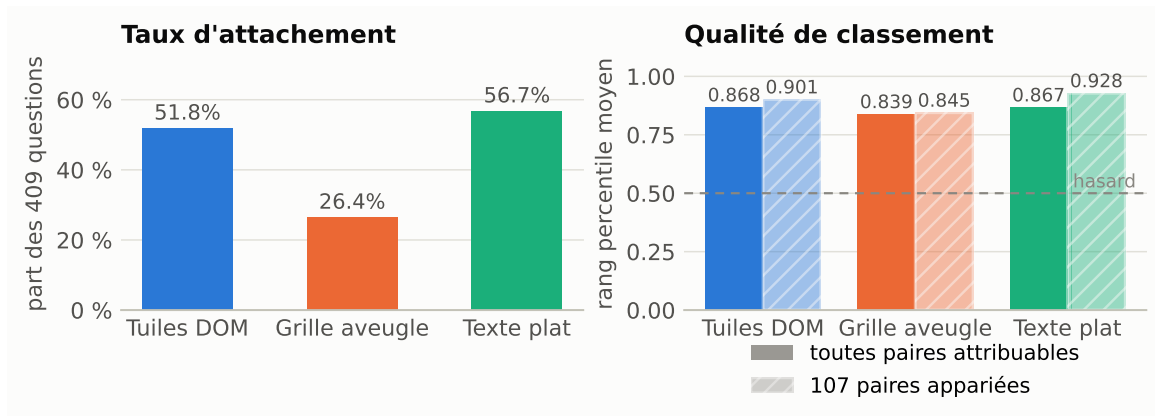


Figure 3.2 – À gauche, le taux d’attachement. À droite, le rang percentile moyen sur toutes les paires attribuables (barres pleines) et sur les 107 paires appariées (barres hachurées). La ligne pointillée marque la valeur attendue sous classement aléatoire.

3.2.4 Qualité de classement

Tableau 3.2 rapporte le rang percentile (équation (1.1)), sur toutes les paires attribuables puis sur les 107 paires attribuables dans les trois conditions simultanément (comparaison strictement appariée).

Trois lectures s’imposent. **L’artefact de vivier est massif**, puisque la grille atteint 0,991 de Recall@5, un score presque parfait qui n’exprime rien d’autre que le fait qu’elle ne propose que 4,9 candidats par page, ce qui valide empiriquement la nécessité du rang percentile. **L’avantage du tuilage DOM sur la grille n’est pas établi**, car les 0,868 contre 0,839 sur toutes les paires représentent un écart de 0,029, quand le rang percentile de la grille se déplace de 0,015 entre deux exécutions du même protocole (tableau E.1) : le bruit d’exécution vaut la moitié de l’effet revendiqué, et aucun test apparié n’a été conduit. **Le texte plat devance le tuilage DOM sur les paires appariées**, avec 0,928 contre 0,901, contrairement à la prémisse partagée par PixelRAG, par MAGE-RAG et par notre propre introduction.

Cette dernière observation porte sur la comparaison la plus propre du protocole, celle où les trois conditions traitent exactement les mêmes questions. Un découpage textuel plat de 400 caractères y classe mieux qu’un tuilage guidé par la structure, et c’est un résultat contraire à la thèse de ce travail.

Tableau 3.3 – Modèle de base et modèle affiné par LoRA contrastif, sur la partie de validation (330 questions, articles jamais vus à l’entraînement). Le classement porte sur toutes les tuiles de la page de la question.

Scoreur	R@1	R@3	R@5	MRR
TF-IDF (lexical, référence)	0,461	0,618	0,724	0,579
Modèle de base (sans LoRA)	0,433	0,645	0,752	0,578
Modèle affiné (LoRA contrastif)	0,785	0,915	0,952	0,856

3.2.5 Pourquoi le texte plat reste compétitif

Trois facteurs plausibles l’expliquent, et aucun n’a été testé. **Le scoreur n’exploite aucune information spatiale**, car TF-IDF, employé dans les trois conditions pour isoler l’effet du découpage, ne sait pas qu’une cellule appartient à une ligne de tableau ou qu’un titre coiffe un paragraphe, et voit un sac de mots là où repose précisément la valeur du tuilage structuré. Le résultat mesurerait alors le tuilage sur une métrique aveugle à ce qu’il apporte, et rejouer la mesure avec le scoreur affiné de l’évaluation B trancherait (annexe A.4). **La troncature n’est pas la même**, le scoreur lisant un aperçu de 200 caractères pour une tuile ou une bande et la fenêtre entière de 400 caractères pour le texte plat (section 2.6.4), de sorte que la condition textuelle est jugée sur davantage de texte. **Le texte plat ne porte aucune information visuelle**, puisqu’un chiffre lu sur un axe ou une correspondance ligne-colonne dans un tableau complexe ne subsiste pas dans du texte extrait, une limite que nos 18 pages, majoritairement textuelles, ne font pas ressortir.

Une quatrième explication, plus simple, ne peut pas être écartée. Sur des articles encyclopédiques essentiellement textuels, le tuilage visuel n’apporte peut-être rien de plus qu’un texte bien découpé, et ce sont les documents où la mise en page porte l’information, un rapport financier ou une notice technique, qui sépareront les deux conditions. Quelques pages de ce type suffiraient à trancher, et elles manquent au corpus. En l’état, la formulation exacte est que le texte plat classe mieux sur ce corpus et sous ce scoreur, et que la raison n’est pas connue.

3.3 Effet de l’entraînement contrastif du lecteur

3.3.1 Protocole

Cette évaluation mesure l’apport de l’entraînement contrastif (section 2.5) sur un partitionnement au niveau de l’**article**, en proportion 80/20, avec des articles strictement disjoints entre entraînement et validation (section 2.5.7). Chaque question de validation est classée contre toutes les tuiles de sa page, un scénario de récupération réaliste plutôt que le petit vivier de négatifs minés vu à l’entraînement, avec et sans l’adaptateur chargé sur le même modèle de base, en le désactivant plutôt qu’en chargeant un second modèle. Sur 338 questions de validation, 330 sont retenues après exclusion des pages à moins de deux tuiles ou à tuile positive absente du manifeste. Ces 330 questions proviennent d’une poignée d’articles seulement, le corpus n’en comptant que dix-neuf au total, et ne constituent donc pas 330 observations indépendantes (section 2.5.7).

3.3.2 Résultat et analyse

Le gain est net, le Recall@1 passant de 0,433 à 0,785, un quasi-doublement, et le MRR de 0,578 à 0,856. Le partitionnement au niveau de l’article et l’évaluation sur toutes les tuiles

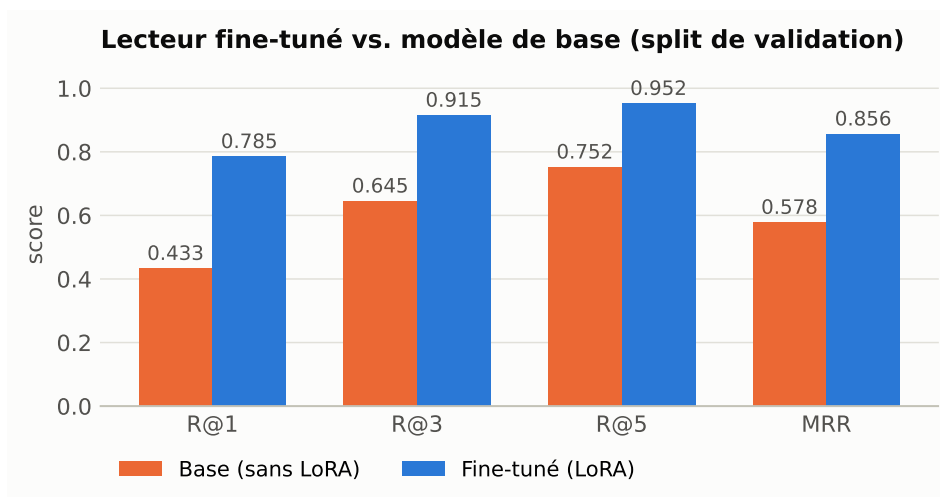


Figure 3.3 – Gain de l’entraînement contrastif sur la partie de validation. Le Recall@1 passe de 0,433 à 0,785 et le MRR de 0,578 à 0,856, sur un partitionnement par article strictement disjoint.

de la page écartent les deux mécanismes de fuite les plus courants, si bien que le gain reflète un transfert à des articles jamais vus.

Une troisième fuite, plus profonde, subsiste. Pour chaque question, la tuile positive est exactement l’image dont un modèle de la même famille a tiré cette question (section 2.5.1). Ce que le modèle apprend pourrait donc être de retrouver l’image d’où provient un texte, et non de récupérer une preuve. Le gain rapporté ici vaut pour des questions synthétiques issues de la même génération, et le contrôle qui trancherait est la même mesure de récupération sur les questions humaines de HotpotQA. Ce contrôle est une évaluation sur 91 questions, non un entraînement ; il est écrit et n’a pas été exécuté (annexe A.3). C’est la vérification manquante la plus décisive de ce rapport.

La comparaison avec TF-IDF n’est pas une victoire uniforme du scoreur lexical. Sur le Recall@1 et le MRR, les métriques les plus pertinentes pour une sélection à un seul candidat, le modèle vision-langage non adapté est à parité ou légèrement en retrait (MRR 0,578 contre 0,579), et ne devance TF-IDF que sur Recall@3/Recall@5. Le gain de Recall@1 vient donc entièrement de l’adaptation contrastive, et non d’une capacité de récupération générale déjà présente avant affinage. La comparaison n’oppose toutefois pas deux fonctions de score sur la même entrée, car TF-IDF lit l’aperçu textuel de la tuile quand le modèle en lit l’image ; ce qui change entre les deux lignes n’est pas seulement la fonction de score.

Ce résultat désigne aussi le correctif direct de l’évaluation A. Le scoreur affiné voit l’image de la tuile et pourrait exploiter l’information spatiale que le tuilage guidé préserve, et rejouer l’évaluation A avec lui est l’expérience directement indiquée par l’enchaînement de ces deux résultats (annexe A.4).

3.4 Évaluation C, contrôleur budgété contre sélection top- k statique

3.4.1 Protocole

Cette évaluation mesure la capacité du contrôleur (section 2.6) à rassembler des preuves réparties sur deux documents. Pour chacune des 91 questions inter-pages construites depuis HotpotQA (section 2.8.5), le graphe combiné des deux articles cités est construit, avec recâblage des arêtes `links_to` sur leur point d’entrée réel (section 2.4.6). Le contrôleur et

Tableau 3.4 – Fraction des 91 questions inter-pages pour lesquelles les preuves sélectionnées couvrent une tuile positive sur chacun des deux articles, avant et après le correctif de frontière inter-documents.

Amorce k_{seed}	Contrôleur budgété	Top- k statique
<i>Avant correctif (<code>reading_order</code> seul)</i>		
3 (budget 5)	29,7 % (27/91)	40,7 % (37/91)
5 (= budget)	40,7 % (37/91)	40,7 % (37/91)
<i>Après correctif (+ <code>links_to</code> et <code>continues</code>)</i>		
3 (budget 5)	39,6 % (36/91)	40,7 % (37/91)
5 (= budget)	40,7 % (37/91)	40,7 % (37/91)

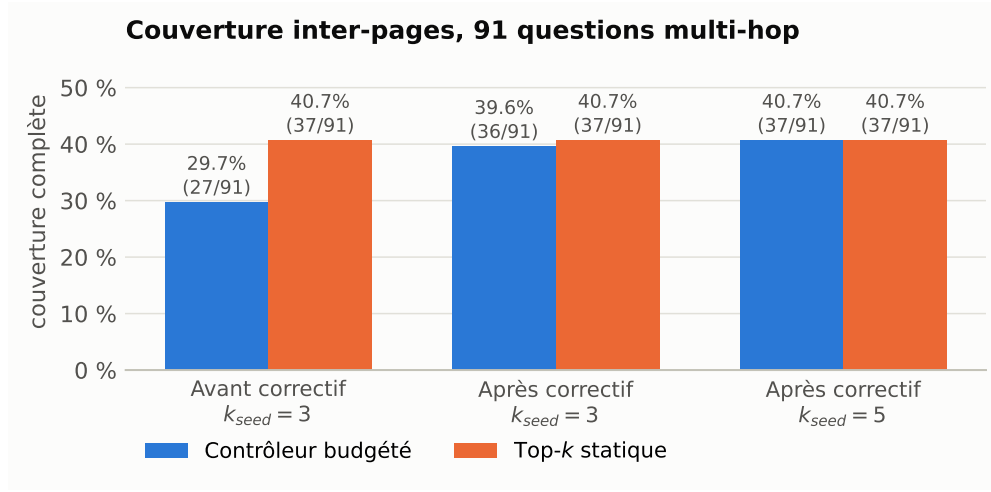


Figure 3.4 – Couverture complète inter-pages. Le correctif produit l’effet prédit par le diagnostic, la couverture du contrôleur passant de 27 à 36 questions sur 91 à l’amorce par défaut, ce qui réduit l’écart avec la référence de dix points à un point.

une référence top- k statique sont exécutés sur les mêmes scores de pertinence, avec le même budget ; seule la politique de sélection diffère. La métrique est la **couverture complète**, c’est-à-dire la fraction des questions dont les preuves sélectionnées contiennent au moins une tuile positive sur chacune des deux pages. Deux configurations d’amorce sont rapportées, $k_{\text{seed}} = 3$ (valeur par défaut, budget 5) et $k_{\text{seed}} = 5$ (égale au budget).

3.4.2 Diagnostic, correctif et résultat

La première mesure, frontière propagée uniquement le long de `reading_order`, donnait 29,7 % de couverture complète pour le contrôleur contre 40,7 % pour la référence à l’amorce par défaut, et une égalité exacte à $k_{\text{seed}} = 5$ (bloc supérieur du tableau 3.4), une égalité attendue puisque à amorce égale au budget le contrôleur ouvre son amorce intégralement et devient la référence plutôt que d’y dégénérer, ce qui vérifie la cohérence de l’implémentation. Le déficit à $k_{\text{seed}} = 3$ s’expliquait par l’incapacité de la frontière à franchir une limite de document, car `reading_order` ne relie jamais deux pages, si bien que le second article n’était atteignable que si l’amorce y avait déjà placé un nœud.

Après implémentation de l’expansion inter-documents (section 2.6.3), la couverture à l’amorce par défaut passe de 27 à 36 questions sur 91, soit de 29,7 % à 39,6 %. L’hypothèse a toutefois été formée en examinant ces mêmes 91 questions, et le correctif y a ensuite été évalué ; il ne s’agit donc pas d’une prédiction hors échantillon, mais d’un diagnostic vérifié

Tableau 3.5 – Précision bout-en-bout du pipeline complet, 20 questions multi-sauts HotpotQA. La taille de l’échantillon ne permet pas d’intervalle de confiance utile ; c’est le mode de défaillance du tableau 3.6 qui porte l’information.

Métrique	Valeur
Correspondance exacte	0,050 (1/20)
F_1 sur tokens	0,057

sur les données qui l’ont suggéré.

Le contrôleur ne devance la référence dans aucune des deux configurations. À l’amorce par défaut il reste derrière d’une question, et à $k_{\text{seed}} = 5$ il lui est rigoureusement identique, c’est-à-dire qu’il n’apporte rien. Les deux politiques tournant sur les mêmes 91 questions, les proportions sont appariées et un test de McNemar est le test approprié. Sur les quinze paires discordantes (7 contre 8, voir ci-dessous), il donne $p = 1$. La conclusion ne change pas, les deux politiques restent indiscernables sur cet échantillon, mais elle repose sur un test apparié plutôt que sur la comparaison de deux intervalles de Wilson indépendants, laquelle serait à la fois trop conservatrice et formellement inadaptée à des mesures appariées.

3.4.3 Analyse

Le contrôleur devance la référence sur 7 questions et la suit sur 8, un échange à peu près équilibré plutôt qu’un déficit systématique. Il ouvre en moyenne 4,86 nœuds contre 5,00 pour la référence, c’est-à-dire qu’il élague parfois un nœud sous le seuil d’ouverture plutôt que de dépenser son budget, ce qui est souhaitable en soi mais sans gain de couverture.

Ce chiffre de 4,86 désigne en réalité la cause la plus directe de la parité, et elle est antérieure au diagnostic de frontière. Les deux seuils du contrôleur étant égaux (section 2.6.2), un nœud une fois activé ne peut plus être élagué gratuitement, le budget est consommé presque intégralement, et la politique se réduit pratiquement à une sélection top- k . Balayer ces deux seuils indépendamment ne demande aucun appel au modèle et n’a pas été fait.

Douze questions sur 91 obtiennent une couverture nulle. Le classement en est une cause possible, puisqu’il ne place pas les bonnes tuiles en tête même sur la page de départ, mais le tuilage en est une autre, tout aussi plausible : si la phrase de support n’est présente dans aucune tuile, aucun scoreur ne la trouvera. Une seule requête sur les tuiles positives de ces douze questions départagerait les deux hypothèses, et elle n’a pas été faite. Si le tuilage était en cause, ce constat entrerait en tension avec le résultat d’attachement de la section 3.2.2, mesuré sur un autre jeu de questions et sous un autre critère.

3.5 Évaluation D, le système assemblé

3.5.1 Protocole et résultat

Cette évaluation mesure le pipeline complet. Le contrôleur post-correctif sélectionne les preuves (budget 5, amorce 3), le modèle vision-langage lit chaque tuile ouverte, et un appel de synthèse final produit la réponse. Elle porte sur un tirage aléatoire reproductible (`random.Random(seed=0).sample`) de 20 des 91 questions inter-pages de l’évaluation C, l’échantillon restant restreint en raison du coût, environ deux minutes par tuile ouverte. Les réponses sont notées par correspondance exacte et F_1 sur tokens après normalisation standard de SQuAD [32]. Ces 20 questions étant un sous-ensemble des 91 de l’évaluation C, les deux évaluations ne sont pas deux voies indépendantes vers le même constat, mais deux mesures du même système sur le même matériel.

Tableau 3.6 – Répartition des 20 prédictions du système assemblé, établie manuellement, question par question. Les cinq catégories sont exclusives et somment à vingt. La catégorie « partiellement correcte » est définie par $F_1 > 0$ et non par le contenu, ce qui y range aussi deux abstentions dont le F_1 est non nul par recouvrement fortuit de tokens. Le F_1 moyen de 0,057 se reconstitue depuis cette répartition, la réponse exacte apportant 1,0, la seule prédiction réellement partielle 0,133 et les deux recouvrements fortuits le reste, soit 1,14/20.

Catégorie	n	Exemple de prédiction
Abstention explicite	13	« Not specified in evidence », « Not found in evidence », « Not determinable from evidence »
Retournement oui/non erroné	2	« No » contre une référence « yes »
Partiellement correcte ($F_1 > 0$)	3	réponse hybride sur « ... Tenerife, Spain ... » contre « Canary Islands, Spain » ($F_1 = 0,133$), et deux abstentions à recouvrement fortuit
Entité erronée	1	« iMac G5 » contre « keyboard function keys »
Réponse exacte	1	« Blake Shelton »

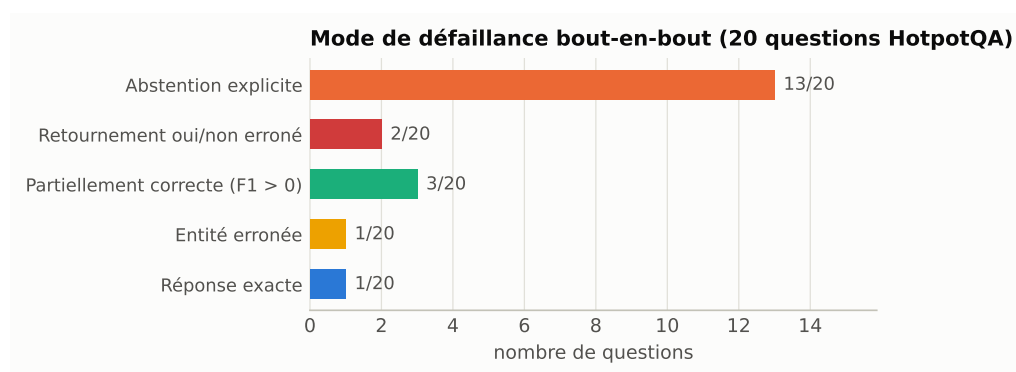


Figure 3.5 – Treize prédictions sur vingt sont une abstention explicite, le système reconnaissant que les preuves rassemblées ne suffisent pas plutôt que de produire une réponse inventée. Deux autres abstentions sont comptées comme partiellement correctes, la catégorie étant définie par $F_1 > 0$.

Une réponse exactement correcte sur vingt et un F_1 de 0,057. Prise seule, cette mesure indique que le système assemblé n’est pas prêt pour un usage réel. Ce qui la rend informative est la nature des erreurs.

3.5.2 Taxonomie des erreurs, sous-rappel de preuves plutôt que fabulation

Les vingt prédictions ont été classées à la main en cinq catégories exclusives (tableau 3.6), dont la lecture porte davantage d’information que les deux métriques agrégées ci-dessus.

Le mode dominant est l’abstention. Treize prédictions sur vingt déclarent explicitement que les preuves ne permettent pas de répondre, et deux autres le font tout en récoltant un F_1 non nul par recouvrement fortuit. Le lecteur n’échoue pas à interpréter les preuves qu’on lui donne, il constate que celles-ci sont insuffisantes, si bien que la défaillance est en amont, dans la sélection.

Cette abstention est prescrite, non émergente. L’invite de synthèse (annexe C) instruit explicitement le modèle de s’abstenir plutôt que d’inventer. Constater qu’il s’abstient vérifie donc qu’il suit sa consigne, et ne démontre pas que le système serait bien calibré. Rejouer les vingt questions sans cette instruction, soit vingt appels, séparerait les deux lectures ; cela n’a pas été fait.

La fabulation reste minoritaire, trois prédictions sur vingt affirmant quelque chose

de faux, deux retournements oui/non et une entité erronée, pour une prédiction réellement partielle et une seule réponse exacte. **La synthèse échoue aussi lorsque la couverture est complète**, car la couverture de 39,6 % de l'évaluation C fixe un plancher optimiste que l'exactitude bout-en-bout (une réponse sur vingt) n'atteint pas ; cet écart supplémentaire n'est pas encore diagnostiqué, entre preuve mal lue, invite sous-spécifiée et faiblesse de lecture du modèle de base, et reste à trier avant d'étendre l'échantillon (annexe A.4).

Une lecture doit en revanche être écartée. Une couverture complète de 39,6 % ne signifie pas que les six autres questions sur dix seraient inrépondables et que l'abstention y serait la bonne réponse. Beaucoup de questions HotpotQA se répondent depuis une seule des deux pages, et des preuves suffisantes peuvent se trouver dans des tuiles non étiquetées comme positives. Traiter l'abstention comme correcte dès que la couverture est incomplète transformerait un échec en succès par définition.

Le retournement oui/non mérite une mention à part, car c'est la seule erreur susceptible d'induire un utilisateur en erreur sans qu'il s'en aperçoive, un « non » assuré ne signalant rien de son incertitude.

3.6 Synthèse, coûts et limites

Ce que ce travail établit se formule sans emphase. Le tuilage guidé par le DOM rend davantage de réponses attachables sous notre critère, sur dix-huit pages Wikipédia, contre une grille aveugle pleine largeur, et pour un budget de pixels moindre à la largeur de rendu employée. L'entraînement contrastif du lecteur améliore nettement le classement sur des articles jamais vus, sur des questions synthétiques issues de la même génération. Le graphe multigranulaire, lui, se construit bien depuis le DOM, mais rien ne montre encore qu'un contrôleur en tire parti.

Les deux étapes en aval convergent vers un diagnostic unique. Le **contrôleur budgété** (section 3.4) n'atteint que la parité avec une sélection top- k statique, et deux causes s'additionnent, l'égalité des seuils qui neutralise l'élagage et une frontière d'exploration longtemps limitée à `reading_order`. Le **système assemblé** (section 3.5) échoue par abstention plutôt que par fabulation, un comportement conforme à l'invite qui le lui prescrit, ce qui localise la défaillance en amont, dans la sélection.

Ces observations, ainsi que le résultat de classement de l'évaluation A, désignent un premier composant, le **score de pertinence du contrôleur**, qui reste un substitut lexical (TF-IDF) alors que l'évaluation B a validé un scoreur nettement supérieur sur exactement cette tâche. Le point d'extension qui permet la substitution est déjà en place dans l'implémentation (section 2.6.4), et la seule barrière est le coût de calcul, un passage avant du modèle par tuile candidate et par question, analysé avec les autres coûts observés en annexe A.2.

Trois limites s'y ajoutent, indépendantes de la première. **Les deux seuils du contrôleur sont égaux**, ce qui rend l'élagage impossible hors amorce et ramène la politique à un top- k ; c'est la seule des trois qui se corrige sans GPU, la simulation ne demandant aucun appel au modèle. **La politique n'emprunte que trois des sept relations**, `layout_adjacency` et `section_hierarchy` étant instanciées mais jamais interrogées, et aucun scoreur ne les rendrait traversables puisque la fonction de voisinage ne les consulte pas (algorithme 2). **Aucune relation n'a été évaluée séparément**, l'ablation prévue pour `semantic_neighbor` n'ayant pas été conduite. Le graphe effectivement parcouru est donc plus proche de deux chaînes cousues ensemble que de la structure à partir de laquelle il est construit.

Le système ne présente donc pas une défaillance diffuse dont on ignorerait l'origine, mais deux composants validés et trois limites nommées et instrumentées. Les limites restantes, portée à Wikipédia, plafond de tuiles, tailles d'échantillon et hyperparamètres non balayés, ainsi que les directions de travail qui en découlent, ordonnées par bénéfice sur coût, sont

détaillées en annexes [A.3](#) et [A.4](#).

Conclusion

Rappel

Ce travail est parti d’une observation dont la simplicité est le principal argument. Pour une page web, la segmentation dont un système de génération augmentée par récupération visuelle a besoin existe déjà. Le moteur de mise en page du navigateur a calculé la boîte englobante exacte de chaque élément avant qu’un seul pixel ne soit recadré. PixelRAG [40] établit qu’un modèle vision-langage peut récupérer directement sur des tuiles de pixels, mais les découpe à l’aveugle ; MAGE-RAG [48] établit qu’un contrôleur de graphe budgété surpasse une sélection statique, mais suppose les éléments du document déjà constitués en amont de son propre pipeline. Les deux laissent, chacun de son côté, la même pièce sur la table. Ce travail les assemble par l’artefact que le premier jette au rendu et que le second n’a jamais eu à construire, le DOM.

Transformer cette observation en système a demandé quatre composants, un tuilage adaptatif aligné sur les frontières d’éléments, un graphe de preuves multigranulaire s’étendant sur plusieurs documents, un lecteur vision-langage adapté par contraste à l’évaluation de pertinence, et un contrôleur budgété décidant à la requête quelles tuiles ouvrir. Le système est implémenté de bout en bout, testé, et évalué sur des pages web réelles plutôt que sur un corpus figé.

Travail établi

Deux résultats sont solidement étayés et constituent l’apport principal de ce projet.

Le tuilage guidé par le DOM rend plus de réponses attachables, pour moins de pixels. Sous notre critère d’attachement, sur dix-huit pages Wikipédia et contre une grille aveugle pleine largeur, la fraction des questions dont la réponse survit dans une unité récupérable passe de 26,4 % à 51,8 %. Ce n’est pas un gain de classement mais un gain de possibilité, car une réponse absente de toute unité est perdue quelle que soit la qualité du récupérateur en aval. C’est le résultat le plus robuste du rapport, l’écart de vingt-cinq points étant sans commune mesure avec la variation observée entre deux exécutions.

Et ce gain s’obtient à un coût inférieur. Notre tuilage produit cinq fois plus d’unités que la grille, mais chacune est treize fois plus petite, de sorte que le total des pixels par page est 2,7 fois moindre. L’origine de ce résultat est identifiée et formalisée précisément. La boîte serrée d’une balise de texte coulant, mesurée sur l’union des rectangles de ses lignes réellement rendues plutôt que sur sa boîte de bloc CSS pleine largeur, élimine une aire de fond blanc qu’un recadrage naïf aurait soumise en pure perte à l’encodeur visuel, et elle en constitue l’enseignement technique le plus transférable.

Ce second chiffre est plus fragile que le premier et doit être cité avec ses conditions. Il est mesuré à une largeur de rendu de 2048 px, quand PixelRAG rend ses bandes à 875 px ; à largeur égale il tomberait à environ 1,16, la marge blanche d’une bande pleine largeur étant facturée à la seule grille. Sa transposition au coût en tokens suppose de surcroît le plafond du processeur d’image relevé douze fois au-dessus du défaut, faute de quoi le gain retombe à 1,3. La courbe du gain en fonction de la largeur de rendu reste à mesurer. Sous ces réserves, et parce que le nombre de tokens visuels croît avec l’aire de l’image dans la famille de modèles utilisée, la trouvabilité et le coût s’améliorent ensemble.

L’adaptation contrastive du lecteur fonctionne nettement. Sur un partitionnement au niveau de l’article strictement disjoint, et en classant chaque question contre toutes les tuiles de sa page plutôt que contre le vivier miné, le Recall@1 passe de 0,433 à 0,785 et le MRR de 0,578 à 0,856.

La comparaison avec la référence lexicale porte un enseignement qui dépasse notre système. Le modèle vision-langage non adapté obtient un MRR de 0,578, pratiquement identique à celui d’une similarité cosinus TF-IDF (0,579), et un Recall@1 légèrement inférieur, mais un Recall@3 et un Recall@5 légèrement supérieurs, ce qui n’est pas un déficit général de récupération. Autrement dit, remplacer un scoreur lexical par un grand modèle multimodal non entraîné à la tâche de récupération n’apporte pas d’avantage net sur le rang de tête, qui est ce qui compte pour une sélection à un seul candidat. Les deux scoreurs ne lisent toutefois pas la même entrée, TF-IDF l’aperçu textuel et le modèle l’image, si bien que la fonction de score n’est pas la seule chose qui change entre les deux lignes. Tout le gain de Recall@1 vient de l’adaptation contrastive, sur des questions synthétiques issues de la même génération ; ce que cette adaptation a réellement appris, récupérer une preuve ou retrouver l’image d’où provient un texte, reste à trancher par le contrôle sur questions humaines qui n’a pas été exécuté.

Ces deux résultats partagent une même racine conceptuelle. Ce travail ne propose ni nouvelle architecture d’encodeur ni mécanisme d’attention inédit, et le lecteur affiné reste un Qwen2-VL générique. Ce qui change est où passe la frontière de recadrage. En alignant la grille de *patches* que consomme le modèle vision-langage sur les frontières d’éléments du DOM plutôt que sur une géométrie arbitraire, le tuilage garantit qu’une tuile porte un contenu visuel homogène, ce qui rend chaque pixel soumis à l’encodeur pertinent pour la question posée, aussi bien au moment de la récupération non entraînée qu’à celui de l’apprentissage contrastif, dont l’objectif n’a plus à démêler plusieurs sujets mélangés dans une même séquence de *patches*. Le gain de trouvabilité et la réduction du budget de pixels sont ainsi deux manifestations mesurées d’une seule décision de conception, prise avant que l’encodeur ne voie la première tuile ; l’apport de cet alignement à l’entraînement contrastif lui-même en est une troisième, plausible mais non mesurée, faute d’un entraînement identique conduit sur des tuiles de grille.

Verrou identifié

Les composants en aval n’atteignent pas encore le niveau des deux précédents, et le principal apport analytique de ce rapport est d’avoir localisé précisément pourquoi.

Le contrôleur budgété reste en dessous d’une sélection top- k statique dans les deux configurations mesurées, et à amorce égale au budget il lui est rigoureusement identique, c’est-à-dire qu’il n’apporte rien. Ce résultat a néanmoins produit un enchaînement méthodologique instructif. Une hypothèse causale falsifiable a été formulée, la frontière d’exploration ne franchissant jamais les limites de document parce que la relation d’ordre de lecture ne relie jamais deux pages, puis le correctif déduit de ce diagnostic a été implémenté et son effet mesuré. La couverture inter-pages est passée de 29,7 % à 39,6 %, réduisant l’écart avec la référence de dix points à un point. L’hypothèse ayant été formée en examinant ces mêmes questions, il ne s’agit pas d’une prédiction hors échantillon. Une cause plus immédiate a par ailleurs été identifiée après coup, l’égalité des deux seuils du contrôleur, qui rend l’élagage impossible hors amorce et ramène la politique à un top- k .

L’évaluation bout-en-bout a ensuite montré que le mode de défaillance dominant du système assemblé est l’abstention explicite, treize prédictions sur vingt, et non la fabulation. Le lecteur n’échoue pas à interpréter les preuves qu’on lui fournit, mais constate que celles-ci

sont insuffisantes. La défaillance est donc située dans la sélection, en amont. Cette abstention est toutefois prescrite par l’invite de synthèse elle-même, si bien qu’elle vérifie le respect d’une consigne plutôt qu’elle ne révèle un système bien calibré.

Quatre observations convergent alors vers le même composant. Le contrôleur n’atteint qu’une couverture complète sur un peu plus d’une question sur trois. Douze des quatre-vingt-onze questions ont une couverture nulle, ce qui pointe vers le score, sans qu’une hypothèse concurrente, l’absence de la phrase de support dans toute tuile, ait été écartée. Le système assemblé s’abstient parce que les preuves manquent. Enfin, le découpage textuel plat devance les tuiles DOM sur le sous-ensemble strictement apparié, le seul résultat contraire aux attentes du rapport : l’explication par une métrique aveugle à la structure est plausible mais n’a pas été testée, et la formulation exacte reste que le texte plat classe mieux sur ce corpus et sous ce scoreur, sans que la raison en soit connue.

Ce composant est le **score de pertinence du contrôleur**, qui reste un substitut lexical alors même que l’évaluation du lecteur a validé un scoreur bien supérieur sur exactement cette tâche. Le point d’extension qui permet la substitution est déjà en place dans l’implémentation ; la seule barrière est le coût de calcul, un passage avant du modèle par tuile candidate et par question contre un score lexical essentiellement gratuit.

À ce premier verrou s’en ajoutent deux autres, que le score ne recouvre pas. La politique de parcours n’emprunte que trois des sept relations construites, et l’adjacence de mise en page comme la hiérarchie de sections restent inexploitées quel que soit le scoreur employé, la fonction de voisinage ne les consultant pas. Les deux seuils du contrôleur sont par ailleurs égaux, ce qui neutralise l’élagage ; c’est le seul des trois correctifs qui ne demande aucun calcul sur GPU. Ce travail établit donc qu’un tel graphe se construit depuis le DOM, non qu’un contrôleur en tire déjà parti.

Il faut dire cela clairement, car c’est une position favorable pour la suite. Le système ne présente pas une défaillance diffuse dont on ignorerait l’origine, mais deux composants validés et trois verrous nommés, localisés, instrumentés, et dont les correctifs sont spécifiés séparément.

Enseignements méthodologiques

Trois enseignements de méthode dépassent le cas particulier de ce système et méritent d’être retenus.

Une métrique de récupération doit être invariante à la taille du vivier. Sur nos données, la grille aveugle atteint un Recall@5 de 0,991, un score presque parfait qui n’exprime rien d’autre que le fait qu’elle ne propose que cinq candidats par page. Rappporter des Recall bruts pour comparer des stratégies de découpage produit une conclusion inversée. Le rang percentile, dont l’espérance sous classement aléatoire vaut 0,5 quelle que soit la taille du vivier, est la correction nécessaire. Cette précaution nous paraît sous-estimée dans la littérature sur le découpage documentaire. Elle a sa propre limite, et la dire complète l’enseignement : le percentile corrige le biais en espérance sous classement aléatoire, mais ne dit rien sous budget absolu. Or notre contrôleur ouvre un nombre fixe de nœuds, et en rang absolu la grille devance le tuilage DOM. Les deux lectures sont nécessaires, et rapporter la seule qui avantage la méthode proposée serait le symétrique de l’erreur dénoncée ci-dessus.

Le taux d’attachement mérite d’être mesuré explicitement. La proportion de questions dont la réponse survit à l’étape de découpage détermine un plafond que le meilleur récupérateur du monde ne peut pas franchir, et c’est sur cette métrique que l’écart entre les

deux stratégies de tuilage est le plus grand. Elle est pourtant absente des protocoles usuels, qui se concentrent sur la qualité du classement en supposant l'attachement acquis.

Une évaluation par composant peut masquer un système qui ne fonctionne pas.

Deux de nos quatre évaluations sont franchement positives, et le système assemblé répond correctement à une question sur vingt. Sans l'évaluation bout-en-bout, le rapport aurait pu conclure sur deux succès. C'est elle qui a rendu lisible le mode de défaillance réel, et donc localisé le verrou. Nous retenons de cette expérience qu'une évaluation par composant est nécessaire mais jamais suffisante, et que les évaluations défavorables gagnent à être rapportées au même endroit que les autres, ce que la structure du chapitre 3 applique.

Perspectives

La suite immédiate est déterminée par le diagnostic. Il s'agit de brancher le lecteur validé dans le score de pertinence du contrôleur, à la place du substitut lexical, et de rejouer à l'identique les évaluations du contrôleur et du système assemblé. Une atténuation praticable du coût consiste à adopter l'architecture classique du classement en deux étapes, TF-IDF présélectionnant une courte liste que le lecteur affiné réordonne. La même substitution appliquée à l'évaluation du tuilage testerait directement si l'avantage observé du texte plat est un artefact du scoreur lexical.

À plus longue portée, deux directions nous paraissent les plus prometteuses. D'une part, faire réellement exploiter au contrôleur les relations du graphe qu'il n'interroge pas encore. Remonter au titre de section d'une tuile ouverte, ou atteindre l'en-tête d'un tableau dont une ligne a été retenue, sont des besoins de récupération concrets auxquels le graphe répond déjà structurellement. D'autre part, sortir du périmètre MediaWiki, ce qui permettrait de séparer empiriquement ce qui relève de la liste de sélecteurs de nettoyage, spécifique par construction, de ce qui relève de la logique de tuilage, qui ne l'est pas.

Conclusion

L'affirmation centrale de ce travail résiste à l'examen dans le périmètre où elle a été éprouvée, car le DOM contient bien la segmentation que le RAG visuel reconstruit autrement, et l'exploiter rend davantage de contenu trouvable, sur dix-huit pages Wikipédia et sous notre critère d'attachement, pour un coût en pixels moindre à la largeur de rendu employée. L'affirmation plus ambitieuse, celle qu'un graphe de preuves construit sur cette structure et parcouru par un contrôleur budgété produit un meilleur système de questions-réponses, n'est pas démontrée. Elle n'est pas réfutée non plus, car elle attend des expériences identifiées, dont les obstacles sont un budget de calcul et une politique de parcours à étendre, non une inconnue conceptuelle. Le pipeline complet est publié (annexe D), y compris les résultats qui ne sont pas allés dans le sens attendu, parce que la localisation du verrou est elle-même un résultat utilisable par qui reprendra ce travail.

Annexe A

Résultats complémentaires

Cette annexe rassemble ce que le chapitre 3 utilise sans le détailler, les contraintes de protocole de l'évaluation A (annexe A.1), les coûts de calcul observés (annexe A.2), les limites du travail (annexe A.3) et les directions qu'elles ouvrent (annexe A.4). Les quatre évaluations elles-mêmes, y compris celles dont le résultat est défavorable, sont rapportées dans le corps du texte.

A.1 Contraintes de protocole de l'évaluation A

Deux caractéristiques du protocole de l'évaluation A conditionnent la lecture des chiffres. **Un plafond de 25 tuiles DOM par page** (section 2.3) borne aussi l'évaluation, car aucune tuile au-delà n'a été présentée au modèle générateur et aucune question ne peut donc en dépendre. Sur 17 des 18 pages, le nombre de tuiles DOM est exactement 25 (figure A.1), si bien que la valeur de 24,0 tuiles par page est un plafond atteint et non une mesure de densité de tuilage réelle, qui serait supérieure. **Les trois conditions couvrent la même région de page**, car le protocole calcule une frontière verticale, le bas de la plus basse tuile DOM retenue, et y restreint la grille et les fenêtres de texte, de sorte que les comparaisons de taux d'attachement et de budget de pixels portent bien sur le même contenu.

A.2 Coûts de calcul

La génération du jeu de données coûte environ deux minutes par tuile en exécution locale, soit de l'ordre de soixante heures de calcul séquentiel sur 1 784 tuiles ; l'émission concurrente des appels au sein d'une page (section 2.7.4) et le cache sur disque (section 2.7.2) ramènent ce coût à un ordre praticable. L'entraînement contrastif de l'évaluation B a été mené sur une carte A100/H100 de 80 GB louée, lot de 4, deux négatifs difficiles par question, trois époques sur les 1 339 exemples contrastifs ; le *gradient checkpointing* (section 2.5.6) coûte 20 % à 30 % de temps supplémentaire.

C'est le **coût du scoreur appris à l'inférence** qui explique la principale expérience non menée, puisque substituer le lecteur affiné à TF-IDF dans le contrôleur impose un passage avant du modèle par tuile candidate et par question, là où un score lexical est essentiellement gratuit. Sur les 91 questions de l'évaluation C, chacune sur un graphe de plusieurs dizaines de tuiles, cela représente plusieurs milliers de passages avant sur GPU loué, et pour une seule configuration d'amorce. Ce coût, non une difficulté d'implémentation, est ce qui a placé l'expérience hors de portée du projet.

À l'inverse, le calcul propre au tuilage est négligeable, la complexité quadratique de l'élagage et du regroupement (section 2.3.3) ne se manifestant jamais aux tailles de page réelles. Ce n'est pas dire que la condition DOM soit gratuite. Elle exige de lancer un navigateur par page, d'attendre *networkidle*, de faire défiler le document et d'en capturer jusqu'à 46 000 px de hauteur, soit quelques secondes par page dominées par l'attente réseau, non mesurées ici de façon systématique. Elle impose surtout une dépendance au HTML vivant que la condition textuelle n'a pas, avec les conséquences de reproductibilité de la annexe E.5. Rapporté au coût des appels au modèle, ce surcoût reste faible, si bien que le résultat de la section 3.2.3 n'échange pas du temps de calcul contre des tokens économisés.

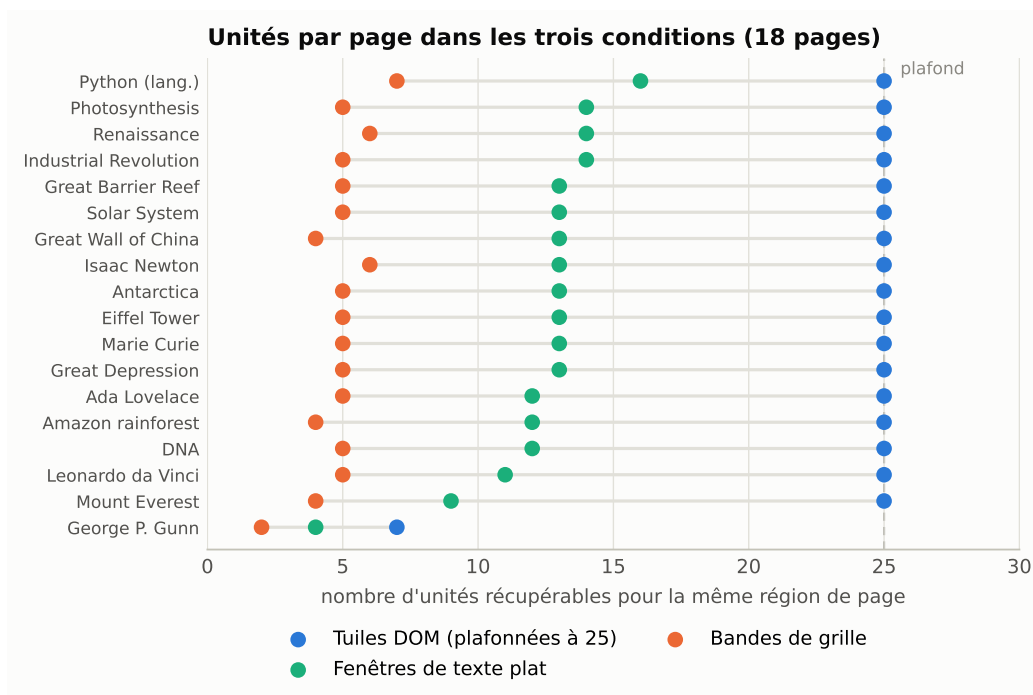


Figure A.1 – Nombre d’unités récupérables produites par chaque condition, page par page, pour la même région de page. Le plafond de 25 tuiles DOM est atteint sur 17 des 18 pages ; seule la page la plus courte (*George_P._Gunn*, 7 tuiles) reste en dessous. La grille aveugle produit systématiquement entre 2 et 7 bandes, soit un ordre de grandeur de moins.

A.3 Limites

Les trois limites du contrôleur sont indépendantes. Établies en section 3.6. Le scoreur reste lexical alors que le lecteur affiné de l’évaluation B est une fonction de similarité bien meilleure, et la cause de cette non-substitution est le coût de calcul (annexe A.2). La politique, elle, n’emprunte que trois des sept relations instanciées (tableau 2.2), de sorte que la richesse du graphe reste inexploitée quel que soit le scoreur employé. La troisième est la plus immédiate et la moins coûteuse à corriger : les deux seuils sont égaux (section 2.6.2), ce qui rend l’élagage impossible hors amorce et ramène la politique à un top- k , et les balayer ne demande aucun appel au modèle. Les trois correctifs sont distincts et peuvent être conduits séparément.

Confusion de famille de modèles. Les questions synthétiques et le lecteur affiné proviennent de la même famille Qwen, une contrainte d’infrastructure (section 2.5.3), Ollama n’exposant aucun accès aux poids pour l’affinage. Une partie du gain de l’évaluation B pourrait donc refléter les idiosyncrasies du générateur de questions plutôt qu’une amélioration transférable à des questions humaines. Un contrôle direct, la même mesure de récupération sur les questions humaines de HotpotQA, est implémenté mais n’a pas été exécuté faute de budget de calcul. Le seul indice disponible est la précision bout-en-bout de 5 % de l’évaluation D, nettement inférieure aux chiffres de l’évaluation B et cohérente avec un déficit de généralisation, sans le démontrer, car l’évaluation D change aussi la tâche (pipeline complet, questions inter-pages, mesure de réponse plutôt que de rang).

Le jeu contrastif ne couvre pas tout le corpus. Seize unités de page dont les paires question-réponse ont été générées ne figurent pas dans le jeu contrastif, leur minage de

négatifs n'ayant pas abouti lors de la construction. Ces unités portent 378 des 1 717 questions, soit 22 % du total pour 8,6 % des unités. Elles sont en moyenne deux fois et demie plus grandes que le reste du corpus, si bien que la perte frappe préférentiellement les pages longues, celles-là mêmes que ce travail vise. Le lecteur de l'évaluation B est donc affiné sur un corpus biaisé vers le court, et la proportion 80/20 annoncée en section 2.5.7 porte sur ce sous-ensemble. La cause n'est pas une contrainte mais un défaut, une reprise sur échec qui ne couvre pas les étapes postérieures à la génération des questions (section 2.8). Régénérer le minage sur ces unités serait peu coûteux, le scoreur étant lexical, mais imposerait de réaffiner l'adaptateur et de rejouer l'évaluation B.

Portée limitée aux pages sémantiquement balisées. Tous les résultats quantitatifs portent sur Wikipédia, avec une liste de sélecteurs de nettoyage propre à MediaWiki (section 2.2) ; aucune vérification, même qualitative, n'a été menée hors MediaWiki. La liste de sélecteurs est spécifique ; la logique de tuilage, avec ses boîtes englobantes, sa mesure serrée, ses fusions, ses découpes et sa résolution des chevauchements, ne l'est pas et ne connaît rien de MediaWiki. L'approche n'est pas non plus revendiquée pour les applications monopages hydratées par JavaScript ni les mises en page en soupe de div, le régime que les agents web [9, 46, 47] étudient déjà côté action.

Le plafond de tuiles restreint la portée de l'évaluation A. Le plafond de 25 tuiles (annexe A.1) borne l'évaluation à la portion supérieure des pages longues. Cette portion n'est pas neutre, et elle joue en faveur du tuilage : les infoboîtes flottantes et les introductions y sont concentrées, or l'infoboîte flottante est précisément le cas pour lequel la mesure serrée de la section 2.2.4 est conçue. Le biais est donc favorable et non quantifié, et le lever demanderait de rejouer l'évaluation sur la moitié basse des pages. La valeur de 24,0 tuiles par page est de même un plafond atteint et non une mesure de densité, car sans lui le rapport d'environ cinq entre tuiles DOM et bandes de grille serait plus élevé, dans une direction non mesurée pour le rapport de pixels. Lever ce plafond exigerait de régénérer les paires question-réponse sur l'ensemble du corpus, donc de répéter le coût de l'annexe A.2.

Tailles d'échantillon, et taille du corpus. Dix-huit pages et 409 questions pour l'évaluation A. 330 questions de validation pour l'évaluation B, la plus solide des quatre. 91 questions pour l'évaluation C, sur lesquelles un test de McNemar apparié ne conclut qu'à une indiscernabilité statistique. 20 questions pour l'évaluation D, qui ne supportent qu'une lecture qualitative du mode de défaillance, et qui sont un sous-ensemble des 91 précédentes. La conséquence à en tirer est plus simple que cette énumération, car le corpus entier tient en dix-neuf articles d'un seul site, et les quatre évaluations sont des variations sur ces mêmes pages. C'est cela, plus que le nombre de questions, qui borne ce que ce travail démontre.

Autres limites. La découpe des éléments démesurés (section 2.3.3) reste indifférente au contenu à l'intérieur d'un seul élément DOM, une version étroite du défaut reproché à la grille. Le parcours de corpus est peu profond (un saut, au plus trois tuiles porteuses de liens, section 2.4.5) ; il convient à HotpotQA, pas à une chaîne de trois sauts ou davantage. Le minage de négatifs reste mono-page hors du chemin HotpotQA. Tous les hyperparamètres de l'annexe B sont des valeurs uniques justifiées mais non balayées, faute de budget de calcul. La relation `semantic_neighbor` et l'ablation de résolution (section 2.8.4) sont implémentées mais non évaluées.

A.4 Directions de travail

Les directions suivantes sont ordonnées par bénéfice sur coût. Les cinq premières ne demandent aucun GPU.

(1) Exécuter le contrôle de généralisation déjà écrit (annexe A.3), une évaluation sur 91 questions et non un entraînement. C’est la vérification la plus décisive du rapport, puisqu’elle seule sépare, dans le gain de l’évaluation B, ce qui est de la récupération de ce qui est un appariement à la source des questions. (2) Balayer les deux seuils du contrôleur, dont l’égalité neutralise l’élagage (section 2.6.2) ; la simulation ne demande aucun appel au modèle. (3) Rejouer le budget de pixels à la largeur de rendu de 875 px et tracer le gain en fonction de la largeur, ce qui rendrait le rapport de 2,7 indépendant d’une constante de rendu (section 2.2). (4) Contrôler à la main, sur une trentaine de questions, la correspondance entre le critère d’attachement (section 2.8.3) et le jugement humain, et mesurer le taux de SKIP des trois conditions sur le même corpus. (5) Annoter l’ordre de lecture de quelques pages pour donner un taux d’erreur à l’heuristique de la section 2.4.1, la seule relation parcourue à l’intérieur d’une page, et vérifier sur les douze questions à couverture nulle de l’évaluation C si la phrase de support figure dans une tuile.

(6) Brancher le lecteur affiné dans le score de pertinence du contrôleur, dont le point d’extension existe (section 2.6.4) et dont seule l’exécution est coûteuse ; une atténuation praticable serait un classement en deux étapes, TF-IDF présélectionnant une courte liste que le lecteur affiné réordonne. (7) Rejouer l’évaluation A avec ce même scoreur, pour trancher si l’avantage du texte plat est un artefact du scoreur ou un résultat robuste. (8) Mesurer ce que l’alignement DOM apporte à l’entraînement lui-même, en menant un entraînement contrastif identique sur des tuiles de grille (section 2.5.3). (9) Faire exploiter au contrôleur `section_hierarchy` et `layout_adjacency`, aujourd’hui jamais interrogées, et conduire l’ablation de `semantic_neighbor`. (10) Miner les négatifs difficiles avec le lecteur lui-même une fois un premier tour d’affinage effectué. (11) Évaluer le tuilage hors MediaWiki, sur quelques pages où la mise en page porte l’information, un rapport financier ou une notice technique, ce qui séparerait à la fois le nettoyage de la logique de tuilage et le gain structurel du gain lexical. (12) Étendre l’évaluation D aux soixante-onze questions restantes de l’évaluation C, une fois triée la cause de l’écart entre couverture de preuves et exactitude bout-en-bout.

Annexe B

Hyperparamètres

Toutes les valeurs référencées dans le chapitre 2 sont rassemblées ici. Chacune est une valeur unique, documentée et centralisée dans un module de configuration ; aucune n’a fait l’objet d’un balayage (annexe A.3). Les aires sont exprimées en pixels carrés, les longueurs en pixels.

Trois d’entre elles ne sont pas des réglages neutres, car elles décident directement de chiffres annoncés dans le chapitre 3. La largeur de rendu de 2048px fixe le rapport de budget de pixels de 2,7 (section 2.2). Le plafond `max_pixels` fait passer ce rapport de 1,3 à 2,7 lorsqu’il est transposé au coût en tokens (section 2.3.4). Et l’égalité des deux seuils du contrôleur neutralise l’élagage (section 2.6.2). Les deux premières demanderaient un nouveau rendu pour être balayées ; la troisième ne coûte rien.

Tableau B.1 – Hyperparamètres de la partie hors ligne de la chaîne, extraits du module de configuration.

Paramètre	Valeur	Référence
<i>Rendu et extraction</i>		
Largeur du <i>viewport</i> de rendu	2 048 px	section 2.2
Hauteur du <i>viewport</i> (sans effet, capture pleine page)	1 000 px	section 2.2
Facteur d'échelle de périphérique	1	section 2.2
Condition d'attente avant capture	networkidle	section 2.2
<i>Tuilage</i>		
Hauteur maximale de tuile (avant découpe)	2 048 px	section 2.3.3
Seuil de mise en tampon (hauteur d'élément)	80 px	section 2.3.3
Tolérance d'imbrication (élagage)	1 px	section 2.3.1
Seuil de chevauchement $\rho_{\min}$	0,15	section 2.3.2
Hauteur de page avant découpage en sections	20 480 px ($10 \times h_{\max}$)	section 2.3.5
Budget de hauteur d'une section trop haute	20 480 px (même seuil que la ligne précédente)	section 2.3.5
Tuiles retenues par unité de page (plafond)	25	section 2.3
<i>Graphe</i>		
Tolérance de regroupement en rangées	20 px	section 2.4.1
Tuiles porteuses de liens conservées (k_{liens})	3	section 2.4.5
Pages suivies depuis l'amorce (m)	3	section 2.4.6
Seuil de voisinage sémantique (cosinus)	0,2	section 2.4.4
Voisins sémantiques maximum par tuile	3	section 2.4.4
<i>Scoreur lexical TF-IDF</i>		
Plage de n -grammes	1–1 (unigrammes)	section 2.6.4
Liste de mots vides	"english" (scikit-learn)	section 2.6.4
Fréquences documentaires minimale et maximale	1 / 1,0 (pas de borne supérieure)	section 2.6.4
Pondération sous-linéaire des fréquences	non	section 2.6.4
Longueur de l'aperçu textuel par tuile	200 caractères	section 2.3

Tableau B.2 – Hyperparamètres de l’entraînement, du contrôleur en ligne et des conditions de référence.

Paramètre	Valeur	Référence
<i>Entraînement contrastif</i>		
Négatifs difficiles minés par question (K)	2	section 2.5.2
Rang LoRA (r)	16	section 2.5.4
Facteur d’échelle LoRA (α)	32	section 2.5.4
Dropout LoRA	0,05	section 2.5.4
Paramètres entraînables	44 302 336 (0,63 % du total)	section 2.5.4
Optimiseur, taux d’apprentissage, échauffement	AdamW, 1×10^{-4} , aucun	section 2.5.4
Température InfoNCE (τ)	0,05	section 2.5.5
Taille de lot (B)	4	section 2.5.5
Époques	3	annexe A.2
Proportion du partitionnement (par article)	80/20	section 2.5.7
<i>Contrôleur</i>		
Budget d’ouvertures (B)	5	section 2.6.2
Amorce (k_{seed})	3	section 2.6.2
Seuil d’ouverture (θ_{ouv})	0,05	section 2.6.2
Seuil d’activation (θ_{act})	0,05	section 2.6.2
Appels au modèle concurrents	4	section 2.7.4
<i>Modèles et décodage</i>		
Modèle lecteur (inférence)	qwen3-v1 (Ollama)	section 2.7
Modèle lecteur (affinage)	Qwen2-VL-7B-Instruct	section 2.5.3
Aire d’image minimale du processeur (<code>min_pixels</code>)	3 136 px ²	section 2.5.6
Aire d’image maximale du processeur (<code>max_pixels</code>)	12 845 056 px ²	tableau 2.1
Température de décodage	0 (glouton)	section 2.7.2
Fenêtre de contexte	8 192 tokens	section 2.7.2
<i>Références et ablations</i>		
Seuil de découpage en sections, propre à l’évaluation A	51 200 px ($25 \times h_{\text{max}}$, contre $10 \times$ pour le corpus principal)	section 2.8.2
Hauteur de bande de la grille aveugle	1 024 px	section 2.8.2
Taille de fenêtre du texte plat	400 caractères	section 2.8.2
Largeur de bande de la grille aveugle	2 048 px (largeur de rendu)	section 2.8.2
Facteur d’échelle de compression (Lanczos)	0,5 59	section 2.8.4
Graine du tirage aléatoire, sous-échantillon	0	section 3.5

Annexe C

Invites utilisées

Les invites sont reproduites telles qu'elles figurent dans le module de configuration. Celles adressées au modèle générateur de questions sont en français, celles de la synthèse finale en anglais, la langue des articles Wikipédia évalués et des réponses de référence de HotpotQA.

C.1 Génération de paires question-réponse

Cette invite accompagne l'image d'une unique tuile (section 2.5.1).

```
1 Tu vois un unique fragment recadré (une "tuile") d'une capture d'écran de
2 page web, pas la page complète.
3
4 Génère UNE question précise et autonome dont la réponse est visible dans
5 cette image, et UNIQUEMENT dans cette image (quelqu'un qui n'a pas vu cette
6 tuile ne pourrait pas deviner la réponse par hasard). Évite les questions
7 vagues ("de quoi parle cette image ?") : vise une question factuelle et
8 spécifique (un nombre, un nom, une date, une définition, une légende...).
9
10 Si l'image ne contient aucune information exploitable (décorative, vide,
11 illisible), réponds exactement: SKIP
12
13 Sinon, réponds STRICTEMENT en JSON, sans texte autour, au format :
14 {"question": "...", "answer": "..."}

```

Trois propriétés de cette invite portent le protocole. L'exigence d'autonomie de la question rend la paire utilisable comme exemple positif de récupération, car une question dont la réponse serait devinable sans voir la tuile n'apprendrait rien au modèle. La sortie SKIP explicite évite d'avoir à interpréter un refus formulé librement. Et le format JSON strict, sans texte d'encadrement, rend l'analyse syntaxique déterministe.

C.2 Instruction d'encodage PromptEOL

Cette instruction suit l'entrée, question textuelle ou image de tuile indifféremment, et l'état caché de la dernière couche à la position de son dernier token est pris comme plongement (section 2.5.3).

```
1 Résume ce qui précède en un mot :

```

Son rôle est de contraindre le modèle à comprimer le sens de l'entrée en une position unique, et c'est son application identique aux deux modalités qui place question et tuile dans le même espace, sans tête de projection additionnelle.

C.3 Juge de faux négatifs

Ce second filtre, optionnel et non activé pour les jeux de données évalués, est adressé au modèle avec l'image d'une tuile candidate au statut de négatif difficile (section 2.5.2).

```
1 Tu vois un fragment ("tuile") d'une capture d'écran de page web.
2
3 Question : {question}
4 Réponse attendue (trouvée dans une AUTRE tuile de la même page) : {answer}

```

```

5
6 Cette image permet-elle, à elle seule, de retrouver cette même information
7 -- même reformulée (synonyme), ou exprimée visuellement (couleur, logo,
8 forme) plutôt que textuellement ? Réponds STRICTEMENT par un seul mot :
9 OUI ou NON.

```

L'insistance sur les formulations visuelles est le point de cette invite, car c'est exactement ce que le filtre lexical par sous-chaîne ne peut pas détecter.

C.4 Lecture d'une tuile ouverte

Adressée au modèle pour chaque tuile que le contrôleur a ouverte (section 2.6.5).

```

1 Question : {query}
2 Extrait uniquement les informations de cette image utiles pour y répondre.

```

C.5 Synthèse de la réponse finale

Adressée au modèle en texte seul, une fois les preuves rassemblées.

```

1 Here are excerpts from a web page, selected as relevant evidence to answer
2 a question.
3
4 Evidence:
5 {evidence}
6
7 Question: {question}
8
9 Answer the question using ONLY this evidence, in English. If the evidence
10 does not allow you to answer, say so explicitly rather than making
11 something up.

```

La dernière phrase de cette invite est directement responsable du mode de défaillance observé en section 3.5.2, puisqu'elle instruit explicitement le modèle de s'abstenir plutôt que d'inventer. Les treize abstentions sur vingt prédictions sont donc un comportement conforme aux instructions face à des preuves incomplètes, et non une défaillance du lecteur. Cette abstention étant prescrite, elle ne peut pas être présentée ailleurs comme une propriété émergente ni comme une preuve que le système est bien calibré ; la section 3.5.2 en tire la conséquence.

Annexe D

Organisation du dépôt

Le code source est publié en accès ouvert à l'adresse [\[URL du dépôt\]](#), à la révision [\[identifiant de commit\]](#), sous licence [\[licence\]](#). Toutes les commandes de l'annexe E s'y rapportent.

Le dépôt est organisé avec un module par étage du pipeline. Cette correspondance directe entre les sections du chapitre 2 et les modules est délibérée, puisqu'elle permet de localiser sans ambiguïté l'implémentation de chaque décision de conception décrite.

D.1 Modules du pipeline

Tableau D.1 – Modules du pipeline et sections correspondantes.

Module	Rôle	Section
config.py	constantes centralisées, invites	annexe B
dom_extraction.py	rendu Playwright, extraction des boîtes englobantes, mesure serrée du texte coulant	section 2.2
tiling.py	élagage, regroupement des chevauchements, appariement, fusion, découpe	section 2.3
sectioning.py	découpage des pages démesurées aux frontières h1/h2	section 2.3.5
graph_builder.py	graphe à deux niveaux, sept relations, export XML	section 2.4
corpus_builder.py	fusion multi-pages, préfixage, points d'entrée	section 2.4.6
vlm_client.py	client vision-langage, cache sur disque, concurrence	section 2.7
qa_generation.py	génération synthétique de paires question-réponse	section 2.5.1
hard_negative_mining.py	minage TF-IDF, double filtre anti-faux-négatif	section 2.5.2
contrastive_dataset.py	jeu de données PyTorch, partitionnement par article	section 2.5.7
lora_finetune.py	tête PromptEOL, LoRA sur ViT et décodeur, perte InfoNCE	section 2.5
evidence_controller.py	machine à états, politique best-first, frontière inter-documents	section 2.6
multihop_qa.py	construction des exemples inter-pages depuis HotpotQA	section 2.8.5
image_compression.py	sous-échantillonnage Lanczos pour l'ablation de résolution	section 2.8.4
visualization.py	superpositions de tuiles, graphes interactifs	aucune

D.2 Scripts d’expérimentation

Tableau D.2 – Scripts d’expérimentation et évaluations correspondantes.

Script	Rôle	Évaluation
<code>build_dataset.py</code>	pilote le pipeline sur une liste de pages, sauvegarde par unité	section 3.1
<code>build_multihop_dataset.py</code>	construit les exemples HotpotQA, journalise les abandons	section 3.1
<code>grid_baseline.py</code>	référence en grille aveugle, indépendante de <code>tiling.py</code>	A
<code>dom_vs_grid_experiment.py</code>	construit les trois conditions et classe les questions	A
<code>eval_retrieval.py</code>	récupération sur la partie de validation, avec et sans adaptateur	B
<code>eval_retrieval_tfidf.py</code>	référence lexicale sur le même partitionnement	B
<code>eval_multihop_controller.py</code>	couverture inter-pages, contrôleur contre top- k	C
<code>eval_multihop_e2e.py</code>	pipeline complet, correspondance exacte et F_1	D
<code>eval_hotpotqa_reader.py</code>	contrôle de généralisation (<i>écrit, non exécuté</i>)	annexe A.3
<code>compress_dataset_images.py</code>	produit le dossier miroir en résolution réduite	section 2.8.4
<code>generate_paper_figures.py</code>	superpositions de tuilage des figures 1 et 2.3	aucune

D.3 Tests

Les tests suivent la séparation par dépendances décrite en section 2.7. Les tests de `tiling`, `graph_builder`, `sectioning`, `hard_negative_mining`, `multihop_qa`, `evidence_controller`, `corpus_builder` et `image_compression` s’exécutent sur des cas synthétiques, sans réseau, sans modèle et sans GPU. Les tests de `dom_extraction` s’appuient sur Playwright et vérifient l’extraction de bout en bout contre une page HTML fournie avec le dépôt, incluant un cas dédié à l’élément flottant partageant sa rangée avec du texte (section 2.2.4). La couverture de ces tests, exprimée en pourcentage de lignes exécutées, vaut **[couverture des tests]** ; la liste des modules testés ne dit rien de la proportion de code atteinte.

Annexe E

Protocole de reproduction

E.1 Installation

```
1 python3 -m venv .venv
2 source .venv/bin/activate
3 pip install -r requirements.txt
4 playwright install chromium
5
6 # Backend d'inférence (aucun GPU requis)
7 ollama pull qwen3-vl
```

E.2 Vérification de l'installation

Les tests sans dépendance lourde s'exécutent sur une page locale et ne nécessitent ni réseau ni modèle.

```
1 python3 -m pytest tests/ -x
2 python3 tests/test_pipeline_smoke.py
```

E.3 Reproduction des évaluations

```
1 # Construction du jeu de données (long : ~2 min par tuile en local,
2 # mis en cache sur disque ; reprise automatique par unité de page)
3 python3 -m scripts.build_dataset
4 python3 -m scripts.build_multihop_dataset
5
6 # Évaluation A -- tuilage guidé, grille aveugle, texte plat
7 python3 -m scripts.dom_vs_grid_experiment
8
9 # Évaluation B -- lecteur affiné (nécessite un GPU pour l'entraînement)
10 python3 -m src.lora_finetune --epochs 3 --batch-size 4 \
11 --gradient-checkpointing
12 python3 -m scripts.eval_retrieval
13 python3 -m scripts.eval_retrieval_tfidf
14
15 # Évaluation C -- contrôleur contre top-k statique
16 python3 -m scripts.eval_multihop_controller
17
18 # Évaluation D -- pipeline complet
19 python3 -m scripts.eval_multihop_e2e
```

E.4 Application de démonstration

```
1 uvicorn webapp.backend.main:app --reload
2 # puis ouvrir http://127.0.0.1:8000/
```

Chaque requête recalcule l'intégralité du pipeline sur la page indiquée, sans corpus préconstruit, et le temps de réponse va de quelques dizaines de secondes à plusieurs minutes selon la page et le budget du contrôleur (section 2.7).

E.5 Notes de reproductibilité

Trois points conditionnent la reproductibilité exacte des chiffres rapportés. Le **décodage doit être glouton** (température nulle, section 2.7.2), car une température positive rend les appels non reproductibles, avec un symptôme trompeur, la même question pouvant recevoir une réponse correcte puis une abstention sans aucune différence dans les tuiles ouvertes. La **fenêtre de contexte doit être portée à 8 192 tokens**, car à la valeur par défaut de 4 096 un raisonnement long l'épuise avant la réponse, produisant un contenu vide silencieusement interprété comme SKIP. Enfin, les résultats portent en grande partie sur des **pages web vivantes** ; Wikipédia évolue, et les taux de conservation des questions HotpotQA (section 3.1) dériveront avec le temps. Pour l'évaluation A, `scripts/pin_wikipedia_snapshot.py` fige les révisions des dix-huit pages retenues, chargées automatiquement par `dom_vs_grid_experiment.py` avec repli sur le contenu en direct ; les évaluations B, C et D n'en bénéficient pas encore.

La conséquence est à assumer telle quelle, car les évaluations B, C et D ne sont pas reproductibles à l'identique, puisqu'elles tournent sur des pages vivantes. Étendre le mécanisme de figeage existant est un travail de quelques heures ; à défaut, les dates de collecte et les identifiants de révision devraient au minimum être publiés avec le dépôt.

Vérification de stabilité. Une réexécution de l'évaluation A sur ce contenu figé (tableau E.1) reproduit le classement et les conclusions qualitatives du tableau 3.2, la grille gardant le meilleur R@1 mais le pire taux d'attachement, texte plat et tuilage DOM restant proches et stables.

Ce tableau se lit toutefois dans les deux sens. Le rang percentile de la grille se déplace de 0,015 entre les deux exécutions, quand l'avantage de classement revendiqué pour le tuilage DOM sur la grille vaut 0,029 : le bruit d'exécution atteint la moitié de l'effet, ce qui est un argument contre le résultat de classement plutôt qu'une confirmation, et c'est pourquoi la section 3.2.4 ne le présente pas comme établi. L'écart de taux d'attachement, lui, vaut vingt-cinq points contre une variation maximale de 0,7 point entre exécutions ; là, le bruit est sans commune mesure avec l'effet, ce qui renforce le résultat principal du rapport.

Tableau E.1 – Évaluation A, chiffres du rapport (tableau 3.2, contenu Wikipédia en direct au moment du *run*) contre une réexécution sur le contenu figé par `pin_wikipedia_snapshot.py`. Mêmes dix-huit pages, mêmes 409 questions ; le classement entre conditions et l'écart d'attachement sont préservés.

	<i>n</i>	R@1	Attachement	%rang
<i>Tuiles DOM</i>				
Rapport	212	0,547	51,8 %	0,868
Réexécution, contenu figé	211	0,550	51,6 %	0,869
<i>Grille aveugle</i>				
Rapport	108	0,722	26,4 %	0,839
Réexécution, contenu figé	105	0,762	25,7 %	0,854
<i>Texte plat</i>				
Rapport	232	0,612	56,7 %	0,867
Réexécution, contenu figé	231	0,602	56,5 %	0,864

Considérations éthiques

F.1 Accès aux données

Le constructeur de corpus télécharge une page d’amorce et un nombre borné de pages liées, avec temporisation et reprise plutôt que téléchargement agressif. Un déploiement en production devrait néanmoins préférer l’API REST de Wikimedia ou un *dump* statique au téléchargement en direct ; le prototype ne le fait pas par défaut, et nous le signalons comme une limite de son mode de fonctionnement plutôt que comme un choix défendable à grande échelle.

F.2 Licences

Le texte de Wikipédia est publié sous licence CC BY-SA. Les images qui y sont incorporées portent en revanche des licences hétérogènes, dont certaines non libres. Les captures et les tuiles produites par le pipeline sont donc des œuvres dérivées que nous ne redistribuons pas ; seul le code est publié.

F.3 Statut des données synthétiques

Les questions générées en section 2.5.1 sont produites par un modèle et n’ont pas été vérifiées par un humain. Elles doivent être étiquetées comme telles en cas de diffusion. Cette réserve n’est pas formelle, car une question mal formée ou dont la réponse est inexacte introduit du bruit dans l’entraînement contrastif, et le taux de telles questions n’a pas été mesuré.

F.4 Aptitude au déploiement

Le mode de défaillance observé en section 3.5.2 est rapporté pour la sécurité du déploiement autant que pour la complétude. L’erreur dominante est l’abstention explicite, la plus sûre des deux pour un système de cette précision. Mais deux prédictions sur vingt étaient une réponse oui/non assurée et fausse (tableau 3.6), le mode de défaillance le plus susceptible d’induire en erreur un utilisateur qui ne vérifierait pas une affirmation factuelle brève, puisqu’un « non » concis ne signale rien de son incertitude. Deux cas sur cet échantillon ne permettent pas d’estimer un taux, mais suffisent à établir que le mode existe et n’est pas détectable par l’utilisateur sans vérification externe.

Nous ne considérons donc pas le système apte au déploiement là où une réponse oui/non erronée porterait des conséquences réelles, et nous rapportons ces résultats précisément pour que cette décision ne se prenne pas sur la seule motivation de la conception.

Bibliographie

- [1] Akari Asai, Zequi Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG : Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.
- [2] Luiz Bonifacio, Hugo Abonizio, Marzieh Fadaee, and Rodrigo Nogueira. Inpars : Data augmentation for information retrieval using large language models, 2022.
- [3] Deng Cai, Shipeng Yu, Ji-Rong Wen, and Wei-Ying Ma. VIPS : a vision-based page segmentation algorithm. Technical Report MSR-TR-2003-79, Microsoft Research, 2003.
- [4] Deng Cai, Shipeng Yu, Ji-Rong Wen, and Wei-Ying Ma. Block-based web search. In *Proceedings of the 27th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pages 456–463, 2004.
- [5] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [6] Xingyu Chen, Zihan Zhao, Lu Chen, Danyang Zhang, Jiabao Ji, Ao Luo, Yuxuan Xiong, and Kai Yu. WebSRC : A dataset for web-based structural reading comprehension. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- [7] Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B. Hall, and Ming-Wei Chang. Promptagator : Few-shot dense retrieval from 8 examples, 2022.
- [8] Chao Deng, Jiale Yuan, Pi Bu, Peijie Wang, Zhong-Zhi Li, Jian Xu, Xiao-Hui Li, Yuan Gao, Jun Song, Bo Zheng, and Cheng-Lin Liu. LongDocURL : a comprehensive multi-modal long document benchmark integrating understanding, reasoning, and locating. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1 : Long Papers)*, pages 1135–1159, Vienna, Austria, July 2025. Association for Computational Linguistics. doi : 10.18653/v1/2025.acl-long.57. URL <https://aclanthology.org/2025.acl-long.57/>.
- [9] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web : Towards a generalist agent for the web, 2023. NeurIPS 2023.
- [10] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA : Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, 2023.
- [11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words : Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

- [12] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global : A graph RAG approach to query-focused summarization, 2024. Microsoft.
- [13] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. Colpali : Efficient document retrieval with vision language models, 2024.
- [14] Alexis Firome. DOM-Graph RAG : DOM-guided adaptive tiling and budgeted online evidence control for multimodal web-page RAG, 2026. Version courte soumise en double aveugle à l’atelier DocInsights 2026 (EMNLP 2026). Le présent rapport en constitue la version longue.
- [15] Tianyu Gao, Xingcheng Yao, and Danqi Chen. SimCSE : Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021.
- [16] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models : A survey, 2023.
- [17] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora : Low-rank adaptation of large language models, 2021.
- [18] Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. LayoutLMv3 : Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia (ACM MM)*, 2022.
- [19] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2021.
- [20] Ting Jiang, Shaohan Huang, Zhongzhi Luan, Deqing Wang, and Fuzhen Zhuang. Scaling sentence embeddings with large language models. In *Findings of the Association for Computational Linguistics : EMNLP 2024*, pages 3182–3196, 2024.
- [21] Zhengbao Jiang, Frank F. Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [22] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.
- [23] Omar Khattab and Matei Zaharia. ColBERT : Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2020.
- [24] Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. OCR-free document understanding transformer. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.

- [25] Kenton Lee, Mandar Joshi, Iulia Turc, Hexiang Hu, Fangyu Liu, Julian Eisenschlos, Urvashi Khandelwal, Peter Shaw, Ming-Wei Chang, and Kristina Toutanova. Pix2Struct : Screenshot parsing as pretraining for visual language understanding. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [26] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020.
- [27] Junlong Li, Yiheng Xu, Lei Cui, and Furu Wei. MarkupLM : Pre-training of text and markup language for visually-rich document understanding. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.
- [28] Xueguang Ma, Sheng-Chieh Lin, Minghan Li, Wenhui Chen, and Jimmy Lin. Unifying multimodal retrieval via document screenshot embedding. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024.
- [29] Yubo Ma, Yuhang Zang, Liangyu Chen, Meiqi Chen, Yizhu Jiao, Xinze Li, Xinyuan Lu, Ziyu Liu, Yan Ma, Xiaoyi Dong, Pan Zhang, Liangming Pan, Yu-Gang Jiang, Jiaqi Wang, Yixin Cao, and Aixin Sun. Mmlongbench-doc : Benchmarking long-context document understanding with visualizations, 2024.
- [30] Renyi Qu, Ruixuan Tu, and Forrest Sheng Bao. Is semantic chunking worth the computational cost ? In Luis Chiruzzo, Alan Ritter, and Lu Wang, editors, *Findings of the Association for Computational Linguistics : NAACL 2025*, pages 2155–2177, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. doi : 10.18653/v1/2025.findings-naacl.114. URL <https://aclanthology.org/2025.findings-naacl.114/>.
- [31] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021.
- [32] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. SQuAD : 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016.
- [33] Nils Reimers and Iryna Gurevych. Sentence-BERT : Sentence embeddings using siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.
- [34] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework : BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4) :333–389, 2009.
- [35] Nandan Thakur, Nils Reimers, Andreas Rüchlé, Abhishek Srivastava, and Iryna Gurevych. BEIR : A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Advances in Neural Information Processing Systems 34 (NeurIPS), Datasets and Benchmarks Track*, 2021.
- [36] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step

- questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [37] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding, 2018.
 - [38] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017.
 - [39] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL : Enhancing vision-language model’s perception of the world at any resolution, 2024.
 - [40] Yichuan Wang, Zhifei Li, Zirui Wang, Paul Teietche, Lesheng Jin, Matei Zaharia, Joseph E. Gonzalez, and Sewon Min. Pixelrag : Web screenshots beat text for retrieval-augmented generation, 2026. UC Berkeley / BAIR / Berkeley NLP. Code : github.com/StarTrail-org/PixelRAG.
 - [41] Zilong Wang, Yiheng Xu, Lei Cui, Jingbo Shang, and Furu Wei. LayoutReader : Pre-training of text and layout for reading order detection. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 4735–4744, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi : 10.18653/v1/2021.emnlp-main.389. URL <https://aclanthology.org/2021.emnlp-main.389/>.
 - [42] Yang Xu, Yiheng Xu, Tengchao Lv, Lei Cui, Furu Wei, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, Wanxiang Che, Min Zhang, and Lidong Zhou. LayoutLMv2 : Multi-modal pre-training for visually-rich document understanding. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.
 - [43] Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. LayoutLM : Pre-training of text and layout for document image understanding. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2020.
 - [44] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W. Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA : A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018.
 - [45] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct : Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
 - [46] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. GPT-4V(ision) is a generalist web agent, if grounded, 2024. ICML 2024.
 - [47] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena : A realistic web environment for building autonomous agents, 2023. ICLR 2024.

- [48] Yilong Zuo, Xunkai Li, Jing Yuan, Qiangqiang Dai, Hongchao Qin, and Ronghua Li. MAGE-RAG : Multigranular adaptive graph evidence for agentic multimodal RAG in long-document QA, 2026. Code : github.com/laonuo2004/MAGE-RAG.